

AI-Powered Legal Document Analysis System:
Clause Detection, Category-Level Risk Prioritization
and Plain-Language Explanation on CUAD

by

Jerin Aktar

22101279

Afnan Mazumder

24141229

Shoyeb Hasan Sayem

22101386

A project submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
July 2026

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The project submitted is our own original work while completing degree at Brac University.
2. The project does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The project does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Jerin Aktar

22101279

Afnan Mazumder

24141229

Shoyeb Hasan Sayem

22101386

Approval

The project titled “AI-Powered Legal Document Analysis System: Clause Detection, Category-Level Risk Prioritization and Plain-Language Explanation on CUAD” submitted by

1. Jerin Aktar (22101279)
2. Afnan Mazumder (24141229)
3. Shoyeb Hasan Sayem (22101386)

of Summer, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in July 2026.

Examining Committee:

Supervisor:
(Member)

Utsha Kumar Roy

Designation
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Ethics Statement

This project uses the Contract Understanding Atticus Dataset (CUAD), a public research dataset of 510 business contracts taken from public filings and labelled by legal experts for The Atticus Project. We did not collect or use any private, confidential or personal documents, and no people took part as research subjects. All experiments ran on our own computers. The outside models we fine-tuned (DistilBERT, FLAN-T5) and all the software libraries are used under their open-source licences, and every outside source is cited.

The system is a research prototype meant to help with contract review. It is not legal advice, and the app says so. The ethical risks of relying on its output (false comfort from missed clauses, too much trust by non-lawyers, rules that differ between countries, whose side the risk is judged from, and privacy in real use) are discussed in Section 3.4.

Abstract

This project builds and tests a system of three models for reviewing contracts automatically, using CUAD (510 contracts, 41 clause categories) and a held-out test set made of 20% of the contracts. The system finds which clause types a contract contains, extracts the clause text, gives each clause type a High, Medium or Low risk level with a plain-English line, and runs as a working app. In our first setup, a DistilBERT model that reads the contract in windows lost to a simple TF-IDF model that reads the whole document (micro-F1 0.692 against 0.775), and the gap survived changes to the training data, seed, pooling rule and threshold. We then found that the model read only about half of each window. Once it reads the whole window and both models are tuned on the same validation split, the transformer ties the tuned baseline (0.797 against 0.785), an ensemble of the two beats it (0.809, with a 95% interval for the lead of $[+0.011, +0.037]$), and a recall-first setting finds 90.3% of High-risk clauses, against 63.1% for the first setup’s best ensemble. The fine-tuned summarizer outputs one of 41 sentences we wrote ourselves, word for word, for every input, and scores 0.116 against clause-specific references. A calibration map fitted on the held-out contracts lowers the confidence score’s calibration error from 0.206 to 0.016. Over the whole system only 22.1% of real clauses first got through all three steps: the presence model found the right window 82.6% of the time, but the span model, trained with the answer always near the start of its input, then succeeded in only 28.0% of those cases. Retrained on the chunks it really reads, it reaches a token-F1 of 0.779 on any window that contains the clause and succeeds in 76.4% of those cases, and 59.6% of real clauses now get through.

Keywords: legal NLP; CUAD; clause extraction; category-level risk prioritization; contract summarization; held-out evaluation

Acknowledgement

First of all, all praise to the Almighty for allowing us to complete this project without any major problem. We thank our supervisor, Utsha Kumar Roy, for his guidance and patience in every part of this work, and especially for insisting on a strict held-out test: that one rule shaped how this report presents its results. Finally, we thank our families for their constant support. Without them, this work would not have been possible.

Table of Contents

Declaration	i
Approval	ii
Ethics Statement	iii
Abstract	iv
Acknowledgement	v
Table of Contents	vi
List of Figures	x
List of Tables	xi
Nomenclature	xii
1 Introduction	1
1.1 Background	1
1.1.1 System at a Glance	1
1.2 Rationale of the Study or Motivation	1
1.3 Problem Statement	2
1.3.1 Research Questions	3
1.3.2 Contributions	4
1.4 Objective	4
1.5 Methodology in Brief	5
1.6 Scopes and Challenges	5
1.7 Report Organization	6
2 Literature Review	7

2.1	Preliminaries	7
2.1.1	Transformer Encoders and Model Distillation	7
2.1.2	Long-Document Strategies and Multiple-Instance Learning . .	7
2.1.3	Text-to-Text Models for Summarization	8
2.2	Review of Existing Research	9
2.2.1	Legal NLP and the CUAD Benchmark	9
2.2.2	Other Contract Datasets and Benchmarks	9
2.2.3	Large Language Models and Contract Review	10
2.2.4	Legal Risk Assessment and Decision Support	10
2.2.5	Plain-Language Legal Communication	11
2.2.6	Faithfulness and Hallucination in Generated Summaries . . .	12
2.3	Summary of Key Findings	12
3	Requirements, Impacts and Constraints	14
3.1	Final Specifications and Requirements	14
3.1.1	Functional requirements	14
3.1.2	Non-functional requirements	14
3.1.3	Data and resource requirements	15
3.1.4	Design constraints	15
3.2	Societal Impact	16
3.3	Environmental Impact	16
3.4	Ethical Issues	17
3.4.1	Data Handling and Privacy	18
3.5	Standards	18
3.6	Project Management Plan	19
3.6.1	Schedule	19
3.6.2	Budget	19
3.6.3	Resource management	19
3.7	Risk Management	19
3.8	Economic Analysis	20
4	Proposed Methodology	23
4.1	Design Process or Methodology Overview	23
4.2	Preliminary Design or Design (Model) Specification	24
4.2.1	Stage 2A: Presence Classification	24

4.2.2	Stage 2B: Span Extraction	27
4.2.3	Stage 1: Plain-Language Summarization	28
4.2.4	Category-Level Risk Prioritization	28
4.2.5	Avoiding Data Leakage	30
4.3	Data Collection	30
4.3.1	Data Cleaning	31
4.3.2	Data Transformation	31
4.3.3	Data Integration	32
4.3.4	Data Reduction	32
4.3.5	Summary of Preprocessed Data	32
4.4	Implementation of Selected Design	34
4.4.1	How We Validated, and Where the Settings Came From . . .	35
4.4.2	Optimization Schedule	36
4.4.3	Training the Presence Classifier	37
4.4.4	Training the Span Extractor	38
4.4.5	Training the Summarizer	38
4.4.6	Engineering Notes: Training on Consumer Hardware	38
4.4.7	Measured: What Sparse Attention Would Have Cost	39
4.4.8	End-to-End Data Flow	40
4.4.9	The Clausify Application	41
5	Result Analysis	49
5.1	Performance Evaluation	49
5.1.1	Presence Classification	50
5.1.2	Span Extraction	53
5.1.3	Summarization	54
5.2	Analysis of Design Solutions	56
5.2.1	End-to-End: What Actually Reaches the User	57
5.2.2	Risk-Weighted Evaluation: Does the Metric Match the Purpose?	60
5.2.3	What the Errors Actually Look Like	63
5.2.4	Does the AND Rule Hurt Rare Categories More Than Common Ones?	64
5.3	Final Design Adjustments	66
5.3.1	Was Max-Pooling the Right Way to Combine Window Scores?	66

5.3.2	The Threshold and What the Confidence Score Means	67
5.3.3	Reading the Whole Window, and Tuning Both Models the Same Way	69
5.4	Statistical Analysis	72
5.4.1	How Much of That Ranking Is Real?	73
5.4.2	How Much Changes If We Only Change the Random Seed? .	74
5.4.3	Was the Transformer Simply Undertrained?	75
5.4.4	A Third Source of Variation: The Split Itself	76
5.5	Comparisons and Relationships	77
5.6	Discussions	78
5.6.1	Usability	79
5.6.2	Honest Limitations of the Demo	79
6	Conclusion	82
6.1	Summary of Findings	82
6.2	Contributions to the Field	83
6.3	Threats to Validity	84
6.3.1	Internal Validity: Could Our Own Method Have Produced These Numbers?	85
6.3.2	Construct Validity: Do Our Scores Measure What We Claim?	85
6.3.3	External Validity: Where Do These Results Apply?	86
6.3.4	Statistical Conclusion Validity: Can the Intervals Be Trusted?	86
6.4	Limitations	87
6.5	Recommendations for Future Work	88
	Bibliography	90
A	Full Risk Taxonomy	93
B	Reproducibility Checklist	95
C	Repository and Artifact Layout	99

List of Figures

1.1	Three questions the system answers	2
4.1	System pipeline	23
4.2	Retrieval ceiling	25
4.3	Sliding-window generation	26
4.4	Multiple-instance max-pooling	26
4.5	Ensemble rules	27
4.6	Presence vs. focus window	28
4.7	Risk-level distribution	30
4.8	Per-category split balance	34
4.9	Learning-rate schedules	37
4.10	Presence classifier loss	38
4.11	Summarizer loss	39
4.12	Pipeline data flow	41
4.13	Clausify input screen	42
4.14	Highlighted contract and checklist	43
4.15	Clausify results view	44
4.16	High-risk clause cards	45
5.1	Presence model micro-F1	50
5.2	Confusion matrix heatmap	51
5.3	Top-12 per-category F1	52
5.4	Threshold and calibration	68

List of Tables

2.1	Related work	13
3.1	Resource requirements	15
3.2	Project schedule	20
3.3	Risk management plan	21
3.4	Cost–benefit comparison	22
4.1	Retrieval hit-rates	25
4.2	Train/test split	33
4.3	Training configurations	34
4.4	Validation arrangements	36
4.5	Longformer vs. DistilBERT cost	40
4.6	Application latency	48
5.1	Presence model comparison	50
5.2	AND-ensemble confusion matrix	51
5.3	Confusion-matrix metrics	51
5.4	Categories with zero F1	52
5.5	Span extraction results	53
5.6	Summarizer ROUGE-L	54
5.7	Fine-tuned vs. zero-shot output	55
5.8	End-to-end clause survival	57
5.9	Span-stage loss breakdown	58
5.10	First and retrained span model	59
5.11	Results by risk band	61
5.12	Results by risk band, re-run setups	62
5.13	Representative errors	63
5.14	Rare vs. common categories	65

5.15	Rare vs. common categories, re-run setups	65
5.16	Ways to combine window scores	66
5.17	Presence models after the re-run	70
5.18	Effect of reading the whole window	71
5.19	Cluster bootstrap results	73
5.20	Seed variance	74
5.21	Training-budget control	75
5.22	Split variance	76
5.23	Final test results	77
5.24	What the app reads in long contracts	80
A.1	Full risk taxonomy	93

Nomenclature

The list below gives the abbreviations used in this report.

BERT	Bidirectional Encoder Representations from Transformers
CI	Confidence Interval
CPU / GPU	Central Processing Unit / Graphics Processing Unit
CUAD	Contract Understanding Atticus Dataset
ECE	Expected Calibration Error
EM	Exact Match
FN / FP	False Negative / False Positive
IP	Intellectual Property
LLM	Large Language Model
LR	Logistic Regression
MIL	Multiple-Instance Learning
MPS	Metal Performance Shaders (Apple GPU backend)
NLP	Natural Language Processing
QA	Question Answering
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
RQ	Research Question
SD	Standard Deviation
SEC	US Securities and Exchange Commission
SQuAD	Stanford Question Answering Dataset
TF-IDF	Term Frequency-Inverse Document Frequency
TN / TP	True Negative / True Positive

Glossary of terms

AND-ensemble A rule that reports a clause category only when both the TF-IDF model and the DistilBERT model say it is present.

Bootstrap A way to measure uncertainty: the test contracts are drawn again at random, with repeats, many times, and the score is recomputed each time.

Calibration Whether a model’s score can be read as a real chance. A well-calibrated score of 0.8 is right about 80% of the time.

Clause category One of the 41 types of contract clause labelled in CUAD (for example *Governing Law*, *Non-Compete*, *Cap on Liability*).

Contract-level split A train/test split that puts each whole contract on one side, so no clause from a training contract can appear in the test set.

Max-pooling (over windows) Scoring every window of a document and using the highest window score as the score for the whole document.

Micro-F1 One F1 score computed over all (contract, category) decisions together, so every decision counts the same.

Micro-F2 The same, but treating recall as twice as important as precision, so a missed clause costs more than a false alarm.

Presence classification Deciding, for each clause category, whether a contract contains at least one clause of that category.

Recall-first setting A setup whose rule and thresholds were chosen on the validation split to maximize micro-F2; Clausify uses one by default.

Retrieval ceiling The most a “search first” design can ever find: a clause in a window that the search never picks can never be found.

Span extraction Finding the exact text of a clause inside the contract, treated as a question-answering task.

Token-F1 How much the extracted text overlaps the real clause, word by word (1.0 means a perfect match).

Validation split The 81 training contracts held out in the re-run to choose settings such as thresholds, so that the test contracts play no part in any choice.

Weaker party The person or company signing the contract with less bargaining power; the risk table is written from their side.

Chapter 1

Introduction

1.1 Background

Contracts are part of almost everything people and businesses do, and reading them takes a lot of time. A typical contract in our dataset is about 33,000 characters long (roughly 5,000 words), and the longest is more than 338,000 characters. Somewhere in that text are the parts that decide what the signer is agreeing to: who owns new intellectual property, whether there is a limit on how much you can owe, whether the contract renews on its own, and whether you are stopped from working with competitors after it ends. Having a lawyer read a contract takes hours and costs money that many people and small businesses cannot afford, so people often sign contracts they do not fully understand and take on risks they never noticed.

The Contract Understanding Atticus Dataset (CUAD) [1] was created to help automate this kind of review. It has 510 real business contracts in which legal experts marked every clause that belongs to one of 41 important categories, work said to be worth about two million US dollars of lawyers' time. CUAD treats the problem as finding text: given a category, find the clause. Once it finds a clause such as "Uncapped Liability", it stops. It does not warn the reader what the clause means for them, explain it in simple words, or check how clauses affect each other.

1.1.1 System at a Glance

Chapter 4 explains the system in detail, and Figure 1.1 shows it in one picture. The system answers three questions about a contract in order: *what is in it*, *where is it*, and *what does it mean*. CUAD was built to test the first two. The third step, shaded in the figure, is what this project adds.

1.2 Rationale of the Study or Motivation

Our project tries to close the gap between finding a clause and understanding it. On top of finding clauses we add two things: a simple risk table that gives every clause category a High, Medium or Low level, judged from the side of the weaker

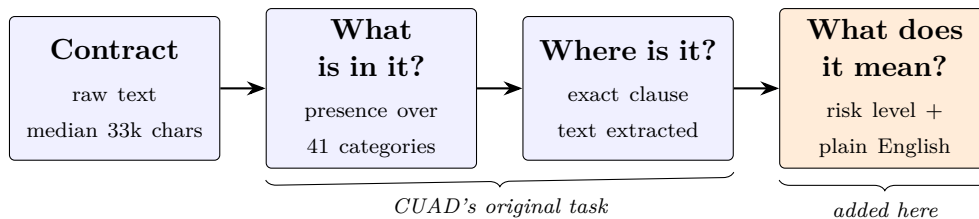


Figure 1.1: Three questions the system answers

party, and a text model meant to turn each clause into one plain-English sentence. We built the clause-finding models and the risk table into *Clausify*, a prototype web app where a user pastes or uploads a contract and gets back a list of clauses sorted by risk. The text model worked less well than we hoped: it learned to repeat a fixed sentence for each category instead of describing the actual clause (Section 5.1.3). Chapters 5 and 6 report these results as they are.

The reason for this study is practical as well as academic. Earlier work on CUAD answers the question “where is the clause?” for a lawyer who already knows what to look for. The people who need help most, such as individuals, freelancers and small businesses who sign contracts without a lawyer, also need to know which clauses matter most to them and what each one means in everyday language. A system that answers those questions, and says how often it is wrong, could make contract review possible for people who cannot pay for it today. We also wanted to measure how well such a system works instead of assuming it, since a contract-review tool that looks confident but misses clauses without saying so is worse than no tool at all.

1.3 Problem Statement

The project works on four connected problems:

1. Finding which clauses are present. For each of the 41 CUAD categories, decide whether a contract has at least one clause of that type. The main difficulty is length: a typical contract is about 16× longer than the amount of text a standard transformer model can read at once.
2. Finding the exact clause text. For each category that is present, find the exact words of the clause in the contract. We treat this as a question-answering task in the same style as SQuAD [2].
3. Explaining in plain language. Rewrite each clause as one sentence that a non-lawyer can understand.
4. Showing the risk. Give every clause found a risk level and a one-line reason, so the result reads like a ranked list of warnings. We did not test whether this helps users (Section 6.3).

1.3.1 Research Questions

Our objectives lead to five research questions, which the rest of this report answers. We give the short answers here so that readers can check each claim against the evidence without searching for it.

RQ1 *Can a transformer that reads a contract in small windows beat a simple word-based model that reads the whole contract, at finding which clauses are present?*

It ties, and together they win. In our first setup the simple model won by +0.082 micro-F1, and the gap survived more training data (Section 5.4.3) and three random seeds (Section 5.4.2). That setup let the transformer read only about half of each window. With the whole window read and both models tuned on the same validation split, the transformer ties the tuned simple model (0.797 against 0.785), and an ensemble of the two beats it (0.809, with an interval that excludes zero; Section 5.3.3).

RQ2 *How well can the system find the exact clause text in long contracts?*

The first span model reached a token-F1 of 0.764 (95% CI [0.743, 0.782]) only on windows centred on the clause. On plain windows that contain the clause it reached 0.366, and 0.350 when the presence model chose the window with the app’s setting. Retrained on the chunks it really reads, it reaches 0.779 on plain windows and 0.673 when the presence model chooses (Sections 5.1.2, 5.2.1 and 5.2.1).

RQ3 *Does the summarizer really describe each specific clause in plain English?*

No. For every input it copies one of 41 sentences we wrote in advance. Against clause-specific reference sentences, it scores no better than just printing that fixed sentence without any model. Larger FLAN-T5 models, up to about 780 million parameters, used without fine-tuning mostly copy the clause instead of explaining it (Section 5.1.3).

RQ4 *How do errors add up across the whole system?*

They multiply, and at first it was worse than the separate scores suggested: with the app’s setting only 22.1% of real clauses got through all three steps, compared with the $\approx 77\%$ that the separate scores predicted (Section 5.2.1). Both causes were in our own setup. Reading the whole window fixed most wrong-window choices, and retraining the span model on the chunks it really reads raised the share to 59.6% (Section 5.2.1). What is left is split about evenly between the two causes.

RQ5 *Do the system’s mistakes avoid the clauses that would hurt a signer most?*

Not when the setup is chosen by overall score. In the first setup, the ensemble with the best micro-F1 misses 36.9% of High-risk clauses, while the model we called weaker misses only 15.9% (Section 5.2.2). Choosing the setup on a validation split for recall instead, the transformer misses 9.7% (Section 5.3.3), and this is what the app uses.

RQ3 is answered negatively and RQ4 partly so, and the answers to RQ1, RQ4 and RQ5 changed once we found flaws in our own setup. We report all of them as results

of the project, because each is something a reader would have got wrong by looking only at the separate model scores.

1.3.2 Contributions

1. A complete system that goes from a raw contract to a risk-sorted report: it finds which of the 41 clause types are present, quotes the clause, gives the type a risk level with a reason, and adds a plain-English line. CUAD work so far stops at finding the clause.
2. A controlled comparison of a whole-document word model and a window-based transformer for deciding which clauses a contract contains. It shows how a strong simple baseline can look unbeatable when the transformer reads only half of each window and is held to an untuned threshold, and that the two tie once both are tuned on the same validation split, with their ensemble ahead.
3. A risk table for the 41 CUAD categories, judged from the weaker party's side with written rules and a reason for each level, which no earlier CUAD work provides (Section 4.2.4).
4. *Clausify*, a working app that runs all the trained models on a user's own contract, with a recall-first setting chosen to miss as few High-risk clauses as possible.

Section 6.2 gives the full list, including the negative results.

1.4 Objective

- Build and fairly test a complete three-model CUAD system on an ordinary laptop, using an 80/20 split by contract so that every reported number comes from contracts the models have never seen.
- Check whether a transformer really beats a strong simple model at finding which clauses are present.
- Measure how well the exact clause text is found, using token-level F1 and exact match on the test set.
- Test the summarizer against two different sets of reference sentences, because one automatic score turned out to measure the reference sentences more than the task.
- Measure how many clauses survive the whole system from start to finish, as well as the score of each step.
- Put the trained models into a working demo app that shows clauses sorted by risk.

1.5 Methodology in Brief

We tested every design before building on it, using one fixed 80/20 split of CUAD by contract (408 training contracts and 102 test contracts). After exploring the data (Section 4.3.5), we first built a simple word-based model (TF-IDF with logistic regression) that reads the whole contract, before training any transformer. We also measured the highest recall a “search first” design could ever reach (Section 4.2.1). Based on that, we trained a DistilBERT model that scores the contract in small windows and takes the highest window score as the answer for the whole contract (multiple-instance learning), and we combined it with the simple model using an AND rule that reports a clause only when both models agree. A DistilBERT question-answering model finds the exact clause text, a fine-tuned FLAN-T5-small model writes a one-sentence plain-English explanation, and a hand-made table gives each of the 41 categories a High, Medium or Low risk level.

Every model was tested once on the 102 test contracts. We report each score with confidence intervals from a bootstrap over contracts, and we checked how the results change with the amount of training data, the random seed, the way window scores are combined, the decision threshold and the train/test split. We also tested the full system from start to finish. These checks led us to a flaw in the first setup, and we re-ran the presence comparison with the transformer reading whole windows and both models tuned on a validation split taken from the training contracts. The whole-system test also showed that the span model had been trained on windows unlike the ones it reads, so we retrained it on those. Finally, we put the trained models into *Clausify*, a Streamlit web app that gives a risk-sorted clause report for any contract. Chapter 4 describes the method in full.

1.6 Scopes and Challenges

The project covers English business contracts like those in CUAD: 510 contracts from US SEC filings, marked for 41 clause categories. Within that scope it handles four tasks (finding which clauses are present, finding their exact text, explaining them in plain language and showing their risk level) and delivers a working prototype app. Some things are left out on purpose: legal advice of any kind; contracts in other languages or countries (including Bangladeshi contracts, which inspired the work but were not tested); judging the risk of a clause’s exact wording; finding conflicts between clauses; and a formal user study of the app. All models were sized so they can be trained and run on one ordinary laptop.

Six challenges shaped the design:

1. Very long documents. A typical contract is about 33,000 characters, roughly $16\times$ the 512-token limit of a standard transformer (about 2,000 characters), and the longest contract is more than 338,000 characters, about $170\times$.
2. Very uneven categories. Some categories appear in 100% of contracts and some in only 2.5%, and a few have only a handful of examples in the whole dataset.

3. A small, fixed dataset. The 510 contracts are all there is, and expert marking is too costly to add more, so we had to be careful about over-fitting and about test data leaking into training.
4. Limited hardware. Training ran on an Apple-Silicon laptop without a CUDA GPU. Our first-choice model (DeBERTa-v3) was unstable on the MPS backend, and GPU memory was not freed between training runs.
5. No plain-language answers to learn from. CUAD has no plain-English versions of clauses, so we had to write our own reference sentences, which limited what the summarizer could learn.
6. No agreed answer for risk. Contract risk depends on which side you are on, the situation and the country, and there is no expert-checked risk model for CUAD’s 41 categories.

1.7 Report Organization

The rest of this report is organized as follows. Chapter 2 reviews background and earlier work. Chapter 3 sets out the system’s requirements, its effect on society, the environment, ethics and cost, and the project plan. Chapter 4 explains the method: how we designed the system, which other designs we considered, how the data was prepared, and how the models and the Clausify app were built. Chapter 5 analyses the results on the test set, including the statistics and the changes they led to. Chapter 6 sums up the findings, contributions and limits, and suggests future work. The appendices contain the full 41-category risk table, a checklist for repeating the work and the layout of the project files.

Chapter 2

Literature Review

2.1 Preliminaries

This section explains the models and methods the project is built on.

2.1.1 Transformer Encoders and Model Distillation

The transformer [3] is a type of neural network that reads text with self-attention instead of word by word. BERT [4] showed a simple recipe that this project relies on: first train a model on a very large amount of general text, then fine-tune it for a specific task on a much smaller amount of labelled data. The recipe suits us, because 510 contracts is a small dataset for deep learning.

Because all our work runs on one laptop, we use DistilBERT [5], a smaller, six-layer version of BERT that keeps about 97% of BERT’s language ability while being 40% smaller and 60% faster. The published CUAD results use much larger models, so we give up some accuracy in exchange for running everything on normal hardware. We point out this size difference whenever we discuss our results.

2.1.2 Long-Document Strategies and Multiple-Instance Learning

Contracts are much longer than a transformer can read at once. There are three usual ways to deal with this: (i) *retrieval*, where a quick, cheap method picks the few parts of the text most likely to contain the answer and only those go to the transformer; (ii) *sliding windows*, where the model reads every part of the text and the scores are combined into one answer for the whole document; and (iii) *sparse-attention models* such as Longformer, which can read a longer piece of text at once.

The third option is often dismissed in one line as “too expensive”, which is not enough of a reason. We did not run a Longformer experiment, so what follows is a design choice. The one part of it we could check cheaply on our own laptop was the cost, and Section 4.4.7 reports that check.

We had three reasons, and only the third is about cost. The first and most important is that Longformer would not solve the problem. It can read 4,096 tokens at once. At about four characters per token, our typical contract of 33,143 characters is about 8,300 tokens and the longest is about 85,000, so Longformer would still see only half of a typical contract and one-twentieth of the longest one. It makes the window eight times wider, which is still not wide enough for this data. We would still need windows and a way to combine their scores, which is the design we ended up using anyway, only with a much more expensive model underneath. This reason comes from arithmetic about the dataset and not from an experiment, but the arithmetic is clear: CUAD contracts are about ten times too long for a wider window to fix.

The second reason is availability. Our whole system uses small (distilled) models so that it can be trained and run on one laptop. We do not know of a small version of Longformer that is as reliable and well supported as DistilBERT, so using Longformer would have meant giving up the small models that make the rest of the project easy to repeat.

The third reason is cost in practice. We believed that fine-tuning a Longformer with 148.7M parameters at 4,096 tokens was not possible on one Apple-Silicon laptop. We were also worried about special attention code on a non-CUDA machine, after DeBERTa-v3 produced NaN (broken) values on the MPS backend (Section 4.4.6). When we later measured it (Section 4.4.7), the memory part of this belief turned out to be wrong, since one training step does fit at batch size 1. The time part was right by a very large margin. The app adds its own cost: *Clausify* scores every window of an uploaded contract for all 41 categories while the user waits, so it is only usable if this takes seconds. That can be measured without any training, and Section 4.4.7 gives the speed and peak memory of Longformer-base at 4,096 tokens on our laptop, next to the DistilBERT setup we used.

On a CUDA computer with more memory, the third reason would be weaker and the second might disappear, but the first would still hold. The choice is backed by one measurement and some arithmetic. It does not prove that sparse attention cannot work on CUAD. A properly trained Longformer with windows would be a fair comparison to ask for, and we list it as future work.

Reading a document in windows and taking the highest window score is known as *multiple-instance learning* (MIL). The document is treated as a bag of windows, and the bag counts as positive if any one window is positive, which is what taking the maximum score does. MIL has a well-known weakness, and our experiments confirmed it: it produces too many false alarms. A document has dozens of windows, so the chance that some window fires by mistake grows with the length of the document, and precision drops.

2.1.3 Text-to-Text Models for Summarization

T5 [6] treats every language task as “text in, text out”, which suits clause simplification: legal text goes in and a simple sentence comes out. FLAN-T5 [7] is a version of T5 trained on many tasks with instructions, so it already gives useful answers to a simple prompt such as “summarize in plain English:” without extra training. We score summaries with ROUGE-L [8], which measures how many words the output

shares, in the same order, with a reference sentence. Chapter 5 shows why, when there are only a few reference sentences, this score has to be checked by reading the outputs.

2.2 Review of Existing Research

This section reviews earlier work on legal language processing, legal risk, plain-language legal writing, and the accuracy of machine-written summaries.

2.2.1 Legal NLP and the CUAD Benchmark

People have long wanted to use language technology on legal text, and progress was slow for a simple reason: labelling legal text needs lawyers, and lawyers are expensive. CUAD [1], released by The Atticus Project [9] at the NeurIPS 2021 Datasets and Benchmarks track, changed this. It provides 510 real business contracts from public SEC filings, in which experienced lawyers marked every piece of text that belongs to one of 41 important clause categories, such as governing law, non-compete, limits on liability and IP assignment. The data uses the SQuAD format [2]: each (contract, category) pair is asked as a question about the contract text, and the marked clauses are the answers, with their exact character positions.

The original CUAD paper treats the task as question answering and finds that bigger models do much better. Its best results come from the extra-large DeBERTa models, and even those scores are modest, which shows how hard it is to find legal clauses. Two features of CUAD affect any model built on it. Categories are very uneven: some appear in almost every contract, while others appear only about a dozen times in the whole dataset. Contracts are also long, far longer than a transformer can read at once, so every practical system first has to work out how a model that reads 512 tokens at a time can read a 50,000-character contract.

2.2.2 Other Contract Datasets and Benchmarks

CUAD is one of several expert-annotated contract resources, and the others show what a clause finder leaves out. ContractNLI [10] asks a different question of 607 non-disclosure agreements: whether each of a fixed set of statements is entailed by the contract, contradicted by it or not mentioned, with the supporting text as evidence. Its authors find that existing models fail badly and that the way contracts are written, for example negation by exception, makes the task hard. Finding a clause is therefore not the same as knowing what it commits a party to, which our category-level risk table does not try to do (Section 2.2.4). MAUD [11], from the team behind CUAD, goes deeper into one kind of contract, with over 39,000 expert-annotated examples about merger agreements. LegalBench [12] collects 162 legal reasoning tasks written by legal professionals, including tasks built from CUAD, and was designed to test large language models. LegalBench-RAG [13] traces those tasks back to their source documents, including CUAD, ContractNLI and MAUD, to test the retrieval step of retrieval-augmented systems, and finds that retrieving

the right short passage from a long contract is far from solved. That agrees with the retrieval limit we measured before training (Section 4.2.1).

2.2.3 Large Language Models and Contract Review

Since CUAD appeared, general-purpose large language models (LLMs) have become the obvious alternative to a pipeline of small trained models, and a reader may fairly ask why we did not use one. The evidence points both ways. GPT-4 passes the Uniform Bar Examination [14], and an industry preprint reports that GPT-4 and Claude matched or beat junior lawyers at spotting the legal issues in procurement contracts in a fraction of the time, while trailing them slightly at locating the relevant text [15]. Models trained on legal text are also released openly, such as the 7-billion-parameter SaulLM-7B [16]. On the other side, ChatGPT without task-specific training reached an average micro-F1 below 50% across the LexGLUE tasks [17], far below the fine-tuned models reported for that benchmark [18]. Legal hallucination is common: asked verifiable questions about real court cases, public LLMs were wrong at least 58% of the time and often accepted a user’s false legal premise [19], and commercial legal research tools that combine an LLM with retrieval still hallucinated between 17% and 33% of the time [20]. Long inputs add a further problem, since language models use information in the middle of a long context less reliably than information near its start or end [21], and a CUAD contract is long.

We did not use an LLM for four reasons, and the last of them is a limitation of this project. First, privacy: contracts are usually confidential, and Clausify runs entirely on the user’s own computer, which a hosted model would not allow. Second, reproducibility and cost: every number in this report can be reproduced on a laptop from saved models, while a hosted model can change without notice and charges for every contract. Third, the people this tool is for are the least able to notice a fluent wrong answer, the same concern Dahl et al. raise for people without legal help [19]. Fourth, we had no budget for paid model access and did not run an LLM on our 102 test contracts. We therefore cannot say whether an LLM would beat our system on CUAD under our protocol, and Section 6.5 lists that comparison as future work.

2.2.4 Legal Risk Assessment and Decision Support

The risk part of this project has the least earlier work behind it, which is a weakness of our work. Since we tell a non-lawyer which clauses to worry about, the obvious question is what this advice is based on. We did not use an existing framework. We wrote the risk table ourselves (Section 4.2.4), which is why we call it a prototype. The closest precedent is CLAUDETTE [22], which uses machine learning to flag potentially unfair clauses in the terms of service of online platforms. It is the clearest example of a system that tells the weaker party which clauses to worry about, but its contracts, its weaker party (a consumer) and its legal standard all differ from ours, so its labels cannot be reused for CUAD.

Earlier work does explain why this is hard. Legal language benchmarks usually treat “risk” indirectly, by sorting text into legally meaningful categories without

judging how serious something is. LexGLUE [18] collects tasks such as predicting court decisions and classifying laws, and LEGAL-BERT [23] shows that models pre-trained on legal text do better on them. None of these tasks asks how dangerous a clause is for a particular person signing it. CUAD itself [1] describes its categories as the clauses lawyers look for when checking a deal, which marks them as important without saying how important.

How serious a clause is remains an open question, and it is hard to learn from data alone, for three reasons. Risk depends on which side you are on: the same unlimited indemnity is dangerous for the party giving it and useful for the party receiving it, so a risk label only makes sense once you pick a side, a problem Section 4.2.4 handles only by making an assumption. Risk depends on the situation, since a \$50,000 limit on liability means little to a large company but can ruin a one-person business. And risk depends on the country, since a non-compete that is binding in one country may be invalid in another. Our table gives one fixed level per category, so it ignores all three on purpose.

This affects how our work should be read. We know of no published risk model that covers CUAD’s 41 categories, so our table cannot be compared with an existing one. Work on explainable legal AI argues that tools in this area should show their reasoning instead of just giving a score. Our table does this: every level comes with a written reason that can be questioned line by line (Appendix A). That openness makes the table easy to check, though it does not make the table correct.

2.2.5 Plain-Language Legal Communication

Chapter 5 finds that our summarizer does not really summarize, and the closest earlier work is about simplifying text. The two goals differ. Summarizing makes text shorter, while simplifying says the same thing in easier words. For a contract clause, shorter is often the wrong goal, because the conditions and exceptions a summary would drop are often what decides what the clause does.

The closest earlier work is Manor and Li [24], who collected plain-English summaries of contract sections and found that models trained on them tend to give general, repeated answers, an early version of the problem our template test measures. Measuring how easy a text is to read is a much older field [25], and it offers a check our evaluation lacks: that the output is easier to read, and not only shorter. We do not report any readability score. Looking back, that is a gap, since this part of the project exists to make clauses easier to understand.

Our results agree with this work: a score computed against reference sentences depends heavily on which reference sentences are used. With only 41 different target sentences, ROUGE-L rewards the model for recognising the category. Section 5.1.3 measures this directly by changing the reference sentences while keeping everything else the same.

2.2.6 Faithfulness and Hallucination in Generated Summaries

When a system shows machine-written sentences next to a user’s own contract, whether those sentences are true is a safety issue. Research on summarization has shown that fluent machine-written summaries often say things that are not in the source. Maynez et al. [26] find that many generated summaries contain content the input does not support, and that ROUGE agrees poorly with human judgements of accuracy. Kryściński et al. [27] suggest checking factual accuracy as a separate task. Scores such as BERTScore [28] measure meaning better than simple word overlap, but they do not check facts either.

Our system has an unusual version of this problem. A model that only outputs fixed template sentences cannot invent details in the usual way, because it can only produce one of 41 sentences written by people, and each one correctly describes its category. Its failure is that it says nothing about the specific clause, and when it picks the wrong template (30% of the time, Section 5.1.3) the output is fluent and confident but describes a completely different kind of clause. For a legal assistant this may be worse than an invented detail, because the output gives no sign of the error. The sentence is well written and true of its own category, and only a comparison with the clause shows that it does not describe it.

Our evaluation is therefore missing the checks this research recommends. We report ROUGE-L against two sets of reference sentences and we read outputs by hand. We do not report BERTScore, an automatic fact check, or a human judgement of whether the legal meaning is kept, and for a part that non-lawyers will see, the human judgement matters most. Section 6.5 lists it as future work.

2.3 Summary of Key Findings

Earlier CUAD work stops once the clause is found. A person signing a contract probably needs more: which clauses matter, how much, and what they mean in plain language. This project tries to add that layer, with a prototype risk table for each category, plain-English template sentences and a demo app. It also re-tests, on a strict test set, the common belief that a transformer must beat simple word-based methods at deciding which clauses a contract contains. The demo app works, and the risk table gives every category a level with a written reason. The plain-English part outputs category templates instead of summaries, so it is not solved. The added layer mostly gives a measurement of how far it is from working. Recent work tests general-purpose LLMs on legal tasks with mixed results (Section 2.2.3); we did not compare with one, and that is the most obvious missing baseline.

Table 2.1 compares this work with the earlier work above. The first difference is scope. Earlier work stops at finding the clause text, while our output is a risk-sorted, plain-English report, which is the part a non-lawyer can act on. The second is in the “Long inputs” column. Every system on CUAD must choose how to handle long documents, and usually the choice is just stated. We measured ours before any training: the 64.0% retrieval limit in Section 4.2.1 ruled out retrieval and led

us to multiple-instance learning. The Model column also shows a cost. Our small models (67M parameters) are about ten times smaller than the DeBERTa-xlarge model behind CUAD’s best published result, so our extraction scores cannot be compared directly with theirs, and we do not claim they can.

Table 2.1: Related work

Work (data)	Task	Model	Long inputs	Risk	Plain English	Evaluation
Hendrycks et al. [1] CUAD, 510 contracts	Span extraction	DeBERTa-xlarge	Full-document QA with a large model	No	No	Held-out; precision at 80% recall
Chalkidis et al. [23] several legal corpora	Domain pre-training and classification	LEGAL-BERT	Truncation to 512 tokens	No	No	Per-task held-out
Chalkidis et al. [18] LexGLUE, 7 tasks	Legal language understanding	BERT and RoBERTa models	Depends on the task	Indirect (task labels)	No	Multi-task benchmark
Manor and Li [24] TL;DR-Legal style	Plain-English summarization	Seq2seq	Section-level inputs	No	Yes	ROUGE and manual review
Beltagy et al. [29] long-document benchmarks	New architecture	Longformer	Sparse attention, 4,096 tokens	No	No	Task benchmarks
Koreeda and Manning [10] ContractNLI, 607 NDAs	Inference with evidence spans	BERT-style span classifier	Context segmentation	No	No	Held-out test split
Guha et al. [12] LegalBench, 162 tasks	Legal reasoning benchmark	20 LLMs, prompted	Mostly short task inputs	No	No	Per-task scores
Martin et al. [15] procurement contracts	Issue spotting against lawyers	GPT-4, Claude, prompted	Long-context LLM	Indirect (legal issues)	No	Against senior lawyers
Retrieval-QA pipelines various	Span extraction	Various	Retrieval then reader (ceiling usually assumed)	No	No	Various
This project CUAD, 510 contracts	Presence, span, category risk and template text	DistilBERT and FLAN-T5 (small)	Sliding-window MIL; ceiling <i>measured</i> , sparse attention <i>benchmarked</i>	Yes (category level, prototype)	Yes (template based)	Contract-level held-out; cluster bootstrap; seed, budget and split controls; end-to-end

Note. “Long inputs” is how each work handles documents longer than the encoder window; “Evaluation” is the level at which results are reported.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

3.1.1 Functional requirements

- FR1** The system shall accept a contract as pasted text or as an uploaded `.pdf`, `.docx` or `.txt` file.
- FR2** For each of the 41 CUAD clause categories, the system shall decide whether the contract has a clause of that type, and shall show the detection score of each type it finds.
- FR3** For each category found, the system shall show the clause text from the user's own contract.
- FR4** The system shall give every clause found a High, Medium or Low risk level and a one-line reason in plain English.
- FR5** The system shall show the clauses grouped by risk level (High first) and sorted by score within each group.
- FR6** The system shall let the user analyze either a whole contract or a list of separate clauses, with one card per clause in the second case.

3.1.2 Non-functional requirements

- Privacy: all models shall run on the user's own computer, and no contract text shall be sent to another service or saved to disk.
- Hardware: all models shall train and run on one ordinary laptop without a CUDA GPU.
- Speed: analyzing a typical contract shall take no more than tens of seconds on a CPU (measured: median 74.7s, worst case 116.2s; Table 4.6). Reading whole windows doubled the time of the first setup, so long contracts now exceed this target.

- Fair testing: every reported score shall come from contracts the models never saw during training (split by contract), and no setting shall be chosen by looking at the test set.
- Repeatability: the split, the baseline model and all trained models shall be saved, and a separate notebook shall recreate every reported number from them.
- Openness: the app shall say that it is not legal advice, and every risk level shall come with a written reason (Appendix A).

3.1.3 Data and resource requirements

Table 3.1 lists the data, hardware and software the project uses. All of it is either free to use or was already owned by the team.

Table 3.1: Resource requirements

Resource	Specification
Dataset	CUAD v1 (The Atticus Project), 510 annotated contracts: <code>master_clauses.csv</code> and <code>CUAD_v1.json</code> , obtained from the Hugging Face hub
Hardware	Apple M4 laptop, 16 GB unified memory, MPS (Metal) GPU backend; no CUDA GPU
Language and runtime	Python 3.13.5
Deep learning	PyTorch 2.8.0, Hugging Face <code>transformers</code> 5.12.1, <code>datasets</code> 5.0.0
Classical ML and data	scikit-learn 1.6.1, NumPy 2.1.3, pandas 2.2.3, <code>rouge_score</code>
Pre-trained models	<code>distilbert-base-uncased</code> (presence and span), <code>google/flan-t5-small</code> (summarizer)
Application	Streamlit (≥ 1.40) single-page web application
Development tools	Jupyter notebooks, Git and GitHub, $\LaTeX$

3.1.4 Design constraints

Four limits shaped the final design. One laptop cannot train very large models such as the DeBERTa-xlarge model used in the original CUAD work, so we chose smaller (distilled) models. The 510 contracts cannot be extended, so we started from pre-trained models and split the data by contract. The app must score every window of a contract for all 41 categories quickly, which is why it checks every clause type in the first 30 windows of a contract and in only 5 more windows after that, picked by the TF-IDF model. And the models must run locally for privacy, so online language-model services were ruled out.

3.2 Societal Impact

Having a lawyer review a contract takes hours and costs money that many people and small businesses cannot afford, so many people sign contracts they do not fully understand. A tool that points a non-lawyer to the clauses most likely to hurt them (unlimited liability, IP assignment, non-competes, automatic renewal) can make this easier and help people ask the right questions before signing. The users we have in mind are freelancers, start-ups and small businesses, the groups least able to pay for professional review.

The system has no direct effect on physical health or safety. Its indirect effect is on money and stress: signing a contract without understanding it can lead to disputes, losses and worry, which better-informed signing could reduce.

The system is not legal advice, and it says so. Its biggest risk to society is false reassurance, since a missed clause gives no warning and no warning can be read as “nothing to worry about”. Our measurements show this risk is real. Even with the retrained span model only 59.6% of real clauses get through the whole system (Section 5.2.1), so the tool must be presented as a help for reading a contract and never as a replacement for a lawyer.

CUAD contains US business contracts written in English. Risk levels that make sense under US law may not apply in other countries, and contracts are written differently in different places. The work was inspired by the situation in Bangladesh, but we tested no Bangladeshi contract, so we cannot yet claim these benefits there.

3.3 Environmental Impact

The project was designed to need little computing power. All models are small (DistilBERT with 67M parameters and FLAN-T5-small). Training the first setup’s three models took about 1.5 hours on a laptop, and the re-run presence model the app now uses took another 3.2 hours, and the app runs on a CPU with at most about 2.1 GB of memory (Table 4.6).

Including the extra test runs (the full-data presence run of 140.7 minutes, two extra seeds, the evaluation runs, the Longformer test, and the two re-runs of Section 5.3.3, about ten hours together), the whole project used about twenty hours of laptop computing. If a laptop draws 20–30 W under load (an estimate, not a measurement), that is well under one kilowatt-hour of electricity, very little compared with the many-GPU training often used for large legal language models.

Two design choices also keep the running cost low. The models run locally, so no data centre is used for each contract, and the system avoids large text-generating models, which use far more energy per request. The price is accuracy. Section 4.4.7 shows that larger models were rejected partly because of cost, and Chapter 5 reports what the small models achieve.

3.4 Ethical Issues

The Ethics Statement at the front of this report explains how the data and models were obtained. This section looks at the ethical issues that come from using the system, and the duties these place on the people who build it.

The system is a research prototype meant to help with contract review. It is not legal advice, and the app says so in a notice shown above every analysis. For a tool like this, the harder ethical questions concern what happens when someone relies on its output, and we list them here.

False reassurance is the main danger. A clause the system does not find gives no warning, and no warning is easily read as “there is nothing to worry about”. Our measurements show this reading would be wrong: even the app’s recall-first setting misses 7.0% of real clauses at the first step, only 59.6% get through the whole system (Section 5.2.1), and the first setup’s best-scoring ensemble missed 36.9% of the High-risk clauses (Section 5.2.2). The recall-first setting the app now uses still misses 9.7% (Section 5.3.3). A user who checks a contract with this tool and sees three warnings cannot assume there are only three problems. Missed clauses are silent while false alarms are visible, so we think the share of High-risk clauses found is the measure that matters most ethically, even though the overall score does not reward it.

Non-lawyers may trust the app too much. It is made for people without legal training, who are the least able to notice when it is wrong. Plain-English text shown with a confidence bar looks more certain than the model is, and Section 5.3.2 shows the bar is not even a true probability: scores above 0.9 mean only about a 73% chance that the clause is really there. The plain-English reason on each card is a fixed sentence for the whole category from the risk table, and it does not read the user’s clause. The summarizer, which was meant to describe each clause, also turned out to output one fixed sentence per category (Section 5.1.3). The app labels the summarizer’s sentence clearly and warns when it belongs to a different category than the card. Without these warnings, the bar and the reason would invite more trust than the evidence supports.

Laws differ between countries. CUAD follows US business law, and risk levels that are right there may be wrong elsewhere. A non-compete that is binding in one country may be invalid in another, and then a High badge would mislead. The work was inspired by the situation in Bangladesh and we tested no Bangladeshi contract, so this limitation applies to the people the work was meant for.

The risk table measures one side’s risk. It assumes the user is the side with less bargaining power (Section 4.2.4). For the side that wrote the contract, the same badges are wrong in the other direction, since a clause marked High for the signer may be exactly what the writer wanted. The system cannot tell which side the user is on.

Our experiments used only public contracts, so privacy did not arise during the research. A real deployment would face it at once, because contracts are among the most sensitive documents a business has. Section 3.4.1 explains what the demo app

does and does not do with an uploaded contract, and what a real deployment would need to add.

3.4.1 Data Handling and Privacy

The demo invites users to paste their own contracts, so we state exactly what happens to them.

In the demo, all the models run locally. The DistilBERT models, the TF-IDF model and the FLAN-T5 summarizer are loaded from disk and run in the same program as the app, so no contract text leaves the computer the app runs on. An uploaded `.pdf`, `.docx` or `.txt` file is read into memory and turned into text on that computer. Nothing is saved to disk, no database or log keeps the text, and no usage data is collected. The text exists only in Streamlit’s memory while the page is open and is gone when the browser session ends. The report the user can download is created on request and saved only where the user chooses.

All of this holds only because the demo runs on the user’s own computer. On a shared server, the same code would put the user’s contract in the memory of a machine they do not control, and Streamlit alone would not stop the server’s owner from reading it. The privacy comes from where the app runs, and the app itself does nothing extra to protect it.

Some protections are missing on purpose and would be needed for real use. There is no login, so anyone who can reach the computer can open the app. There is no file-size limit beyond Streamlit’s default and no time limit, so a very large upload can keep the app busy. PDF and Word files are read with standard open-source libraries (`pypdf` and `python-docx`), and a real deployment would need to treat these files as untrusted input, because a harmful file could try to attack the program that reads it. There is no rule for keeping data, because no data is kept. Party names are not removed before processing, which would matter if any part of the system were moved to an online model service.

Our rule for any future change follows from this. As soon as any part of the system calls an online service, privacy changes completely, because the contract would be sent to someone else and might be kept by them. The current design avoids this by using small local models. We think this is worth keeping for its own sake, and it is one of the few places where the limits we worked under led to a better design.

3.5 Standards

No formal engineering standard applies to a research prototype like this, but the project followed several common practices:

- Data format: CUAD uses the SQuAD question-answering format [2], and our clause-text model uses the standard SQuAD start/end-token setup, so the data works with common question-answering tools.

- Licences and credit: the dataset, the pre-trained models and all software are used under their open-source licences (CUAD under CC BY 4.0; DistilBERT, FLAN-T5, `transformers` and Streamlit under Apache 2.0; PyTorch and scikit-learn under BSD licences), and they are credited in the references.
- Testing practice: we followed standard machine-learning practice, with a separate test set split by contract, no tuning on the test set, confidence intervals from a bootstrap over contracts [30], and standard scores (F1, exact match, ROUGE-L [8]).
- Repeatability: Appendix B gives a checklist covering the software versions, data, seeds, file checksums and commands, and all code and reports are kept in Git.
- Writing: references use the IEEE style, and this report follows the Brac University CSE400 report template.

3.6 Project Management Plan

3.6.1 Schedule

The project was done by a team of three during 2026. Table 3.2 shows the main stages and what each one produced.

3.6.2 Budget

The project cost no money. The dataset, the pre-trained models and all software are free, and all training and testing ran on a laptop the team already owned. No cloud computing, paid services or paid labelling were used.

3.6.3 Resource management

The one shared laptop was the scarcest resource. Training runs were done one at a time, because the MPS backend does not free GPU memory between runs in the same program, and the summarizer was trained on the CPU for the same reason (Section 4.4.6). Everything needed for testing (the split, the baseline model and the three trained models) was saved to disk, so tests could be repeated without retraining. Code, notebooks and reports were kept in Git.

3.7 Risk Management

Table 3.3 lists the main risks we found during the project and what we did about each one. Several of them actually happened, and the last column records the response.

Table 3.2: Project schedule

Period (2026)	Activities	Deliverables
Before June	Problem definition, literature review, initial design including a cross-clause rule engine	Project proposal
June	Data acquisition, wide-to-long preprocessing, exploratory data analysis	EDA notebook
July	Contract-level 80/20 split; TF-IDF baseline; training of the presence, span and summarizer models; standalone evaluation; Clausify application	Trained models, <code>test.ipynb</code> , Clausify prototype
August	Supervisor review: training-budget and seed controls, cluster bootstrap, span and end-to-end evaluation; second review: risk-weighted evaluation, threshold and calibration, window-score rules, split variance	Project and findings reports, analysis scripts
September	Write-up of the full report and a conference-style paper; presentation	Report, paper, presentation slides
October	Viva preparation; re-training and re-testing of each model; whole-window re-run with validation tuning and a control run; final report in the CSE400 template	Final project report, updated app

3.8 Economic Analysis

Developing the system cost no money: the data, models and software are free, and the laptop was already owned (Section 3.6). The real cost was the team’s time and about twenty hours of laptop computing.

Running it is also cheap. The models run on a CPU with about 2.1 GB of memory, so the system can run on ordinary hardware with no cost per contract. Table 3.4 compares this with the two options a user has today.

The main benefit is quick sorting: the system points the reader to the parts of a long contract that most likely matter and explains them in plain language, at almost no cost. Expert time in this field is expensive (the CUAD labels alone are said to represent about two million US dollars of lawyers’ time [1]), so even saving part of the time needed to find key clauses is worth something.

Table 3.3: Risk management plan

Risk	Impact	Mitigation
First-choice backbone unstable on available hardware	High	Occurred (DeBERTa-v3 produced NaN gradients on MPS). Switched to DistilBERT; the method is unchanged and the backbone can be swapped back on CUDA hardware.
GPU memory exhaustion during training	Medium	Occurred. One training run per process; summarizer trained on CPU.
Data leakage between train and test	High	Contract-level split reused by all three models; retrieval and inference never use gold offsets; ensemble is parameter-free (Section 4.2.5).
Optimistic results from test-set tuning	High	First setup: no hyperparameter was tuned (Section 4.4.1). Re-run: every setting chosen on 81 validation contracts taken from the training split, and the test set scored once (Section 5.3.3).
Over-claiming small differences between models	Medium	Cluster-bootstrap confidence intervals, seed, budget and split controls (Section 5.4.1 onwards).
An architecture that cannot reach the required recall	High	Retrieval ceiling measured before training (64%), which ruled out the retrieval design at no training cost (Section 4.2.1).
Schedule overrun from long training runs	Medium	Distilled models (about 1.5 hours of training for the first setup); subsampled training sets; saved artifacts so evaluation needs no retraining.
Loss of work or results	Medium	Version control on GitHub; trained weights backed up before any re-training.
Users over-trusting the application	High	Not-legal-advice notice; risk badge presented as more reliable than quoted text; limitations stated in the application (Section 5.6.2).

The benefit is limited by how reliable the system is. Only 59.6% of real clauses get through the whole system with the app’s setting (Section 5.2.1), and it still misses 9.7% of High-risk clauses (Section 5.3.3). The system is a free first look at a contract and cannot replace a lawyer.

Table 3.4: Cost–benefit comparison

	Lawyer review	Hosted LLM service	This system (local)
Marginal cost per contract	High (professional fees)	Per-request fee	Near zero
Time per contract	Hours to days	Seconds	About a minute
Privacy	Confidential	Contract leaves the user’s machine	Contract stays on the machine
Reliability	Expert judgement	Unmeasured on this task	Measured; misses many clauses
Legal standing	Legal advice	None	None

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

The design follows the goals and limits from Chapter 3 and a “test before you build” process with four rules: (i) build a strong, simple baseline before any complex model; (ii) measure the best result a design could ever reach before spending training time on it; (iii) fix the train/test split by contract once, use it everywhere, and never tune on the test set; and (iv) test each step on its own, then test the whole system from start to finish. The result is a three-step system with a risk layer on top.

The system works in steps (Figure 4.1). The contract text is cut into windows. The presence step decides which of the 41 categories the contract contains, and the span step finds the exact clause text for each category found. The risk layer adds the category’s High/Medium/Low level, a text-generation step adds a plain-English sentence, and the output is a report sorted by risk. (The Clausify app runs every step; Section 4.4.9.) Section 5.1.3 shows that the last step picks one of 41 sentences written in advance instead of summarizing the clause in front of it, so “sorting into categories and showing a template” describes the system better than “summarizing”. Figure 4.1 names the steps and the models behind them, and Figure 4.12 at the end of this chapter shows the data passed between them, which helps when reading the implementation.

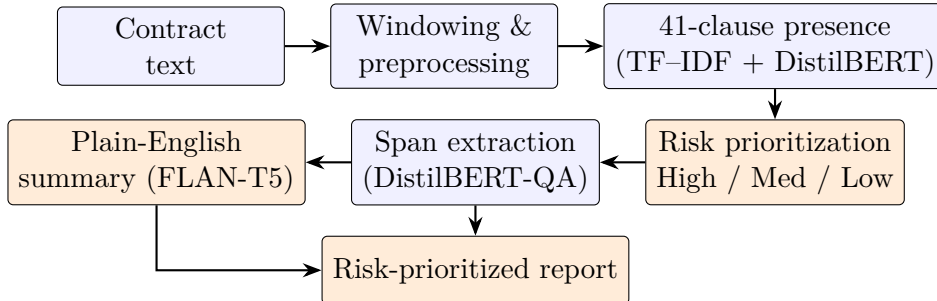


Figure 4.1: System pipeline

4.2 Preliminary Design or Design (Model) Specification

For each step we considered other designs and tested them where we could before choosing one. For finding which clauses are present we compared three designs (a simple word-based model that reads the whole contract, a “search first, then classify” transformer, and a transformer that reads the contract in sliding windows) and three ways of combining the simple model with the transformer. We also considered sparse-attention models and rejected them for the reasons in Section 2.1.2, then measured their cost later (Section 4.4.7). The design chosen for each step is described below.

4.2.1 Stage 2A: Presence Classification

A strong baseline first: full-document TF-IDF

A bag-of-words model has no limit on input length, so a TF-IDF model can read the entire contract, which no windowed transformer can. We built a TF-IDF vocabulary (20,000 features, single words and word pairs, sublinear TF, `min_df = 2`) from the 408 training contracts and trained one balanced logistic-regression classifier per category. Legal writing is very repetitive (“*This Agreement shall be governed by the laws of the State of ...*”), so the words alone carry a lot of information. This baseline sets the score every neural model has to beat.

Measuring the retrieval ceiling, and dropping retrieval

The obvious way to give a long contract to a transformer is to search first. The text is cut into 2,000-character windows that move forward 1,500 characters at a time (drawn in Figure 4.3), each window is scored by its TF-IDF similarity to the category, and the model sees only the best window or windows. The search uses only information that is available when the app runs (the category name or its CUAD question, never the answer), so no answer information leaks in.

Before training anything we measured the search hit-rate: how often the chosen window really contains the clause. Figure 4.2 shows how this works and why the result is a hard limit. No model after the search can do better than this number, because a clause in a window the search never picks can never be found. The best search we tried (using the category name and keeping the top 3 windows) reached only 64.0% (Table 4.1). The CUAD question text did worse than the plain category name, because the questions share so much standard wording that they look almost the same. A recall limit of 0.64 is well below the baseline’s recall of 0.81 (and its micro-F1 of 0.775), so a search-first transformer would lose before it was even trained. The measurement took seconds and saved us a wasted training run.

The number needs one note. We measured the limit of word-based search only. Every search in Table 4.1 compares windows by shared words, so a clause written without the words of its category name cannot be found even when it is clearly

there. A meaning-based search, which turns both the question and each window into vectors, would probably raise the 64.0% figure, possibly above the baseline, and we did not test one. The fair conclusion is therefore narrow: word-based search sets a limit that is too low to build on. That was enough to move us to the window design in Section 4.2.1. Whether meaning-based search could raise the limit far enough to compete is an open question, listed as future work.

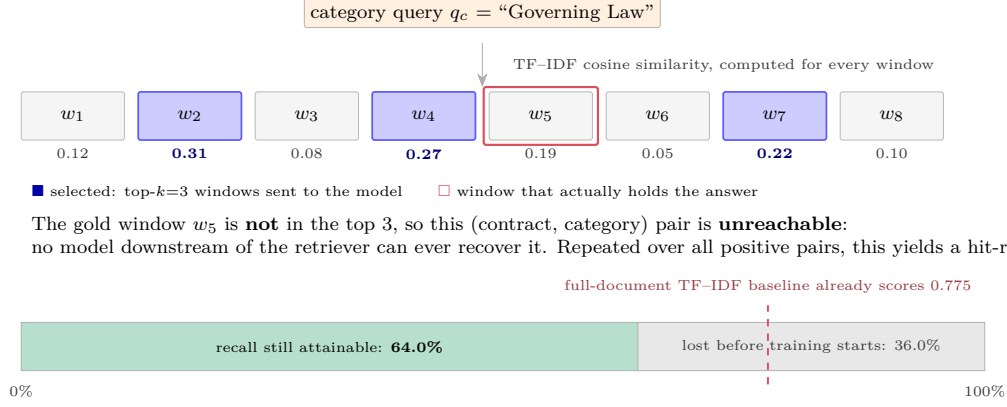


Figure 4.2: Retrieval ceiling

Table 4.1: Retrieval hit-rates

Query strategy	Top- k	Hit-rate
CUAD question text	1	38.6%
Category name	1	44.6%
Category name	3	64.0%
Name + question	3	61.5%

Note. The hit-rate is the recall ceiling a retrieval-based design would impose.

Sliding-window max-pooling (multiple-instance learning)

To remove this limit we trained a window-level classifier instead. Given a pair (category question, window), it predicts whether that window contains a clause of that category. Figure 4.3 shows how the windows are made. A 2,000-character window that moves forward 1,500 characters at a time leaves a 500-character overlap between neighbouring windows. The typical (median) clause is 196 characters long (Section 4.3.5), much shorter than the overlap, so every clause fits completely inside at least one window and none falls between two.

The training labels come from the marked character positions. Any window that contains a marked clause is positive, and for each (contract, category) pair we also take two random negative windows. This is normal supervised learning and does not leak answers: the labels only set the training targets, they never enter the model’s input, and nothing from the answers is used when the app runs. The 408 training contracts give 53,662 window examples (39.2% positive).

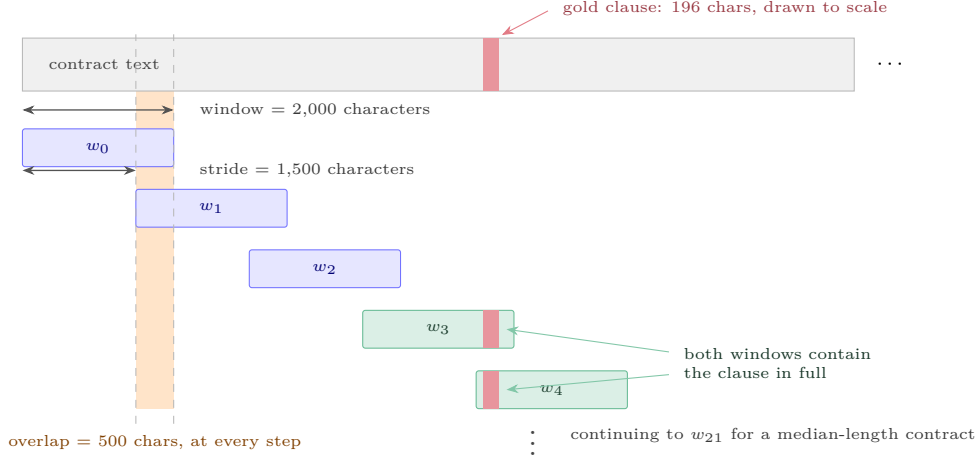


Figure 4.3: Sliding-window generation

When the app runs, the model scores every window of the contract, and the answer for the whole contract is the highest window score. This is the multiple-instance learning rule, shown in Figure 4.4: the document is a bag of windows, the bag is positive if any window is positive, and taking the max does exactly that. Because the windows cover the whole document with overlap, every clause can be reached and the search limit disappears. The cost is precision, and Figure 4.4 also shows why. The maximum is taken over every window, so one wrong window anywhere marks the whole contract as positive. A typical contract has 22 windows (the median; the mean is 35), so the chance that some window passes the 0.5 threshold by mistake grows with length, and long contracts give more false alarms than short ones whatever they contain.

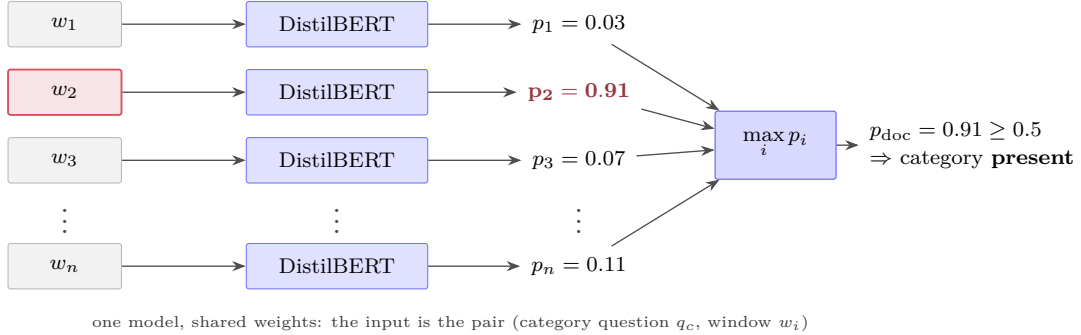
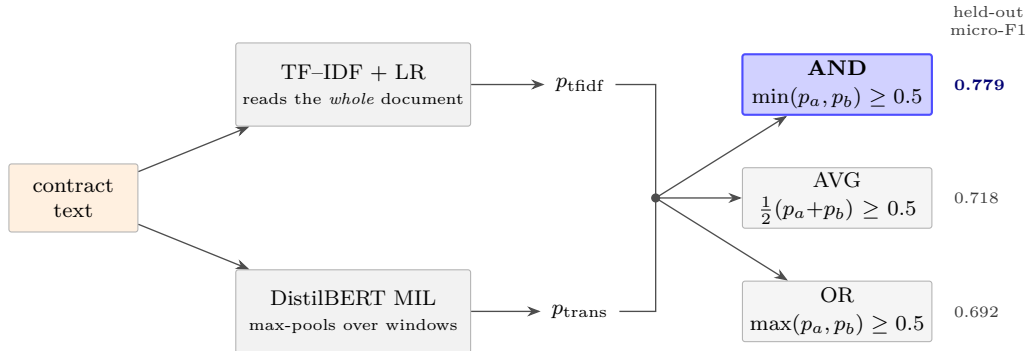


Figure 4.4: Multiple-instance max-pooling

The conjunctive ensemble

Compared category by category, the two models turned out to complement each other. The transformer does better where meaning matters (for example *No-Solicit of Employees* and *Liquidated Damages*), and the simple model does better on categories with clear keywords. We combine them with rules that have no settings to tune: AND (the min of the two scores), OR (the max) and AVG (the average). We chose such rules on purpose so that nothing can be fitted to the test set. Figure 4.5 shows the decision. The AND rule reports a clause only when both models agree,

which removes many false alarms. For a wrong detection to survive, both models must make the same mistake on the same contract and category, and their mistakes are mostly independent. This wins back the precision that taking the maximum loses.



All three rules are **parameter-free**: there is nothing to fit, so the test set cannot tune them. AND fires only when *both* models cross the threshold, so a false positive has to be made twice, independently, to survive.

Figure 4.5: Ensemble rules

4.2.2 Stage 2B: Span Extraction

This step needs the transformer, because a bag-of-words model cannot point to positions in the text. We fine-tune `DistilBertForQuestionAnswering`: the input is (CUAD question, window text), and the model predicts where the answer starts and ends. The character positions in `CUAD_v1.json` are turned into token positions with the tokenizer’s offset mapping.

The project uses two window sizes, and mixing them up makes the design look inconsistent. The 2,000-character window of Section 4.2.1 is a presence window. It covers the document in order, because at that point we do not know where anything is. The window used here is a focus window, 1,200 characters long and placed around the marked answer so that the answer starts about 150 characters in. Figure 4.6 draws the two at the same scale. The smaller, centred window has a simple technical purpose: it keeps the answer inside the model’s 320-token limit. With a 2,000-character window an answer near the end could be cut off, and the model would be trained on an example with the answer missing. This step also has no length limit at all, since it only searches inside a window that the presence step has already marked as relevant.

This is how the first span model was trained, and the end-to-end test showed what it cost: the model learnt to expect the answer near the start of its input (Section 5.2.1). We therefore trained a second span model on the chunks it really reads when the system runs, 1,200-character pieces of the presence window with the answer wherever it falls, or not there at all. Section 5.2.1 describes and tests it, and the app uses it.

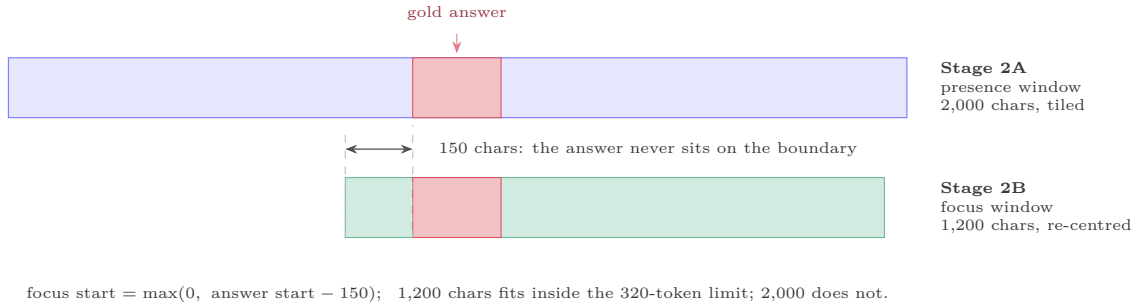


Figure 4.6: Presence vs. focus window

4.2.3 Stage 1: Plain-Language Summarization

CUAD gives us clauses but no plain-English versions to learn from, so we wrote our own: 41 template sentences, one carefully written one-sentence summary for each category, from the weaker party’s side. For *Anti-Assignment*, for example, the template is “You cannot transfer this contract to someone else without permission.” Each training clause is paired with its category’s template, which gives 11,156 training pairs, and FLAN-T5-small is fine-tuned on the prompt “**summarize in plain English:** <clause>”.

We test the summarizer in two ways: with ROUGE-L, and by comparing the fine-tuned model with the original model on the same clauses. We need both. With only 41 different target sentences, a model can get a high ROUGE score by recognising the category and copying its template word for word, without summarizing anything. Chapter 5 shows that this is exactly what happens.

4.2.4 Category-Level Risk Prioritization

The name of this layer limits what we can claim. The system does not read a clause and judge how dangerous that clause is. It finds which of the 41 categories a contract contains and gives each category a fixed level. These are different tasks, and treating them as one would greatly overstate the system. Our layer says “*contracts with a non-compete deserve attention*” and never says “*this non-compete is unusually harsh*”. Throughout this report we therefore call it category-level risk prioritization.

Assignment criteria

We wrote down the rules we used so that a reader can disagree with a particular level for a particular reason. Each category was scored on five points, from the weaker party’s side:

1. **Money at risk.** Can this clause cost money, and is there a limit? Having no limit is the strongest single sign of High risk.
2. **Cannot be undone.** If the clause takes effect, can the situation be reversed? Losing ownership of intellectual property or giving a permanent licence cannot be undone later; a notice period can.

3. **Limits on future freedom.** Does the clause limit what the party can do *after* the contract, or outside it, such as who it may work for, sell to or hire?
4. **One-sided.** Does the duty fall on only one side? A shared duty of confidentiality is less dangerous than a one-way indemnity with the same wording.
5. **How long it lasts.** Does the duty continue after the contract ends?

The rule we used is this. A category is High if it has no money limit (point 1), cannot be undone (point 2), or limits freedom after the contract (point 3). It is Low if it only gives information (it records a fact such as a date, a name or the governing law, or gives a benefit) and scores on none of the five points. Everything else is Medium: a real duty, but limited and reversible. This is why *Cap on Liability* is Medium even though it sounds protective (it limits how much you can recover, which may not cover a real loss), and why *Governing Law* is Low even though it matters in a court case (it is a fact about the contract and puts no one-sided duty on the signer).

The eight High categories fall into three groups, each based on a different point: money you could owe without limit (*Uncapped Liability*, *Liquidated Damages*, *Minimum Commitment*, *Most Favored Nation*); something you cannot get back (*IP Ownership Assignment*, *Irrevocable/Perpetual License*); and your freedom to earn later (*Non-Compete*, *Exclusivity*). The levels are our own judgement, made with these rules and the CUAD category descriptions. The full table with reasons is in Appendix A, and Figure 4.7 shows how the categories are spread across the levels.

Who is the “weaker party”?

The risk table depends on this phrase, which can mean different things. The weaker party is not always the same role. In a distribution contract it may be the distributor, in a licence the licensee, and in a franchise the franchisee. In a balanced contract between equals there may be no weaker party at all.

We defined it as the party with less power over how the contract was written, which in CUAD is usually the party that did not write it. Where the category alone did not make this clear, we scored from the side of the party who carries the duty: *Non-Compete* from the side that is restricted, *Audit Rights* from the side that is audited, and *License Grant* from the side that receives the licence. This rule of thumb has a known weakness. A clause that is High-risk for one side often protects the other, so the same badge is not equally useful to a licensor and a licensee. A system that judged each clause on its own could work out the side from the clause text. Ours cannot, so it assumes the user is the side with less power, and the app states this in the notice shown above every analysis.

Why a lookup table rather than a learned score

The risk table is a simple lookup, which we chose on purpose. It can be checked line by line, its reasoning is written down for each category, and it cannot change silently between versions. The cost is the limit named at the start of this section: it

cannot tell a \$10,000 liability limit from a \$10,000,000 one. Section 5.2.2 measures a second cost we did not expect. Because the levels are fixed per category, the system’s mistakes depend entirely on which categories it finds well, and the High-risk categories turn out to be among those it finds worst.

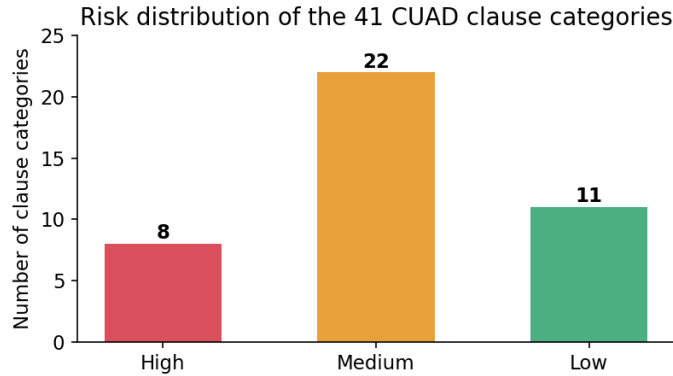


Figure 4.7: Risk-level distribution

4.2.5 Avoiding Data Leakage

With such a small dataset, data leakage is the easiest way to fool ourselves, so we followed one rule throughout: *the answers may be used to create training targets, but never to shape what a model reads or what is chosen when the app runs*. In practice the split is by contract, searches use only the category or its question, the marked answer positions label training windows but never build inputs or choose windows when the app runs, and the ensemble rules have no settings for the test set to tune.

Figure 4.6 shows the one place where the rule seems to be broken. The focus window is placed using the marked answer position, which is part of the answer. In training this is normal supervised learning, because it decides which text becomes a training example. In the real system there is no marked answer, and the span step reads the window chosen by the presence step. The separate span test in Section 5.1.2 is the exception. It builds each test example with the same centred focus window, so it measures extraction when the position of the answer is already known, and its score is a best case that the real system cannot reach. The realistic number comes from the start-to-finish test in Section 5.2.1, which never uses the marked answer position. The retrained span model of Section 5.2.1 does not use centred windows at all: the answer position only labels its training chunks.

4.3 Data Collection

All experiments use the official CUAD release from The Atticus Project, downloaded from the Hugging Face hub ([theatticusproject/cuad](https://huggingface.co/theatticusproject/cuad)). The default `load_dataset()` option gives only the 511 original contract PDFs, which cannot be used for training. The marked data sits in two files in the repository, and each has a different job in our system:

- `master_clauses.csv`: a wide table with 83 columns and one row per contract, which we used to explore the data and build labels;
- `CUAD_v1.json`: the SQuAD-format file with the full contract text and the character positions of every answer, which we used to train the span model and to read whole contracts.

4.3.1 Data Cleaning

Experts prepared the CUAD files, so cleaning meant fixing format problems; the values themselves were sound. We handled four problems:

- Empty cells. Clause-text cells that are empty (NaN) are treated as an empty list, which means the category is absent.
- Badly formed lists. Each clause-text cell holds a Python list written as text. A cell that cannot be read as a list is kept as one clause instead of being thrown away, and blank entries are removed from every list.
- Inconsistent column names. The column name `Notice Period To Terminate Renewal- Answer` contains an extra space that breaks matching by name, so columns were paired by position instead (Section 4.3).
- Empty answers in the JSON file. Answers that are empty after removing spaces are ignored when building the span-training examples and the answer lists.

We removed no unusual values. Very long contracts are real and were kept, because handling length is exactly what the method must do.

4.3.2 Data Transformation

The CSV is wide: 41 categories $\times$ (a clause-text column plus an answer column), with a file-name column at the front. Turning it into a long table with one row per (contract, category) needed two decisions that were not obvious at first:

We paired columns by position. The column name `Notice Period To Terminate Renewal- Answer` contains an extra space, which silently breaks any matching of a category column to its answer column by name. Pairing by position (clause text in odd columns, answer in even columns) is immune to such typos.

We decided presence from the clause-text list and ignored the Yes/No answer. Each clause-text cell holds a Python list written as text, either `[]` or `['clause', ...]`, and a category is present when this list is not empty. The rule works the same way for the eight categories (dates, party names) whose answer column holds a value instead of Yes/No, and it also gives us the clause text directly.

This gives $510 \times 41 = 20,910$ labelled (contract, category) rows, of which 32.1% are positive.

Three more steps prepare the data for the models. Every contract is cut into 2,000-character windows that move forward 1,500 characters at a time, so neighbouring

windows overlap by 500 characters, more than a typical clause, and no clause falls between two windows. For the span model, CUAD’s character positions are turned into start and end token positions with the tokenizer’s offset mapping, inside a 1,200-character focus window around each answer so that it fits in 320 tokens. For the baseline, each whole contract becomes a TF–IDF vector of single words and word pairs with sublinear term frequency. Category labels stay simple present/absent values, and no other scaling is needed beyond what TF–IDF does.

4.3.3 Data Integration

The two CUAD files complement each other. `master_clauses.csv` gives one row per contract with the clause lists we used to explore the data and build labels, and `CUAD_v1.json` gives the full contract text, the 41 category questions and the character positions of every marked clause. All the modelling uses the JSON file: presence labels, answers and contract texts come from it, matched by contract title and by the category name taken from each question ID. We used the CSV to explore the data and to double-check how often each category appears. The split by contract is computed once over the 510 contract titles and applied the same way to every table made from the data.

4.3.4 Data Reduction

We reduced the data only to keep training possible on a laptop and to balance the classes, and never threw away information the app needs when it runs:

- Vocabulary limit for the baseline: the TF–IDF vocabulary keeps only the 20,000 most common single words and word pairs that appear in at least two contracts.
- Fewer negative examples: for window-level training, every positive window is kept, but only two negative windows are taken for each (contract, category) pair, giving 53,662 window examples with 39.2% positives.
- Training on a sample: the presence model was trained on a balanced sample of 24,000 windows, the span model on 8,000 of 11,156 examples (the retrained one on 6,000 clauses from the 327 fit contracts) and the summarizer on 4,000 of 11,156 pairs. Section 5.4.3 tests whether training the presence model on a sample hurt it.

4.3.5 Summary of Preprocessed Data

Four features of the data shaped the design decisions that followed:

1. Very uneven categories. How often a category appears ranges from *Document Name* at 100% and *Parties* at 99.8% down to *Source Code Escrow* at 2.5% and *Price Restrictions* at 2.9%. The median category appears in about 23% of contracts.

2. Almost no examples of the rarest categories. *Source Code Escrow* appears in only 13 contracts in the whole dataset. No model can learn such categories reliably, and their individual scores are mostly noise, which we keep in mind throughout the results chapter.
3. Long contracts. The median is 33,143 characters (about 5,000 words) and the maximum is 338,211 characters. The median is about 16 $\times$ what a transformer can read at once (about 2,000 characters), and the longest is about 170 $\times$, which makes length the central modelling problem of the project.
4. Short clauses. The median marked clause is 196 characters (90th percentile 587, 99th percentile 1,330). A clause easily fits inside one window; the hard part is finding it in a very long document.

The 510 contracts are the whole dataset. Marking them reportedly cost about \$2M of lawyers' time, so more cannot easily be added. This has three effects: we must start from pre-trained models, over-fitting is the main danger and so all our conclusions rest on test-set numbers, and the train/test split must be made by contract.

The 80/20 Train/Test Split

As our supervisor asked, we split the dataset 80/20 by contract. Each whole contract goes to one side or the other, so no clause from a training contract can appear in the test set. The split uses a fixed random seed (42), and all three models use exactly the same split, so "test" always means the same 102 unseen contracts. Table 4.2 summarizes the split, and the almost equal share of positives on both sides shows it is balanced. We saved the two sides as `data/train_80.csv` and `data/test_20.csv` so the split itself can be repeated.

Table 4.2: Train/test split

	Contracts	(Contract, category) rows	Positive rate
Train (80%)	408	16,728	32.0%
Test (20%)	102	4,182	32.2%
Total	510	20,910	32.1%

The overall share of positives in Table 4.2 is almost the same on both sides (32.0% vs. 32.2%). One pair of numbers can still hide a badly uneven split, since the two sides can match overall and differ a lot category by category, so Figure 4.8 compares them for all 41 categories at once. They match closely (Pearson $r = 0.986$, average difference 3.4 percentage points). The clear exception is *Termination For Convenience*, which appears in 47.1% of test contracts but only 33.1% of training contracts, a 14.0-point gap. The other differences are all in the rarest categories, where moving one or two contracts changes the percentage by several points. The split was made once at random and never changed to look better, so we report these gaps as they are. They matter in Chapter 5, because a category that is more common in the test set counts more in the micro-averaged score than its training frequency would suggest.

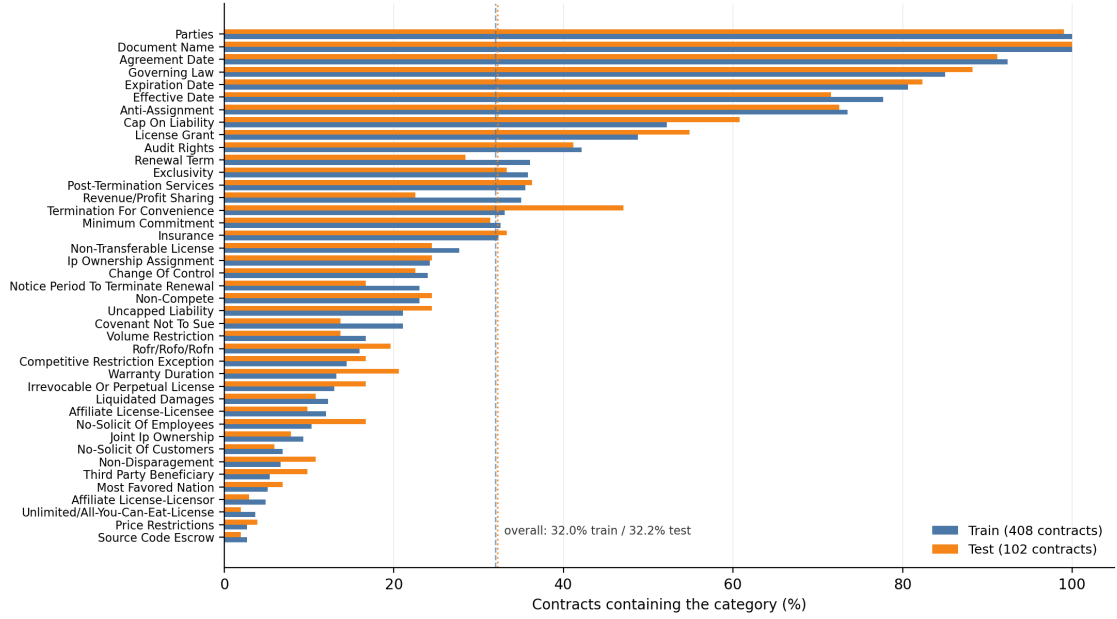


Figure 4.8: Per-category split balance

4.4 Implementation of Selected Design

We built the chosen design in Python as two Jupyter notebooks, `train.ipynb` for preparing data and training and `test.ipynb` for testing on its own, plus some analysis scripts, and then turned it into the Clausify web app. This section describes how each model was trained and how the app works.

All models were trained only on the 408 training contracts (80%). Training ran on one Apple-Silicon laptop with the MPS (Metal) backend, `torch` 2.8 and `transformers` 5.x. The summarizer ran on the CPU for the memory reasons explained in Section 4.4.6. Table 4.3 lists the settings.

Table 4.3: Training configurations

	Presence (MIL)	Span (QA)	Summarizer
Backbone	DistilBERT-base	DistilBERT-base	FLAN-T5-small
Task head	Sequence classification	QA (start/end)	Seq2seq LM
Training examples	24,000 (of 53,662)	8,000 (of 11,156)	4,000 (of 11,156)
Max input length	256 tokens (512 in the re-run)	320 tokens	256 in / 48 out
Epochs	2	2	2
Batch size	16	16	8
Learning rate	2×10^{-5}	3×10^{-5}	3×10^{-4}
Device	MPS	MPS	CPU
Wall-clock	≈ 50 – 52 min	≈ 23 – 26 min	≈ 8 – 13 min

Note. The re-run of Section 5.3.3 changes three presence settings: 512 tokens, three epochs with the best one kept on validation, and training windows drawn from 327 of the 408 training contracts. Its training took 191 minutes. The retrained span model of Section 5.2.1 uses 11,967 chunks of 1,200 characters (6,000 clauses), 384 tokens and two epochs, and took 46.2 minutes.

4.4.1 How We Validated, and Where the Settings Came From

If any setting in Table 4.3 had been picked by trying several options and keeping the one that scored best on the 20% test set, every number in Chapter 5 would be too optimistic and the test set would no longer be unseen. This section gives the source of each setting, including one place where we fell short.

What was tuned, and what was not

Nothing in Table 4.3 was tuned. Every setting is a standard default for the model or a value fixed by reasoning before training, and no setting was ever compared with another on the test set. In detail:

- The learning rates, batch sizes and epochs (2×10^{-5} , 3×10^{-5} , 3×10^{-4} ; 16/16/8; 2 epochs) are the standard fine-tuning defaults for DistilBERT and T5 models. We ran each setting once and did no search. This protects the test set, but it also means we only know these values are common, not that they are good.
- The window of 2,000 characters with a step of 1,500 comes from a measurement on the training data: the 500-character overlap must be longer than a typical clause (196 characters) so that no clause falls between windows (Section 4.2.1). Any window and step that meet this rule would work, and we did not look for the best pair.
- Two negative windows per contract and category balance the classes, keeping about 39% positives instead of letting almost all windows be negative. This was not tuned either.
- Taking the maximum window score was chosen because it is the multiple-instance rule. We did not try other rules at first; Section 5.3.1 does so afterwards.
- The 0.5 threshold is the default boundary for a two-class model. Nobody chose, checked or changed it, and Section 5.3.2 later shows that it is a poor choice for the window-based transformer.

This project avoided tuning on the test set by not tuning at all. That guarantees the test scores are fair, and it also limits them: the transformer may lose to the baseline partly because nobody looked for better settings for it. Section 5.3.3 confirms this. With the whole window read and both models tuned on a validation split, the gap closes.

The validation sets actually used

Table 4.4 shows, for each model, what data was kept aside during training and what it was used for.

Table 4.4: Validation arrangements

Model	Validation data	Use
Presence (MIL)	5% of the <i>training</i> windows, split off before training	Monitoring only. Clean: no test contract contributes.
Span (QA)	None (<code>eval_strategy="no"</code>)	No validation loss exists for this model.
Summarizer (deployed checkpoint)	None	The checkpoint used in every reported result and in Clausify was trained with no evaluation set.

Note. “Monitoring only” means the loss was printed and plotted but never used to select a checkpoint, stop early, or change a setting: all three runs used `save_strategy="no"` and kept the final-step weights.

One thing we got wrong

The first exploratory run of the summarizer in the notebook, whose loss curve appeared in earlier drafts of this report, monitored training on 300 clauses taken from the test split. That was the wrong data to monitor with, and we should not have done it.

Two facts limit the damage. That run kept no checkpoints (`save_strategy="no"`) and had no early stopping, so the numbers it printed could not have been used to choose a model, and nothing about the model depended on them. The summarizer used for all results was also trained again by `build_artifacts.py` with no evaluation set at all, and that model produced every summarizer result in Chapter 5. No reported score comes from a model chosen with test data.

The “validation” loss printed by that run ($0.096 \rightarrow 0.075$) was really a loss on test clauses and should be ignored. The October 2026 re-training rebuilt the summarizer exactly as it is used, with no evaluation set, and Figure 4.11 now shows that run’s training loss, so no test data appears in it. We kept this mistake in the report because it is easy to make: with a small dataset and three models, it is tempting to grab “the other split” when a validation set is needed. Writing down, for each model, exactly what was kept aside catches it, and Table 4.4 now does that.

4.4.2 Optimization Schedule

Table 4.3 gives the highest learning rate of each model, but the schedule that lowers it over time matters as much. All three models use the same optimizer setup: AdamW with a linear decay schedule, no warm-up, and gradient clipping at 1.0. Figure 4.9 plots the three schedules. We read the presence and span settings from the `training_args.bin` file saved next to each model, so the plot shows what actually ran.

Two of these choices need a reason. Skipping warm-up is usually risky, but these are short fine-tuning runs (1,000–3,000 steps) that start from pre-trained weights with

small learning rates. Warm-up mainly keeps the first updates of a randomly started model stable, which hardly applies here, and the smooth loss curves in Figures 4.10 and 4.11 show no early problems. Gradient clipping at 1.0 stayed on throughout. Clipping is important on the MPS backend, because it stops a single bad batch from causing the kind of breakdown we saw with DeBERTa-v3 (Section 4.4.6).

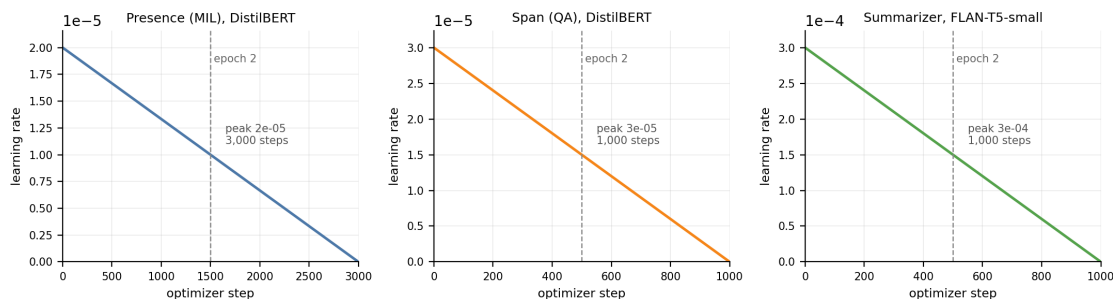


Figure 4.9: Learning-rate schedules

A record of the gradient size during training shows directly whether training stayed healthy, instead of leaving us to infer it from the loss. The original August runs did not keep it, because their temporary checkpoint folders were deleted after training. The October 2026 re-training logged it every 100 steps for all three models, and so did the August full-data presence run in Section 5.4.3.

These logs show something the loss curves do not. In the October runs the gradient size ranges from 1.45 to 10.60 (median 4.30) for the presence model and from 10.11 to 20.92 (median 12.72) for the span model, so every logged step of both runs is above the clipping limit of 1.0. The August full-data presence run looks the same: across its 63 logged points the gradient size ranges from 0.55 to 11.96 with a median of 4.65, and 98.4% of the points are above 1.0. The summarizer’s gradients are much smaller (0.55 to 1.38, median 1.07, with 70% of points above 1.0). Nothing broke in any run: there are no NaN values or large spikes, and the losses fall smoothly. For the two DistilBERT models, however, clipping was active on almost every step, so the update size was cut back all the time and the learning rates in Table 4.3 overstate the real step size. This weakens the warm-up argument above a little, since part of what kept the early updates stable was almost certainly the clipping as well as the small learning rate. Our smooth loss curves are not good evidence that skipping warm-up is safe in general.

4.4.3 Training the Presence Classifier

The TF-IDF + logistic-regression baseline trains in under a minute. On the 20% test set it reaches micro-F1 0.775, weighted-F1 0.777, macro-F1 0.572 over all 41 categories, and 0.666 over the 34 categories with at least ten test positives. The gap between micro and macro scores comes from the rare categories with very few examples.

The window-level DistilBERT was trained on a balanced sample of 24,000 window examples (39.2% positives) to keep training manageable on a laptop. In the October 2026 re-training it took 49.8 minutes. The training loss fell smoothly from 0.57 after

the first 100 steps to 0.26 at the end, and the validation loss went from 0.275 after the first epoch to 0.254 after the second (Figure 4.10). The validation loss stayed close to the training loss, so we saw no sign of over-fitting.

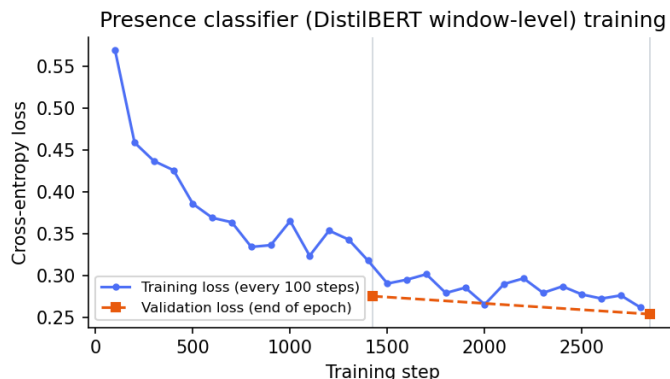


Figure 4.10: Presence classifier loss

4.4.4 Training the Span Extractor

The question-answering model was trained on 8,000 focus-window examples for two epochs. Before training, we turn the labelled token positions back into text for a few examples and check that they match the marked answers. In SQuAD-style systems the character-to-token conversion is easy to get wrong without noticing, and this small check catches that.

4.4.5 Training the Summarizer

FLAN-T5-small was fine-tuned on 4,000 clause-template pairs on the CPU. This took about 8 minutes originally and 12.7 minutes in the October 2026 re-training, which ran while the laptop was busy with other jobs. The training loss dropped quickly, from 1.17 at step 100 to 0.08 by the end of the second epoch (Figure 4.11). There is no validation curve because the model used in the app is trained with no evaluation set (Table 4.4). The re-trained model scores ROUGE-L 0.775 against the templates on the 150 test clauses, the same as the original. In hindsight, the fast drop was the first sign of the template copying we analyze in Chapter 5: with only 41 different targets, the task is easy to learn.

4.4.6 Engineering Notes: Training on Consumer Hardware

A few practical lessons may save time for anyone repeating this work on Apple-Silicon computers:

- DeBERTa-v3 is unstable on MPS. Our first-choice model produced NaN (broken) gradients on the MPS backend and was far too slow on the CPU (about 89 seconds per step). We switched to DistilBERT; the method stays the same, and on a CUDA machine the model name can simply be changed back.

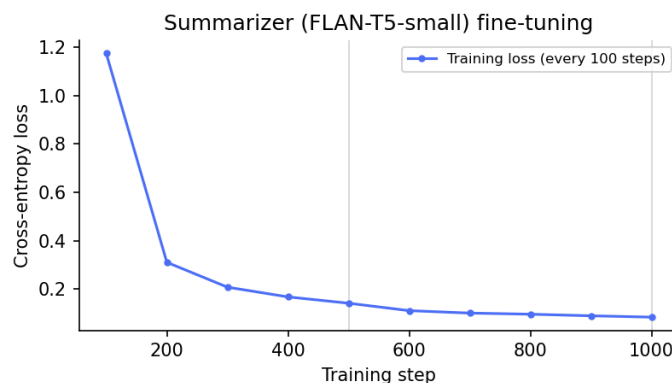


Figure 4.11: Summarizer loss

- MPS memory is not freed inside one program. After two transformer trainings in the same session, the summarizer ran out of MPS memory even after we deleted objects and cleared the cache. What actually works is restarting the kernel or training the (small) summarizer on the CPU, which is what we did.
- Library changes. `transformers` 5.x renamed `Trainer(tokenizer=...)` to `processing_class=...`. We also turned off mixed precision (`fp16=bf16=False`) and converted the loss logits to fp32 to avoid data-type problems on MPS.
- Repeatability. Everything needed (the split in `artifacts/split.json`, the baseline in `artifacts/baseline.pkl`, and the three trained models) is saved to disk, and a separate test notebook recreates every number in Chapter 5 from those files alone.

4.4.7 Measured: What Sparse Attention Would Have Cost

Section 2.1.2 argued against sparse attention partly because of cost, and a cost argument is weak until the cost is measured. The laptop used for this project is exactly the machine in question, so we measured it.

We built Longformer-base at its published size (12 layers, 768 hidden units, 4,098 position embeddings, attention window 512, 148.7M parameters). We timed one forward pass at 4,096 tokens and one full training step (forward pass, backward pass and AdamW update), and compared them with the DistilBERT setup this project used. The weights are random, which does not matter here, because time and memory depend on the size of the tensors and not on their values. Each setup ran in its own program, so the peak memory belongs to that setup alone. The machine is an Apple M4 with 16 GB of shared memory, which the operating system also uses. Table 4.5 shows the results.

This measurement corrected us. Before measuring, we believed that fine-tuning a Longformer at 4,096 tokens was not possible on one laptop (Section 2.1.2). At batch size 1 it fits, using 14.71 GB on a 16 GB machine. That leaves less than 1.5 GB for the operating system, but the model, its gradients and its optimizer state all fit in memory and the step finishes.

Table 4.5: Longformer vs. DistilBERT cost

Model	Operation	Batch	Time	Peak memory
Longformer-base (148.7M)	forward, 4,096 tok	1	3.69 s	2.18 GB
DistilBERT (67.0M)	forward, 256 tok	16	0.19 s	1.15 GB
Longformer-base	training step, 4,096 tok	1	258 s	14.71 GB
Longformer-base	training step, 4,096 tok	2	2,170 s	20.75 GB
DistilBERT	training step, 256 tok	16	3.17 s	4.47 GB

Note. Measured on the Apple M4 (16 GB unified memory) that trained every model reported in this project.

One step further, the memory claim holds. At batch size 2 the memory needed rises to 20.75 GB, more than the machine has, and the timing shows it: the step takes 2,170 seconds instead of the roughly 516 that doubling the batch should take. Being 8.4 times slower for twice the work means the computer is swapping memory to disk. The accurate claim is that Longformer fits only at batch size 1, and any batch size a real training run would use does not fit.

Time is the real limit. At 258 seconds per step and batch size 1, repeating our two-epoch presence training (22,800 training windows, 45,600 examples seen in total) would take about 136 days of non-stop computing. The DistilBERT run it would replace took about 50 minutes. Short tests add start-up time (our own DistilBERT step takes 3.17 s here but about 1.0 s during the real run), so these numbers may be about three times too high. Even then, a third of 136 days is still 45 days, and the project had weeks.

Measurement also confirms the cost when the app runs. Scanning one typical contract (33,143 characters) for all 41 categories takes $22 \text{ windows} \times 41 \times 11.7 \text{ ms} \approx 10.6 \text{ seconds}$ with our DistilBERT, against $3 \text{ windows} \times 41 \times 3.69 \text{ s} \approx 454 \text{ seconds}$ (seven and a half minutes) with Longformer. Clausify is meant to respond within seconds, so this rules Longformer out for the app, whatever a large training computer could do.

None of this shows that a Longformer would have scored worse. It shows only that we could not have found out on this hardware, a much narrower claim than the one we started with. Settling the modelling question needs a CUDA machine, and we list it as future work.

4.4.8 End-to-End Data Flow

Figure 4.1 names the steps, and Figure 4.12 adds what passes between them. The sizes shown are for one contract of typical length.

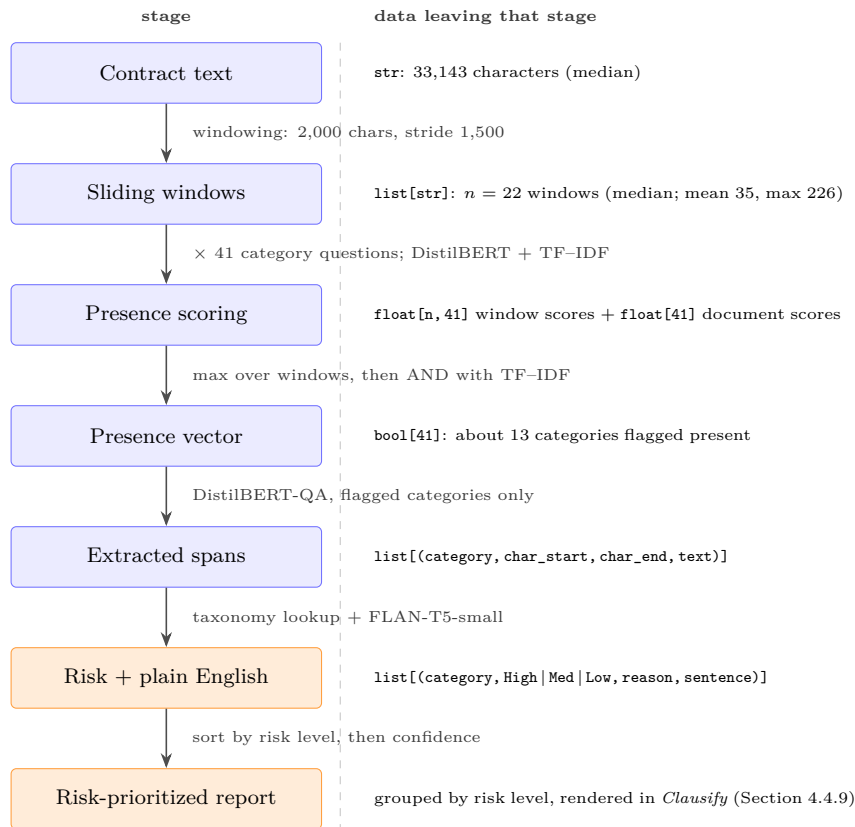


Figure 4.12: Pipeline data flow

4.4.9 The Clausify Application

Purpose and Design

The final result of the project is *Clausify*, a Streamlit web app that runs all of the project’s trained models on any contract the user gives it (the re-run DistilBERT presence model and its TF-IDF partner from Section 5.3.3, the span model, the FLAN-T5 summarizer, and the clause classifier for clause mode) and shows the results as a report sorted by risk. It is the system of Figure 4.1 with an interface. The design goal is that a non-lawyer should see the “watch out before signing” items first. Clauses found are grouped **High** → **Medium** → **Low** and sorted by model score within each group.

User Workflow

1. Add a contract. The user pastes the text or uploads a .pdf, .docx or .txt file, and the text is read on the user’s own computer (Figure 4.13). The user picks *Whole contract*, *Separate clauses* (Section 4.4.9) or *Auto*. A settings panel chooses the detection rule and turns the plain-English lines on or off. The rules are the three re-run setups of Section 5.3.3, with the thresholds chosen there: the recall-first transformer (the default), the recall-first ensemble and the balanced AND-ensemble.

2. Analyze. One click runs all the models, and a progress bar follows the scan of the 41 categories and then the summarizer.
3. Read the report. A summary bar counts the clauses found at each risk level (Figure 4.15), and each clause appears below it as a card. On the made-up test contract the bar shows 5 High, 13 Medium and 9 Low risk clauses, so 27 of the 41 categories are found (re-run model, recall-first default).

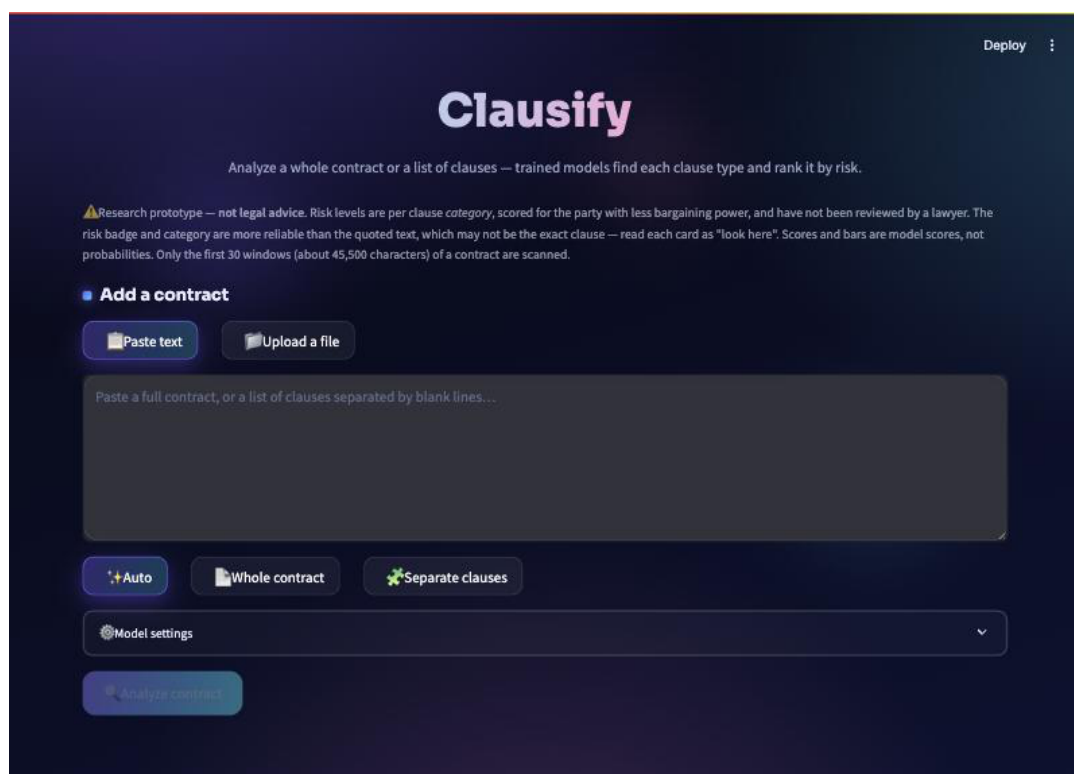


Figure 4.13: Clausify input screen

Each clause card shows the category name with a coloured risk badge, the one-line reason why that category is risky (from the risk table in Appendix A), the summarizer’s plain-English sentence with a warning when it belongs to another category, a bar showing the calibrated chance that the clause type is present, with the DistilBERT score and the threshold set for that clause type, and the clause quoted from the user’s own contract with the span model’s answer highlighted (Figure 4.16). A card whose calibrated chance is below 50% is labelled *Possible — check* and listed after the other cards of its risk level (Section 5.6.2). Below the cards the app adds three more views:

- A highlighted contract: the full text, with every clause found marked in its risk colour and labelled with its category. Each card links to its place in the text.
- A missing-protection checklist: protective clauses the models did not find, namely a limit on liability, termination for convenience, governing law, an expiry date, a warranty period, insurance and, when the contract renews itself, a notice period to stop the renewal. Each comes with a sentence on why its

absence matters, and the view notes that “not detected” can also mean the model missed it.

- A downloadable report: a PDF (summary, every clause with its reason, plain-English line, scores and quoted text, and the checklist) and a CSV file with the same rows.

The screenshot displays a software interface for reviewing a contract. At the top, a header bar contains the text "Your contract, highlighted by risk" on the left and "Deploy" with a menu icon on the right. Below the header, the contract text is presented with several sections highlighted in yellow and labeled with red text boxes on the right:

- 1. DEFINITIONS.** "Territory" means the United States and Canada. "Confidential Information" means any non-public information disclosed by one Party to the other. "Affiliate" means any entity controlling, controlled by, or under common control with a Party.
- 2. LICENSE GRANT.** Subject to the terms of this Agreement, Licensor hereby grants to Distributor a non-exclusive, non-transferable license to market, promote, and distribute the Software to end users within the Territory during the Term. Distributor shall have no right to sublicense, assign, or otherwise transfer the license granted herein except as expressly permitted in writing by Licensor. This license is limited and does not grant Distributor any ownership interest in the Software. *Non-Transferable License*
- 3. EXCLUSIVITY.** During the Term, Licensor agrees that it shall not appoint any other distributor for the Software within the metropolitan New York region, and Distributor shall be the sole and exclusive distributor of the Software in that region. *Exclusivity*
- 4. TERM AND RENEWAL.** This Agreement shall commence on the Effective Date and shall continue in full force and effect for an initial term of three (3) years (the "Initial Term"), expiring on March 13, 2026, unless earlier terminated in accordance with Section 12. Thereafter, this Agreement shall automatically renew for successive one (1) year renewal terms unless either Party provides written notice of non-renewal at least ninety (90) days prior to the end of the then-current term. *Renewal Term*
- 5. INTELLECTUAL PROPERTY OWNERSHIP.** All right, title, and interest in and to the Software, including all improvements, modifications, and derivative works thereof, shall be and remain the sole and exclusive property of Licensor. Any developments or enhancements created by Distributor

Below the contract text, a section titled "Missing-protection checklist" contains a list of items:

- ✓ **Cap On Liability — found**
- ✓ **Termination For Convenience — found**
- ✓ **Governing Law — found**
- ✓ **Expiration Date — found**
- ⚠ **Warranty Duration — not detected**
No stated warranty period — it may be unclear how long you are protected against defects.

Figure 4.14: Highlighted contract and checklist

Figure 4.14 shows the highlighted view and the checklist on the made-up test contract. The renewal paragraph is labelled *Renewal Term*; the first setup’s model labelled it *Minimum Commitment*, which was wrong. The checklist reports the warranty period as not detected although Section 10 of the contract sets one (90 days), and Figure 4.15 shows a false alarm: the liability clause, which caps liability, is reported as *Uncapped Liability*. The summarizer’s warning on that card points to the right category. We kept both mistakes in the figures instead of choosing cleaner examples.

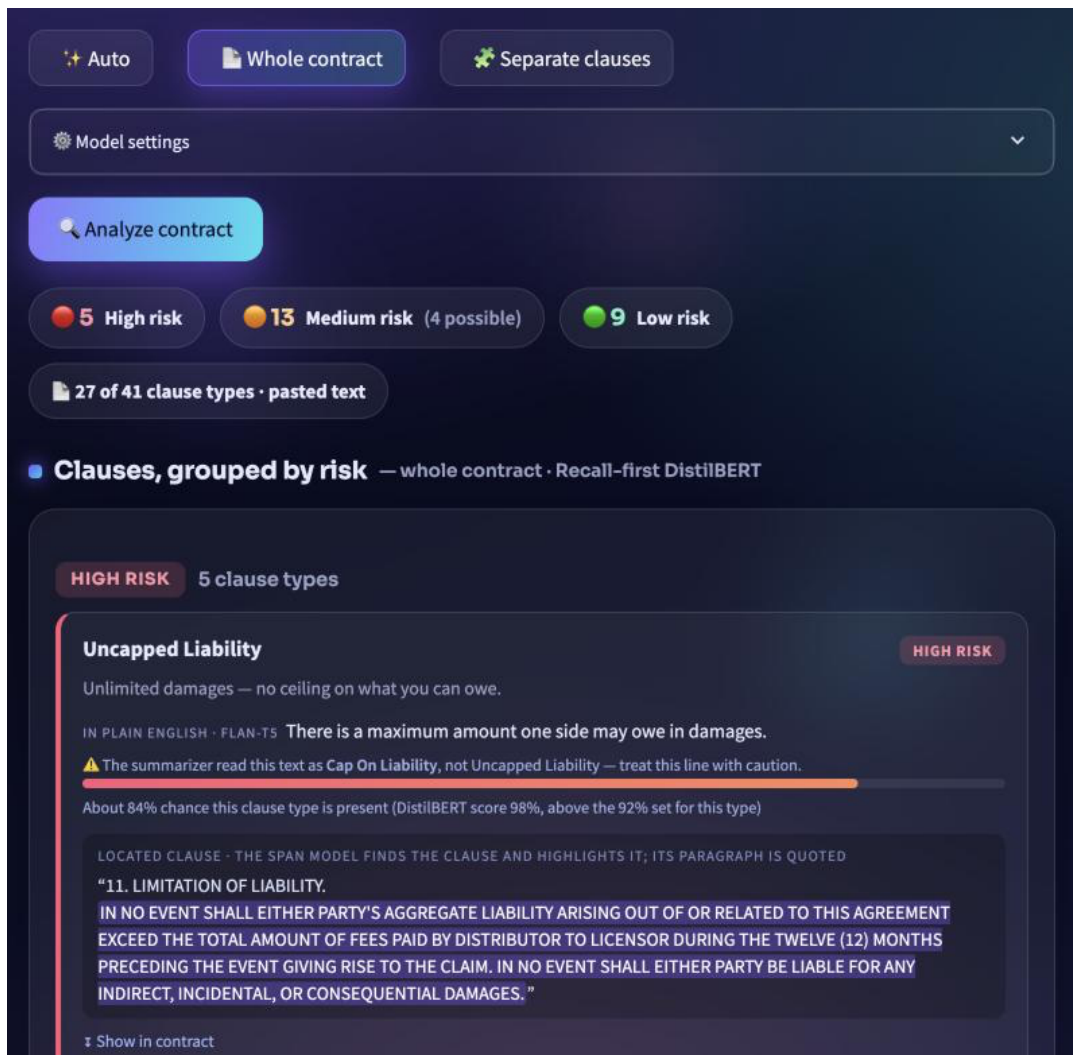


Figure 4.15: Clausify results view

How the Models Run Inside the App

The app follows the same steps that were used in testing:

1. Windows. The contract is cut into 2,000-character windows that move forward 1,500 characters at a time, as in training. Every clause type is checked in the first 30 windows. In a longer contract, the re-run TF-IDF model, applied to each later window on its own, picks the 5 windows most likely to contain each clause type, and those are checked too. This keeps long contracts fast while finding most of their clauses (Section 5.6.2).
2. Model 1, presence. For each of the 41 categories, the exact CUAD question the model was trained on is paired with each of those windows and scored by the re-run DistilBERT, which reads the whole window (512 tokens). The score for the whole contract is the highest window score. A category is reported as present when its score reaches the threshold chosen for it on the validation split, and the best window is remembered.

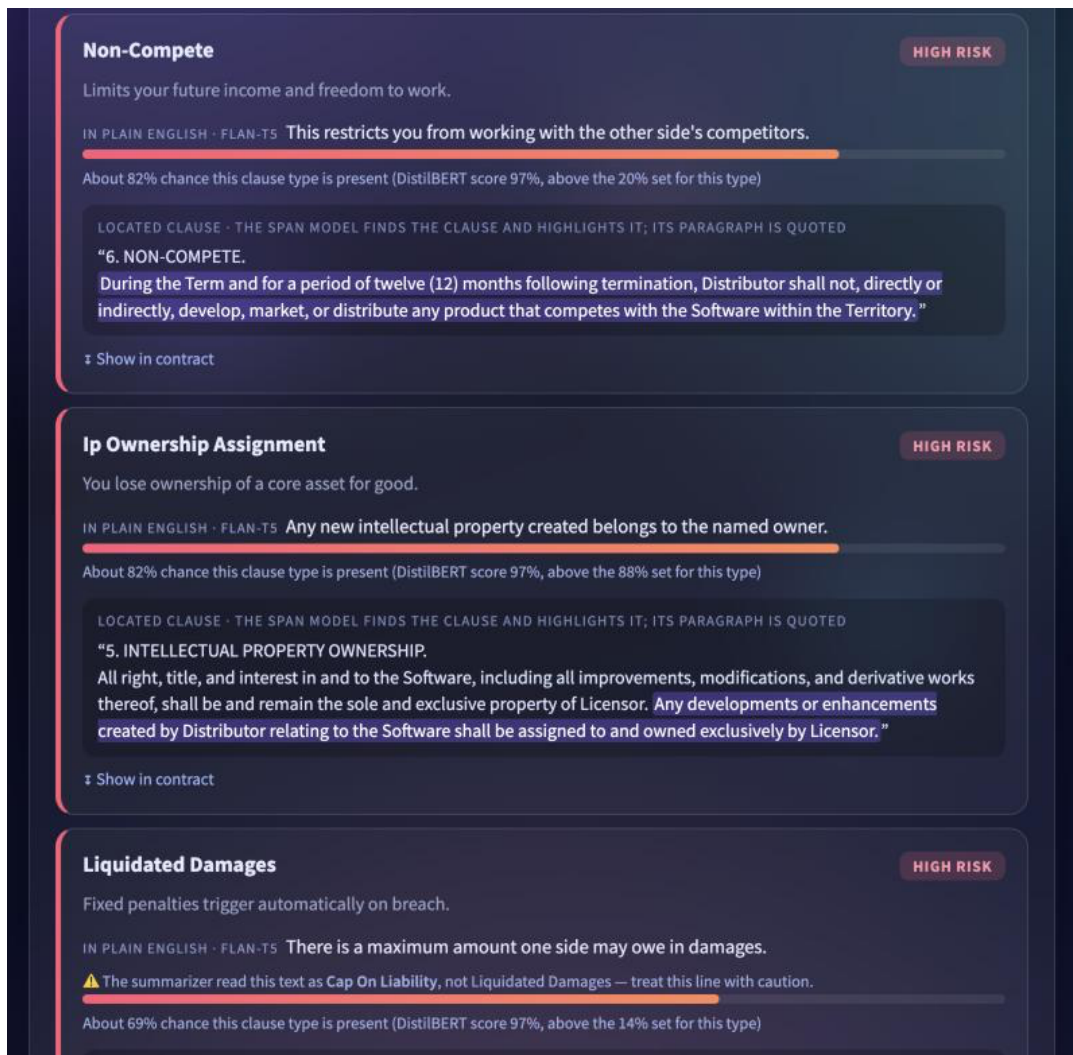


Figure 4.16: High-risk clause cards

3. Finding the clause. Inside that best window, the retrained span model (Section 5.2.1) reads the window in 1,200-character chunks, as in its training, and returns its answer. The card quotes the whole paragraph that contains the answer and highlights the answer inside it. Earlier versions of the app could not trust the first span model this far: on test inputs it returned the same sentence for almost every category, because it preferred text near the position where answers sat during its training (Section 5.2.1). They quoted the paragraph the presence model scored highest instead, which Section 5.2.1 compares with the new method on the test set.
4. Optional ensembles. With these settings the re-run TF-IDF model also scores the whole contract. The recall-first ensemble reports a category when the average of the two scores reaches that category's threshold, and the balanced ensemble when both scores reach their own thresholds, exactly as tested in Section 5.3.3. The thresholds are stored in `decision_v2.json`, which the training code writes and the app reads, so the app cannot drift from the tested setup.

5. Risk layer. The category found is looked up in the fixed risk table (8 High, 22 Medium, 11 Low) for its level and reason.
6. Summarizer. FLAN-T5-small receives each quoted clause with the same prompt and settings as in testing and returns one plain-English sentence.

All the models run on the CPU, so the app works on any computer, and a typical contract takes about a minute once the models are loaded (Table 4.6). The app loads the trained models saved by the training code (`presence_v2`, `span_v2`, `summarizer` and the re-run TF-IDF model, with the first setup’s models as a fallback). When they are not on the computer, as on a cloud server, it downloads the same files once from a public Hugging Face model repository.

Implementation Notes

The app is a single-page Streamlit program (`app.py`) built on two small modules. `model_utils.py` loads the models and handles windows, scoring, clause finding, the summarizer and the risk table, and `features.py` reads uploaded files, builds the highlighted view and creates the reports. Two test contracts come with the app: a real CUAD contract, and a made-up “Master Software License and Distribution Agreement” that we wrote to cover many of the 41 categories. On the made-up contract the app finds clauses at all three risk levels and puts the four High-risk clause types that the contract’s section headings name (*Exclusivity*, *IP Ownership Assignment*, *Non-Compete* and *Liquidated Damages*) at the top of the report, which are the before-you-sign warnings the design aims for. It also reports one High-risk type the contract does not contain (Section 5.3.3).

Clause-by-Clause Mode

Users often paste a few clauses instead of a whole contract and expect one card per clause. Whole-contract mode does not work that way. For each of the 41 categories it asks whether the category appears anywhere in the window, so six related clauses pasted together gave fourteen categories, eight of them wrong. The app therefore has a second mode, chosen automatically for a short list of paragraphs or chosen by the user, in which each paragraph is one clause and gets exactly one category and one risk level. When most paragraphs are numbered (“1.”, “4.2”, “(a)”, “Section 5”), only the numbered ones are treated as clauses. The title, opening paragraph and signature lines are listed separately as skipped, so a contract with 18 numbered clauses gives exactly 18 cards, numbered as in the contract.

In this mode, if a clause heading names a category (“2. EXCLUSIVITY.”), the heading decides the category directly, and headings that do not clearly name one category are ignored. Otherwise a model decides. The first version reused the presence model: it scored the clause against all 41 category questions and adjusted each score by how that category scores on real CUAD clauses of other categories (fitted on 1,586 training clauses), because the raw scores cannot be compared across categories; *Parties*, for example, scores about 0.98 on almost any clause that names the two sides. On 721 clauses from the test split, that gave the right category out

of 41 for 44.9% of clauses (5.1% with the raw scores), the right one in the top three 67.1% of the time and the right risk level 69.9% of the time, and it left 25.8% of clauses unrecognized. The presence model was trained to scan 2,000-character windows, not to label single clauses, so we trained a model for the task itself.

The clause classifier learns from the 7,464 annotated clauses of the 327 contracts the re-run presence model was trained on, each labelled with its category, plus a 42nd class, *none*, made of 2,000 lines of the same contracts that overlap no annotated clause (definitions, notices, signature blocks). We compared a TF-IDF model with logistic regression, DistilBERT fine-tuned for the 42 classes (three epochs, 44.7 minutes) and the average of the two, on a balanced sample of the 81 validation contracts (20 clauses per category, as in the test). The word model was best on validation (69.9%, against 59.3% for DistilBERT and 62.9% for the average), so the app uses it. On the same 721 test clauses it gives the right category 72.1% of the time, against 44.9% before, the right one in its top three 90.3% of the time, and the right risk level 83.1% of the time. Counting any category the same text is also annotated with, it is right 82.5% of the time. It calls only 1.2% of real clauses *none*, against the 25.8% the first version left unrecognized, but the *none* class is weak in the other direction: of 200 test lines that belong to no clause, it recognises 49%, so about half of such paragraphs still get a category. A paragraph the classifier reads as *none* is shown as unrecognized. The app shows these numbers under every clause-mode result.

A word model beating DistilBERT here matches the first presence comparison, before the transformer read whole windows (Section 5.4.1): clause categories are marked by distinctive legal wording. Our DistilBERT was still improving after three epochs (validation accuracy 46.1%, 54.7% and 59.3%), so a longer run might close the gap. On the nine-clause test text of the app, the classifier agrees with the heading on 8 of 9 clauses, and with the headings removed it still gets 8 of 9 categories and risk levels right. Its one miss is the confusion the categories invite: an *Uncapped Liability* clause read as *Cap on Liability*, which turns a High-risk card into a Medium-risk one. Headings remain the most reliable signal, and cards decided by the model alone should be checked against the clause text.

Measured Latency

Chapter 2 explained that we chose a small model partly because the app must respond quickly. Section 4.4.7 measured what the alternative would have cost. Here we measure how fast our app is, running the app's own code on test contracts of different lengths.

A typical contract takes about a minute (74.7 seconds for the median test contract in this run), roughly twice the first setup's 30.1 seconds, because the presence model now reads twice as much text in every window. The 5 extra windows per clause type add about 15% on a long contract: in alternating runs with and without them, the 75th-percentile contract took 111.8 against 95.5 seconds on average, and the longest 119.0 against 104.1 seconds. It is the price of the accuracy gain in Section 5.3.3. Our first guess of 5–15 seconds came from watching the demo on short inputs.

Table 4.6: Application latency

Contract	Characters	Windows per type	Time		Peak RSS
			First setup	Current app	
Shortest in test set	1,283	1	2.7 s	2.0 s	1.16 GB
25th percentile	18,104	12	15.0 s	36.1 s	1.73 GB
Median	35,136	24	30.1 s	74.7 s	1.91 GB
75th percentile	65,547	35	32.7 s	115.4 s	2.07 GB
Longest in test set	289,615	35	34.8 s	116.2 s	2.07 GB
<i>Current app: median 74.7 s mean 68.9 s worst case 116.2 s</i>					

Note. Measured in October 2026 through the application’s own `analyze()` entry point with its default detection rule, on the same machine with nothing else running. First setup: the 256-token presence model. Current app: the re-run 512-token presence model of Section 5.3.3, which checks every clause type in the first 30 windows and in 5 later windows picked by the TF-IDF model (Section 5.6.2), and the retrained span model of Section 5.2.1; the first setup read at most 30 windows. The two longest contracts have 44 and 193 windows. Repeated runs of the same contracts on this laptop vary by up to about a quarter (another run of the current app gave a median of 58.0 s and a worst case of 143.1 s). CPU only, Apple M4, models already loaded (loading adds 0.5 s). Peak RSS is the memory of the whole Python process for the current app.

Two design effects show in the numbers. Time grows in proportion to the number of windows, because most of the work is 41 categories $\times$ windows passes through DistilBERT, with each (category, window) pair scored once and nothing reused between categories. The window limit keeps the worst case short: every clause type is checked in at most 35 windows, so the 75th-percentile contract and the longest one take about the same time although one is four times longer. Without the limit the longest test contract would take more than ten minutes. The limit has a cost in recall that no other Chapter 5 number shows, because those tests read the full document. Reading only the first 30 windows lost about one High-risk clause in seven, and the 5 windows the TF-IDF model picks win most of it back (Section 5.6.2).

Chapter 5

Result Analysis

5.1 Performance Evaluation

Each step gets the score that suits its task. Deciding *which clauses are present* is a yes/no decision for each of the 4,182 (contract, category) test pairs, and we score it with micro-F1 (every decision counts the same), macro-F1 (every category counts the same), and the accuracy, precision, recall and specificity from the confusion matrix. *Finding the clause text* is scored with token-level F1, exact match, and the share of answers that overlap the real clause with $F1 \geq 0.5$. *Summaries* are scored with ROUGE-L against two different sets of reference sentences. For the *whole system* we count the share of real clauses that get through every step, and for the risk layer we count how many clauses it finds at each risk level. Speed is the time and memory the app needs (Table 4.6).

Every model is tested once on the same 102 test contracts, with a separate notebook that loads the saved models from disk. We measure uncertainty with a bootstrap over contracts. We also check whether the main result moves when we change the amount of training data, the random seed, the way window scores are combined, the decision threshold or the train/test split.

All results in this chapter come from the 20% test set, which is the same 102 unseen contracts for every model. The numbers are from the October 2026 re-training (Chapter 4) unless a run is marked as August 2026. A separate test notebook produced them by loading the trained models from disk, so nothing left over from training can affect them.

The chapter reports the presence models in two stages. Most sections use the first setup of Chapter 4, in which the transformer reads 256 tokens per window and no setting is tuned. Section 5.3.3 then fixes a flaw in that setup (the transformer read only about half of each window), tunes both models on a validation split, and tests them again. The re-run changes the main presence result, and where an earlier section's conclusion no longer holds, we say so there.

5.1.1 Presence Classification

Model comparison

Table 5.1 and Figure 5.1 compare the presence models on the 4,182 (contract, category) test decisions. On its own, the window-based transformer loses to the simple TF-IDF model that reads the whole document (0.692 vs. 0.775). Legal wording is distinctive, so word counts over the whole contract carry a lot of signal, and taking the highest window score raises too many false alarms, as Section 4.2.1 predicted. The AND-ensemble, which reports a clause only when both models agree, removes many of these false alarms and comes first (0.779). Its lead over the simple model is small for 102 test contracts, and Section 5.4.1 shows that the gap cannot be told apart from zero. A simple word model is already strong at deciding which clauses a contract contains. In this first setup the transformer helps most as a filter that removes false alarms. Section 5.3.3 shows that much of its loss came from the setup itself: once it reads the whole window and both models are tuned on a validation split, the two are tied on micro-F1 and the ensemble leads the baseline by a margin the bootstrap confirms.

Table 5.1: Presence model comparison

Model	micro-F1
Ensemble AND (both must agree)	0.779
TF-IDF baseline (full document)	0.775
Ensemble AVG (mean probability)	0.718
Transformer (max-pool) alone	0.692
Ensemble OR (either fires)	0.692
<i>August 2026 control run, Section 5.4.3:</i>	
Transformer trained on all 53,662 windows	0.694
Ensemble AND on that transformer	0.782

Note. micro-F1 over all 4,182 test decisions. The transformer (0.6924) is just ahead of OR (0.6916).

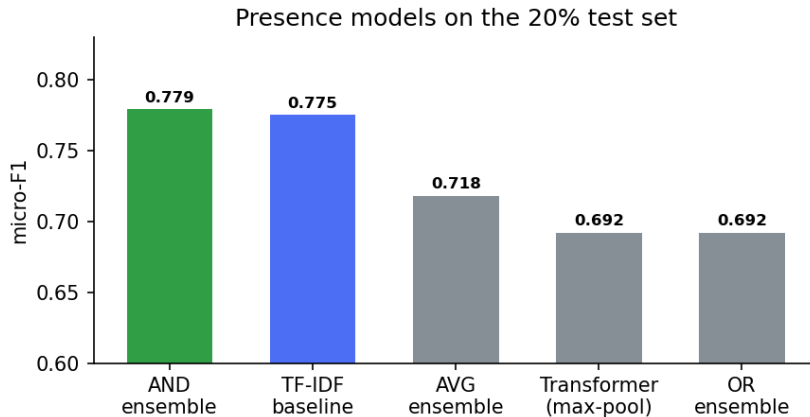


Figure 5.1: Presence model micro-F1

Confusion-matrix analysis

Our test plan asked for a full confusion-matrix analysis of the final model. Table 5.2 gives the matrix for the AND-ensemble over all test decisions, Figure 5.2 shows it as a heatmap, and Table 5.3 lists the scores taken from it.

Table 5.2: AND-ensemble confusion matrix

	Predicted Absent	Predicted Present
Actually Absent	TN = 2,525	FP = 309
Actually Present	FN = 291	TP = 1,057

Table 5.3: Confusion-matrix metrics

Metric	Formula	Value
Accuracy	$(TP + TN)/\text{all}$	85.65%
Precision	$TP/(TP + FP)$	77.38%
Recall (Sensitivity)	$TP/(TP + FN)$	78.41%
Specificity	$TN/(TN + FP)$	89.10%
F1 score	$2PR/(P + R)$	0.7789

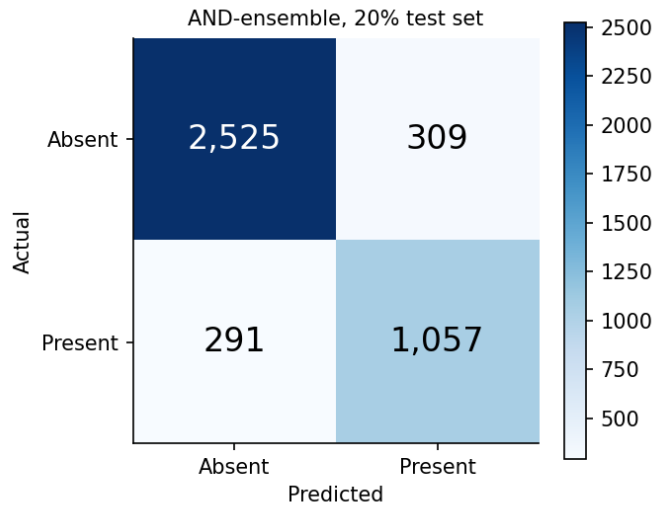


Figure 5.2: Confusion matrix heatmap

Precision and recall are close (77.4% and 78.4%). Taking the highest window score on its own costs precision, and asking both models to agree wins most of it back without losing recall. Specificity is 89.1%, so the system seldom claims a clause that is not there. In a contract-review tool, though, the more harmful mistake is the opposite one. A missed clause gives no warning (Section 3.4), and at a recall of 78.4% about one real clause in five is missed at this step.

Per-category behaviour

Figure 5.3 shows the per-category F1 for the strongest categories. The common, standard categories are close to solved: *Document Name* 1.000, *Parties* 0.995, *Agreement Date* 0.963 and *Governing Law* 0.931. At the other end, five categories with at most seven test examples get an F1 of 0.000 from the AND-ensemble. Table 5.4 shows what that means. In each case the ensemble marked the category as present in no test contract at all (no true and no false positives), so $F1 = 0.000$ is correct, since none of the 2–7 real clauses was found. Precision, however, is undefined (0/0). The transformer alone does find some of these clauses, and the AND rule removes them because the TF-IDF model never agrees (Section 5.2.4). With two to seven examples per category these scores are unreliable anyway, so our main results use micro-averaged scores. Macro-F1 over the 34 categories with at least ten test positives avoids the problem and ranks the models slightly differently: 0.666 for the TF-IDF baseline, 0.658 for the AND-ensemble and 0.611 for the transformer alone.

Table 5.4: Categories with zero F1

Category	Test examples	AND TP / FN	AND precision	Transformer alone F1 (TP/FP/FN)	TF-IDF F1
Most Favored Nation	7	0 / 7	n/a	0.000 (0/0/7)	0.000
No-Solicit of Customers	6	0 / 6	n/a	0.323 (5/20/1)	0.000
Price Restrictions	4	0 / 4	n/a	0.087 (1/18/3)	0.000
Source Code Escrow	2	0 / 2	n/a	0.000 (0/0/2)	0.000
Unlimited/All-You-Can-Eat License	2	0 / 2	n/a	0.500 (1/1/1)	0.000

Note. n/a: the model never said “present”, so precision is 0/0 and has no value. Threshold 0.5, October 2026 models.

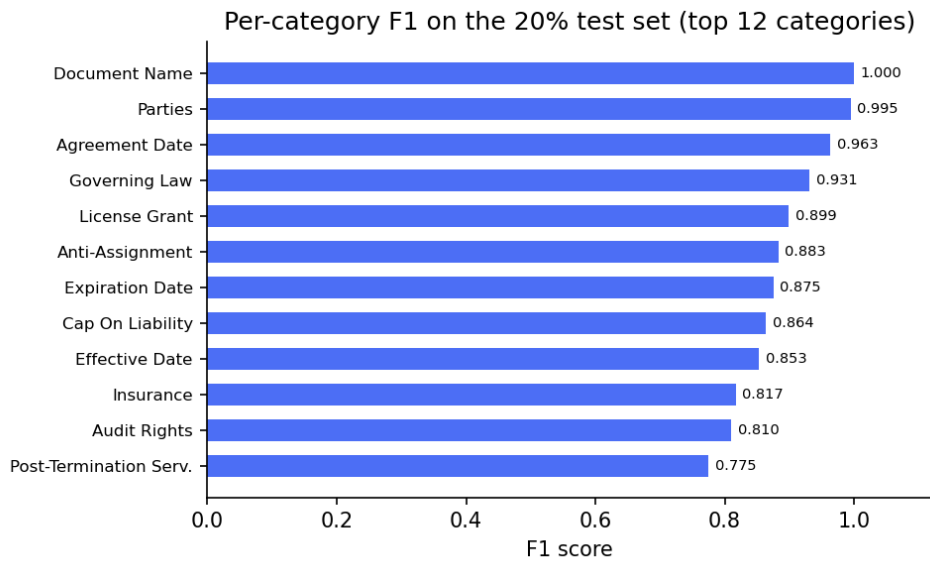


Figure 5.3: Top-12 per-category F1

5.1.2 Span Extraction

We tested the question-answering model on all 2,667 test windows that contain an answer (Table 5.5). It reaches a token-F1 of 0.764, and 82.8% of its answers overlap the real clause at $F1 \geq 0.5$. So when the clause is in front of the model, it captures most of the right text about eight times in ten. Exact match is lower, which we expected: marked legal clauses are long, and their exact start and end are often debatable (should the section number at the start be included?), so overlap is the more useful score here. For comparison, the best published CUAD result uses DeBERTa-xlarge, which has about ten times as many parameters as our 67M-parameter model.

A score on 102 contracts needs a confidence interval next to it (Section 5.4.1), so we ran the same bootstrap over contracts here. The clauses are even more linked to each other than in the presence task. The 2,667 span decisions come from only 102 contracts, and a contract with unusual writing (numbered sub-clauses, many defined terms) pulls all of its clauses in the same direction. Table 5.5 therefore gives two intervals for each score. One resamples whole *contracts*, which is the correct method. The other resamples single *clauses*, which is the naive method, and we show it because the difference between the two is informative.

Table 5.5: Span extraction results

Metric	Score	95% CI (by contract)	95% CI (by clause)
Token-level F1	0.764	[0.743, 0.782]	[0.752, 0.775]
Overlap (token-F1 ≥ 0.5)	82.8%	[80.0%, 85.2%]	[81.4%, 84.2%]
Exact Match	35.5%	[32.6%, 38.6%]	[33.7%, 37.3%]

Note. 95% bootstrap intervals, 10,000 resamples. Quote the interval by contract. The interval by clause is shown only to make clear how much it understates the uncertainty.

Resampling contracts gives a token-F1 interval of [0.743, 0.782], roughly twice as wide as the [0.752, 0.775] we would get by treating clauses as independent. The extra width is what it costs to ignore that clauses from one contract are linked, the same effect that made the contract-level bootstrap necessary in Section 5.4.1. It also tells us how precise the score is. On a test set this size, differences smaller than about two points of token-F1 cannot be measured, so 0.764 should be compared with published figures with that margin in mind.

This test has a limit. It measures extraction when the window contains the answer. More exactly, each test window is the 1,200-character focus window placed around the real answer (Section 4.2.5), so the model is indirectly told where to look. The result is a best case, and the real system cannot reach it: on plain windows that contain the clause the same model reaches a mean token-F1 of only 0.366 (Section 5.2.1). The CUAD paper’s full-document test (precision at 80% recall over all windows, including those with no answer) needs a heavier ranking setup that we did not build, so we make no claim about that score. Section 5.2.1 looks at the other side: how extraction does when the presence step chooses the window.

5.1.3 Summarization

What the reference set actually is

One fact about this test decides what its score can mean, so we state it first. CUAD has no plain-English versions of its clauses, so there was no ready-made set of target sentences to train or test with. We wrote the targets ourselves: 41 one-sentence summaries, one per category, written from the weaker party’s side (Section 4.2.3). Every training pair and every test pair uses one of these 41 sentences, and the target never changes from clause to clause. Two *Cap on Liability* clauses with very different limits have exactly the same target sentence.

This choice has a side effect. With 41 different targets for 2,667 test clauses, a model can score well by recognising the category and repeating its sentence, and the score cannot tell this apart from real summarizing. The test below separates the two.

Two reference sets, the same 150 clauses

We test the fine-tuned FLAN-T5-small on 150 test clauses (chosen with seed 42) against two sets of reference sentences:

1. Template references: the category template, exactly as in training. The main score was produced this way.
2. Clause-specific references: 150 one-sentence summaries, one per clause, each describing that particular clause with its actual limit, notice period and parties. They were drafted from the clause text alone, without seeing any model output, and they are fully separate from the 41 templates. They are a trial reference set that no lawyer wrote, and we claim nothing more for them.

The clauses are the same in both tests, so the gap between the two scores measures one thing: how much of the ROUGE-L score comes from the reference sentences all being the same. Table 5.6 gives the result. The summarizer re-trained in October 2026 gives exactly the same scores as the original.

Table 5.6: Summarizer ROUGE-L

System	Reference set	ROUGE-L	95% CI
Fine-tuned FLAN-T5-small	41 category templates	0.775	[0.718, 0.833]
Fine-tuned FLAN-T5-small	clause-specific (pilot)	0.116	[0.105, 0.128]
Zero-shot FLAN-T5-small	clause-specific (pilot)	0.299	[0.275, 0.322]
<i>Category template itself</i>	clause-specific (pilot)	0.115	[0.104, 0.127]

Note. 95% bootstrap intervals over the 150 clauses (10,000 resamples).

Reading the table

We draw three conclusions from the table.

First, the 0.775 comes from copying templates, and the numbers show it. For all 150 test clauses the fine-tuned model outputs one of the 41 template sentences word for word: 100% of outputs are an exact copy of a template, the right one 70% of the time and another category’s template the other 30%. The model never writes a sentence of its own. When it is wrong, it is wrong the way a classifier is wrong; a *Price Restrictions* clause about a 10% rate cap gets the *ROFR* template. Fine-tuning taught the model to pick one of 41 sentences.

Second, the last row of Table 5.6 shows that a lookup table does as well. The category template alone, with no model at all, scores 0.115 against the clause-specific references, which matches the fine-tuned model’s 0.116 within the margin of error. Whatever the fine-tuned summarizer learned adds nothing we can measure beyond printing the category’s sentence.

Third, fine-tuning made the model worse at the real task. Against the clause-specific references, the original model (before fine-tuning) scores 0.299, more than twice the fine-tuned model’s 0.116. The original FLAN-T5 is still a poor legal summarizer. As Table 5.7 shows, it mostly repeats the source text, and repeating the source raises a word-overlap score when the reference uses the same words as the clause. What we can conclude is narrower and still unwelcome: fine-tuning removed the clause-specific content the original model had and replaced it with smooth, general sentences, which score well only against references made of those same sentences.

Table 5.7: Fine-tuned vs. zero-shot output

Input clause	“Any attempted assignment or delegation in violation of this section by either party without the prior written consent of the other will be void.”
Gold template	You cannot transfer this contract to someone else without permission.
Fine-tuned	You cannot transfer this contract to someone else without permission. (<i>exact template</i>)
Zero-shot	“a) Any attempted assignment or delegation in violation of this section ...will be void.” (<i>repeats the legal text</i>)
Clause-specific reference	Neither side may hand its rights or duties to a third party without the other’s written consent, which cannot be unreasonably refused.

What we therefore claim

What we can claim is a working “sort into a category, then show a template” system. It finds a clause’s category and shows a plain-English sentence for that category, written in advance by people. In an app that still helps: a signer who reads “You cannot transfer this contract to someone else without permission” next to a block of legal text is better off than before. (Clausify shows this line on every card, labelled as the summarizer’s output and placed next to the fixed reason from the risk table; when the sentence belongs to another category, the card says so; Section 4.4.9.) We make no claim of free-text legal summarizing, and the 0.775 should not be used as

evidence for one. The number that describes how well the model summarizes single clauses is 0.116, so this part of the project is a proof of concept.

Would a larger model write clause-specific sentences?

The fine-tuned model copies templates, and the original FLAN-T5-small mostly repeats the clause (Table 5.7). We tested whether a larger model, used without fine-tuning, does better. FLAN-T5-small, FLAN-T5-base (about 250 million parameters) and FLAN-T5-large (about 780 million) each wrote one sentence for the same 150 clauses with four different instructions, twelve combinations in all. We split the 150 clauses at random into 50 for choosing the model and instruction and 100 for testing. The rule, fixed in advance, took the highest ROUGE-L against the clause-specific references among combinations whose outputs were easier to read than the clauses (Flesch reading ease) and contained a number that is not in the clause in at most 5% of cases. It chose FLAN-T5-large asked to “explain in one plain-English sentence what this contract clause means for the people who sign it”. On the 100 test clauses it scores 0.331, against 0.115 for the category template and 0.114 for our fine-tuned model.

Reading the outputs shows that the score comes mostly from copying. On average 80.4% of an output’s three-word sequences appear word for word in the clause, and 68% of the outputs are near-verbatim excerpts of it (at least 90% of their three-word sequences copied). The clause-specific references reuse many of the clause’s words, so copying the clause scores well. Our rule did not catch this, because every combination’s outputs scored as easier to read than the clauses, partly by being shorter. Some outputs also change the meaning. A clause requiring IntriCon to make its records available to an auditor appointed by Dynamic Hearing became “IntriCon has the right to inspect, copy and audit such records”, which reverses who audits whom, and a clause that only names “the Transporter” became “Transporter is a company that transports goods from one place to another”. Showing the model three worked examples from the 50 development clauses first changed little: ROUGE-L 0.324, with 71% of outputs near-verbatim.

We therefore did not put any of these models in the app. A near-copy of the clause labelled “in plain English” would mislead a user, and a fluent sentence that reverses an obligation is worse. The answer to RQ3 stays no: neither our fine-tuned model nor FLAN-T5 up to about 780 million parameters, used directly or with examples, writes a plain-English sentence for the specific clause.

Going further needs target sentences that differ from clause to clause. The trial set above is a small sample of exactly that, and its 150 references took a few hours to draft. Section 6.5 describes a larger version.

5.2 Analysis of Design Solutions

This section looks at the design as a whole: how many clauses get through the full system, whether its mistakes land where they would hurt a signer, what the mistakes look like, and how the ensemble handles rare categories.

5.2.1 End-to-End: What Actually Reaches the User

So far every number measured one step with the other steps out of the way. Presence was scored on its own, extraction was scored with a window already known to contain the answer, and summaries were scored on real clauses. That is the right way to test each part, but it does not tell us what the system does to a contract. The steps run in a chain and errors add up: a category the presence step misses is a clause the span step never sees.

We therefore ran the whole system on all 102 test contracts and asked, for each of the 1,348 real clauses, whether it gets through all three steps. A clause counts only if (i) the presence step finds its category, (ii) the span step, reading the window the presence step chose (never a window chosen with the real answer), returns text that overlaps the real clause with token-F1 ≥ 0.5 , and (iii) the risk level given is the correct one. Table 5.8 shows how many clauses are lost at each step.

Table 5.8: End-to-end clause survival

Step	Condition	First setup (AND)		App (recall-first)	
		Left	Share	Left	Share
–	real clauses in the test set	1,348	100.0%	1,348	100.0%
2A Presence	category found	1,057	78.4%	1,253	93.0%
2B Span	extracted text overlaps the real clause at F1 ≥ 0.5	296	22.0%	298	22.1%
Risk	correct risk level given	296	22.0%	298	22.1%
Clauses that get through every step		296	22.0%	298	22.1%
<i>95% CI, bootstrap over contracts</i>			[19.8%, 24.2%]		[19.7%, 24.6%]

Note. First setup: AND-ensemble at 0.5. App: the re-run transformer with its recall-first thresholds (Section 5.3.3). Each share is of all real clauses still left after that step.

Only 22.0% of real clauses get through with the first setup, and 22.1% with the app’s setting. This is the number that described what a user got, until we retrained the span model (Section 5.2.1), which raises the app’s figure to 59.6%. It is far below what the separate scores suggest: 78.4% presence recall times 82.8% span overlap predicts about 65% for the first setup, and 93.0% times 82.8% predicts about 77% for the app. Both deliver less than a third of that. The rest of this section explains the gap.

The risk step loses nothing. The level is a fixed lookup for each category, so it is right exactly when the category is right and adds no errors of its own. “Correct” here means “matches our risk table”.

Where the clauses are lost

The big loss happens at the span step, and it has two separate causes. We told them apart by checking, for every clause whose category was found, whether the window chosen by the presence model actually contained the clause.

The first cause is that the window with the highest score is often the wrong window. Reading the whole window helps here: the re-run model’s best window misses the clause 17.4% of the time, against 30.9% for the first setup. We chose the maximum rule (Section 4.2.1) because it removes the search limit: if any window fires, the

Table 5.9: Span-stage loss breakdown

	First setup		App	
	Count	Share	Count	Share
Chosen window <i>contains</i> the real clause	730	69.1%	1,035	82.6%
and the span model found it ($F1 \geq 0.5$)	289	39.6%	290	28.0%
but the span model still missed it	441	60.4%	745	72.0%
Chosen window does <i>not</i> contain the real clause	327	30.9%	218	17.4%
but the span still scored ≥ 0.5 (same wording elsewhere)	7		8	

category is reported. Section 4.2.1 already discussed what that costs in precision. We had not expected the second cost. The highest-scoring window is also used to locate the clause, and it is poor at that job, missing the clause entirely 30.9% of the time. The model was never trained to locate clauses. It was trained to answer “does this window contain a clause of this category?”, and a window that talks about liability in general can score higher than the one that sets the limit. For those 327 clauses (218 with the app’s setting) the span step gets the wrong 2,000 characters, and no extraction model could recover them.

The second cause is that the span model is much weaker on windows unlike the ones it was trained on. Even when the window does contain the clause, extraction succeeds only 39.6% of the time with the first setup and 28.0% with the app’s setting, against 82.8% in the separate test of Section 5.1.2. With the window choice improved, this became the larger loss. Where the clause sits in the window explains the drop. The first model read only the first half of each window, so when its chosen window was right, the clause was in the second half only 8.9% of the time. With the re-run model it is 35.8%. In the first half the span model succeeds about equally often for both (41.7% and 40.7%); in the second half it succeeds only 18.5% and 5.4% of the time. All of the drop from 39.6% to 28.0% comes from clauses the first model could not even see. The focus windows explain this (Section 4.2.2). In training, every example was centred so that the answer started about 150 characters into a 1,200-character window. This is allowed, because the answer is used to build a training example and never as an input when the system runs (Section 4.2.5). It still had a cost. The model has only seen windows with the answer near the start, while the real system gives it plain 1,200-character chunks where the answer can be anywhere, and the average token-F1 falls from 0.764 to 0.383 (0.350 with the app’s setting).

What this changes

This changes how the earlier results should be read in three ways.

The span scores in Section 5.1.2 are an upper limit on real extraction quality. They answer “how good is the extractor when given the right window?”, which is a fair question about the model and a misleading one about the system.

A bigger model is also not the most useful next step. The second cause is a data problem with a cheap fix: train the span model on the same plain chunks it sees in real use, with the answer wherever it falls. Section 5.2.1 applies that fix. The first

cause needs a model trained to choose the extraction window, instead of reusing the presence score. Both fit within this project’s computing budget, and neither needs more parameters.

Fixing the second cause: a span model trained on the chunks it reads

We trained a second span model in the same way as the first (DistilBERT, learning rate 3×10^{-5} , two epochs, seed 42) and changed only the training examples. Each example is now one of the 1,200-character chunks the system really reads. For every clause we took one of the 2,000-character windows that contain its start, chosen at random, and cut it into chunks at the same offsets as when the system runs (0 and 800 characters). A chunk that contains the start of the clause is labelled with its real position, wherever that is, and a clause that runs past the end of the chunk is cut there. A chunk that does not contain it is labelled “no answer”, as in SQuAD 2.0 [31], so the model learns that a chunk can hold nothing. The input limit is 384 tokens, which covers the whole chunk. We used 6,000 clauses from the 327 contracts the re-run presence model was trained on, which gave 11,967 chunks (60.6% with an answer). Training took 46.2 minutes.

Two choices were made on the 81 validation contracts, before the test contracts were used: the epoch (first or second), and how answers are compared across the chunks of a window. The second epoch was best, comparing each chunk’s best answer with that chunk’s “no answer” score and allowing answers of up to 120 tokens. On validation it extracted 63.4% of the detected clauses at token-F1 ≥ 0.5 , against 20.9% for the first span model. We then scored the test contracts once (Table 5.10).

Table 5.10: First and retrained span model

	First span model	Retrained span model
<i>App setting (recall-first transformer), 1,348 real clauses</i>		
Clauses that get through every step	298 (22.1%)	803 (59.6%)
95% CI, bootstrap over contracts	[19.7%, 24.6%]	[57.0%, 62.2%]
Extraction succeeds, given the right window	28.0%	76.4%
clause in the first half of the window	40.7%	76.2%
clause in the second half of the window	5.4%	76.8%
Mean token-F1 of the extracted text	0.350	0.673
<i>Balanced setting (AND, tuned for micro-F1)</i>		
Clauses that get through every step	266 (19.7%)	683 (50.7%)
<i>A window that contains the clause, wherever it sits (1,348 clauses)</i>		
Mean token-F1	0.366	0.779
Token-F1 ≥ 0.5	24.4%	77.7%

Note. Same 102 test contracts, same detections and same chosen windows as Table 5.8; only the span model changes. “Extraction succeeds” means token-F1 ≥ 0.5 with the real clause.

With the app’s setting, 59.6% of the real clauses now get through all three steps, against 22.1% with the first span model. When the chosen window contains the clause, the new model extracts it 76.4% of the time, against 28.0%, and it does about as well when the clause sits in the second half of the window (76.8%) as in the first (76.2%); the first model fell from 40.7% to 5.4%. On a plain window that

contains the clause, the first model reaches a mean token-F1 of only 0.366, far below the 0.764 it reaches on centred focus windows (Section 5.1.2), so that 0.764 was a best case it never met in use. The new model reaches 0.779 on the same plain windows.

The category is found for 93.0% of the clauses, the chosen window contains 82.6% of those, and the new model extracts 76.4% of those, which multiplies to 58.7%; another 12 clauses come through from a wrong window because the same wording appears elsewhere, giving the 59.6%. The 545 clauses still lost split three ways: 95 are never detected, 206 are in a window the presence model did not choose, and 244 are in the right window but the span model misses them. The two causes of this section now cost about the same, so a model trained to choose the window and a better span model would each help. The new span model is the one the app uses.

The quote the app shows

Clausify does not show the extracted text alone. It quotes a whole paragraph and highlights the extracted text inside it, because a paragraph is easier to recognise in the contract. Until now the paragraph was the one the presence model scored highest, and this had not been measured on the test set. We measured it in the same test, counting a quote as showing the clause when it contains at least half of the clause’s words. With the paragraph the presence model scores highest, 75.3% of the detected clauses are shown this way (median quote 411 characters). With the paragraph that contains the new span model’s answer, 77.2% are (357 characters). We chose between the two on the validation contracts, where the second was slightly ahead (73.5% against 72.7%), and the app now uses it. Over all real clauses, 71.7% get a card whose quote contains at least half of the clause (95% CI [69.0%, 74.5%]). Under the stricter token-F1 ≥ 0.5 rule the paragraphs score lower (46.9% and 44.9% of detected clauses), because a paragraph is longer than the clause in it, which is why the app highlights the extracted text.

The two ways of choosing the paragraph are close, and on the demonstration contract of Section 4.4.9 they differ on 4 of the 27 cards. The new one quotes the agreement’s title for *Document Name*, where the old one quoted the license grant. On the *Liquidated Damages* card, however, it quotes the limitation-of-liability paragraph, where the old one found Section 13, the liquidated-damages clause. Clausify’s output should therefore still be read as a shortlist: the category is found for 93.0% of the clauses that are really there, and the quote under it contains the clause most of the time, but not always. The app tells users that it is not legal advice and that the quoted text may not be the exact clause the badge refers to (Section 5.6.2).

5.2.2 Risk-Weighted Evaluation: Does the Metric Match the Purpose?

Micro-F1 over 4,182 decisions treats every decision as equally important, and a person signing a contract does not see them that way. The risk layer (Section 4.2.4) exists because some categories matter more than others, so the evaluation should ask whether the system’s mistakes fall where they do the least harm. We had not

asked this before, and the answer reverses the recommendation that the overall score gives.

Table 5.11 splits the 41 categories into the three levels of our risk table and scores each model within each level.

Table 5.11: Results by risk band

Band	Model	Precision	Recall	F1	Found	Missed
High 8 cats 176 positives	Transformer	0.392	0.841	0.534	148	28
	TF-IDF	0.540	0.648	0.589	114	62
	AND ensemble	0.581	0.631	0.605	111	65
	OR ensemble	0.379	0.858	0.526	151	25
Medium 22 cats 498 positives	Transformer	0.429	0.886	0.578	441	57
	TF-IDF	0.621	0.723	0.668	360	138
	AND ensemble	0.655	0.679	0.667	338	160
	OR ensemble	0.424	0.930	0.582	463	35
Low 11 cats 674 positives	Transformer	0.800	0.953	0.870	642	32
	TF-IDF	0.909	0.917	0.913	618	56
	AND ensemble	0.923	0.902	0.912	608	66
	OR ensemble	0.792	0.967	0.871	652	22

Note. Threshold 0.5. “Missed” counts real clauses of that level that the model did not report.

The headline configuration is the worst one for High-risk clauses

Start with the High level. The AND-ensemble, the first setup’s main configuration because of its overall micro-F1, misses 36.9% of High-risk clauses. The transformer alone, the weaker model in the previous sections, misses 15.9%. The more permissive OR rule comes last on overall micro-F1 (0.692, tied with the transformer alone) and misses only 14.2%.

Out of 176 High-risk clauses in 102 contracts, the ensemble finds 111 and misses 65, while OR finds 151 and misses 25. Choosing the ensemble over OR for its micro-F1 costs a signer 40 High-risk clauses (unlimited liabilities, IP assignments, non-competes). In return the ensemble gets a ranking that Section 5.4.1 already showed is a statistical tie.

The cause is the one found in Section 5.2.4, and here it adds up. Several High-risk categories are uncommon. *Most Favored Nation* is among the ten rarest categories, and *Liquidated Damages* (11 test examples) and *Irrevocable or Perpetual License* (17) are also below average. The AND rule is hardest to satisfy where training data is thinnest, so it hurts recall most at the risk level that needs recall most. The Low level shows the opposite picture. Of the 1,348 real clauses, 674 are Low-risk (dates, party names, governing law), and every model scores about 0.9 on them, so the clauses that matter least dominate the overall micro-F1.

What we conclude, and what we do not

We do not change the reported setup because of this table. Picking OR after seeing which rule does best on the High level of the test set would mean choosing a setup with the test set, which Section 4.4.1 says this project avoided. The table is a diagnosis, and the tuning belongs on a validation split.

It does show that our evaluation score never matched our stated goal, and that the mismatch is large enough to reverse the ranking of the models. A system meant to warn a signer before signing should be chosen by how many High-risk clauses it finds, or by a score that weights recall more. An average over 41 categories, where half of the real clauses are Low-risk ones such as dates and party names, is the wrong yardstick. Section 5.3.3 runs the proper version of this test: it fixes a recall-first goal (micro-F2) in advance, chooses the rule and thresholds on a validation split, and reports on the test set once. The recall-first setups it chooses find 86.9% and 90.3% of High-risk clauses.

Every number in Table 5.11 uses our own choice of which categories are High, Medium and Low, so a different risk table would change it.

The same breakdown for the re-run setups

Table 5.12 repeats the breakdown for the setups of Section 5.3.3, whose rules and thresholds were chosen on the validation contracts. The recall-first settings fix the problem this section found. The app’s default misses 17 of the 176 High-risk clauses and 55 of the 498 Medium-risk ones, against 65 and 160 for the first setup’s AND-ensemble. The price is precision: 0.439 on the High level, against 0.581. The balanced setup, chosen for micro-F1, misses more High-risk clauses than any first-setup rule (81), which is the same conflict between the overall score and the purpose of the tool.

Table 5.12: Results by risk band, re-run setups

Band	Setup	Precision	Recall	F1	Found	Missed
High 176 positives	Recall-first transformer (app)	0.439	0.903	0.591	159	17
	Recall-first ensemble	0.515	0.869	0.647	153	23
	Balanced ensemble (AND)	0.736	0.540	0.623	95	81
	TF-IDF, tuned	0.572	0.676	0.620	119	57
Medium 498 positives	Recall-first transformer (app)	0.534	0.890	0.668	443	55
	Recall-first ensemble	0.565	0.890	0.691	443	55
	Balanced ensemble (AND)	0.787	0.659	0.717	328	170
	TF-IDF, tuned	0.620	0.747	0.678	372	126
Low 674 positives	Recall-first transformer (app)	0.843	0.966	0.900	651	23
	Recall-first ensemble	0.868	0.960	0.912	647	27
	Balanced ensemble (AND)	0.939	0.895	0.916	603	71
	TF-IDF, tuned	0.920	0.920	0.920	620	54

Note. Same 102 test contracts as Table 5.11. Every setup uses the per-category thresholds chosen on the 81 validation contracts.

5.2.3 What the Errors Actually Look Like

The confusion matrix counts 309 false positives and 291 false negatives. Counts alone do not show whether the model fails for silly reasons or understandable ones, so Table 5.13 shows real cases from the test contracts. The false positives are sorted by the model’s own confidence, since a system’s most confident mistakes tell us the most about it.

Table 5.13: Representative errors

Category	Scores	Evidence
False positives: reported, but the category is not labelled in this contract		
Insurance	0.82 / 0.96	“...a fixed amount of [*] of gross sales to cover freight, postage, insurance costs on shipments to such Third Party, packing costs...” <i>A royalty clause that lists insurance as a cost. The word is there; the duty to insure is not.</i>
Governing Law	0.81 / 0.95	“...there shall arise the need to decide the question by what municipal or national law this Agreement ...is governed , the following facts shall be excluded...” <i>A clause about how to work out the governing law, not one that states it. A borderline case for the labels.</i>
Parties	1.00 / 0.98	An addendum whose first window looks like the top of a contract. <i>Both models react to the layout; the labels put Parties somewhere else.</i>
False negatives: labelled as present, but both models scored it low		
Liquidated Damages	0.21 / 0.02	“...agrees to pay a cancellation fee of Five Thousand Dollars (\$5,000) per month multiplied by the number of months remaining...” <i>A clear liquidated-damages term that never uses the phrase. The word model has nothing to match; the transformer, with 50 training contracts for this category, has not learned the idea.</i>
Most Favored Nation	0.22 / 0.02	“...Sponsor shall have the right to approve or disapprove any additional sponsor...unless another...” <i>In meaning an MFN-style protection, but written without any of the category’s usual words. This category has 21 training contracts and scores 0.000 overall.</i>
Third Party Beneficiary	0.26 / 0.02	“Notwithstanding the foregoing, UTK may assign this Agreement or any portion of its Compensation ...to its subsidiaries.” <i>This reads like an assignment clause, not a third-party-beneficiary clause. Some rare-category “failures” are disagreements with the labels, not model errors.</i>
Span errors: category found correctly, but the extracted text is wrong (Section 5.2.1)		
Minimum Commitment	F1 0.00	The window <i>does</i> contain the clause, but the model returns only “ the ”. <i>The start and end of the answer fall on the same word, and nothing in the decoding stops this.</i>
Effective Date	F1 0.06	The window does <i>not</i> contain the real clause, yet the model confidently returns “7th day of september, 1999”. <i>When Stage 2A gives the wrong window, Stage 2B still answers: it has no way to say “not here”.</i>
Insurance	F1 0.20	The real clause is the insurance duty; the model returns the indemnity clause next to it. <i>Close in topic, different in law.</i>

Note. Text shortened. Scores are the TF-IDF and transformer probabilities.

Three patterns stand out. Many false positives match the words without the duty: a contract that mentions insurance costs does not require insurance, and a bag-of-words model reading the whole document cannot tell the two apart. Many misses are the reverse, the idea without the usual words. A \$5,000-per-month cancellation fee is liquidated damages, but a model that has seen only about fifty contracts with

this category, most of them using the usual words, will not learn that. Disagreement with the labels is the third pattern and part of the rare-category problem. The *Third Party Beneficiary* case above looks like a labelling mistake, so the 0.000 F1 scores in Section 5.2.4 come partly from the model and partly from the labels. We did not try to measure the split between the two, because that would mean labelling the contracts again, and we are not qualified to do it.

The span errors point to something easier to fix. The one-word answer “**the**” and the confident answer from a window that cannot contain the clause have the same cause: the extraction model has no way to say that nothing is there. Training with unanswerable examples, as in SQuAD 2.0, gives it that option, and the retrained span model of Section 5.2.1 is trained that way.

The app’s recall-first setting makes a different mix of mistakes, because it was chosen to miss as little as possible. On the test set it reports 710 false positives and only 95 misses, against 309 and 291 for the first setup’s ensemble. Its most confident false alarms are informational categories, *Parties*, *Agreement Date* and *Effective Date*, reported with scores above 0.98 in exhibits and addenda where CUAD’s annotators marked no such clause. These are cheap mistakes for a user, since the Low-risk cards sit at the bottom of the report. Its misses are mostly clauses written without the usual words: a renewal “by amendment to this agreement” (*Renewal Term*), and a sponsor’s promise about other sponsors read as *Most Favored Nation*.

5.2.4 Does the AND Rule Hurt Rare Categories More Than Common Ones?

Section 5.4.1 supports the AND-ensemble for the kind of mistakes it makes, since it turns the transformer’s many false alarms into balanced precision and recall. That argument rests on the overall numbers, and an overall number can hide a trade that falls unevenly across categories. The rare categories are the obvious place to look. Agreement between two models is hardest to get where each model has the least training data, and agreement is exactly what the rule requires.

We sorted the 41 categories by how often they appear in the training data and took the ten rarest (11 to 42 positive training contracts, 70 positive test decisions in total). We then compared the AND rule with the transformer alone on these ten and, separately, on the other 31 categories. Table 5.14 gives the result, and the two groups behave differently.

On the common categories the ensemble works as intended. Compared with the transformer alone it gains 20.0 points of precision, loses 12.0 points of recall, and F1 rises from 0.713 to 0.795. That is a good trade, and all of the ensemble’s overall advantage comes from it.

On the rare categories the ensemble gains nothing. The AND rule gains 34.6 points of precision by giving up 30.0 points of recall, and its F1 ends at 0.364, slightly below the transformer alone (0.368). The transformer alone finds 39 of the 70 rare clauses, and requiring agreement cuts this to 18. On these categories the TF-IDF model alone is now the weakest of the three (F1 0.353), yet its vetoes decide what the ensemble reports.

Table 5.14: Rare vs. common categories

Group	Model	Precision	Recall	F1	TP	FN
Rarest 10 (70 test positives)	Transformer	0.275	0.557	0.368	39	31
	TF-IDF	0.429	0.300	0.353	21	49
	AND ensemble	0.621	0.257	0.364	18	52
Other 31 (1,278 test positives)	Transformer	0.577	0.933	0.713	1,192	86
	TF-IDF	0.753	0.838	0.793	1,071	207
	AND ensemble	0.777	0.813	0.795	1,039	239

Note. Micro-averaged within each group. Rarest 10 = fewest positive training contracts (11 to 42).

The per-category details are clearer than the average. On five of the ten rarest categories the AND rule scores exactly 0.000. On three of those (*Price Restrictions*, *Unlimited/All-You-Can-Eat License*, *No-Solicit of Customers*) the transformer alone was finding something (F1 0.087, 0.500, 0.323) before the AND rule removed it.

The overall micro-F1 counts all 4,182 decisions equally, and a signer does not. Missing a rare clause is not clearly cheaper than missing a common one, and it may be much worse. *Most Favored Nation* is among the ten rarest categories, it is one of our eight High-risk categories, and both models score 0.000 on it. A rule that favours precision on common categories, even one with no settings, improves the score without reducing harm. The first setup reported the ensemble as its main configuration because the overall test chose it. On this evidence, though, a system meant to protect signers would do better with the more permissive rule for High-risk categories and the AND rule for the rest, accepting more false alarms where a missed clause costs the most. Choosing that mixed rule here, after looking at the test set, would not be a fair test. Section 5.3.3 includes it among the rules chosen on a validation split. The validation split preferred the AND rule for micro-F1 and the average of the two scores for micro-F2, so the mixed rule was not used.

Table 5.15 repeats the comparison for the re-run setups. The app’s recall-first transformer finds 49 of the 70 rare clauses (F1 0.460, against 0.368 for the first setup’s transformer), and it scores zero on only one of the ten rarest categories (*Price Restrictions*), where the first setup’s ensemble scored zero on five. It also finds one of the seven *Most Favored Nation* clauses. The balanced AND setup repeats the first setup’s pattern: on the rare categories it finds 18 of 70 and scores zero on five, so requiring agreement still costs the most where training data is thinnest.

Table 5.15: Rare vs. common categories, re-run setups

Group	Setup	Precision	Recall	F1	TP	FN
Rarest 10 (70 test positives)	Recall-first transformer (app)	0.343	0.700	0.460	49	21
	Recall-first ensemble	0.449	0.629	0.524	44	26
	Balanced ensemble (AND)	0.692	0.257	0.375	18	52
Other 31 (1,278 test positives)	Recall-first transformer (app)	0.662	0.942	0.777	1,204	74
	Recall-first ensemble	0.694	0.938	0.798	1,199	79
	Balanced ensemble (AND)	0.867	0.789	0.826	1,008	270

Note. Same ten rarest categories as Table 5.14; thresholds chosen on the validation contracts.

5.3 Final Design Adjustments

Three choices made before training were never checked: max-pooling as the way to combine window scores, 0.5 as the decision threshold, and the 256-token input length. After the main tests we went back to all three. The first two are tested in Sections 5.3.1 and 5.3.2 on the original models. The third turned out to be a flaw, and Section 5.3.3 fixes it and re-runs the presence comparison with both models tuned on a validation split. Those re-run results replace the original ones as the project’s answer to RQ1.

5.3.1 Was Max-Pooling the Right Way to Combine Window Scores?

Section 4.2.1 chose max-pooling because it is the standard multiple-instance rule [32], under which a bag is positive if any one of its items is. Sections 4.2.1 and 5.4.3 then blamed max-pooling for the transformer’s many false alarms. Neither statement had been tested against another rule, which is a weak basis for a claim the project depends on. We therefore scored every window once with the October 2026 model and compared six ways of combining the same window scores, so that the rule is the only thing that changes. As a check, max-pooling at 0.5 gives exactly the 0.6924 of the main test.

Table 5.16: Ways to combine window scores

Rule	F1@0.5	P@0.5	R@0.5	Best F1	at threshold	AND F1@0.5
Max (used in this project)	0.6924	0.558	0.913	0.7529	0.90	0.7789
Top-3 mean	0.7115	0.662	0.769	0.7253	0.40	0.7428
Top-5 mean	0.6603	0.714	0.614	0.7218	0.30	0.6722
Mean over all windows	0.1636	0.924	0.090	0.7110	0.10	0.1607
Noisy-OR	0.5908	0.424	0.975	0.7038	0.95	0.7735
Second-highest window	0.6774	0.657	0.700	0.6951	0.40	0.7074
<i>TF-IDF baseline (no windows)</i>	0.7747	0.742	0.810	–	–	–

Note. October 2026 model. “Best F1” is each rule’s micro-F1 at its own best threshold, found by trying thresholds on the test set: an upper limit for comparison, not a setting we use. “AND F1” combines each rule with the TF-IDF baseline.

Three results come out of this, and together they correct an earlier claim of ours.

At the threshold we actually used, max-pooling was not the best rule. At 0.5, the mean of the three highest window scores reaches 0.7115, against 0.6924 for the maximum. That recovers almost a quarter of the gap to the baseline from a one-line change. Averaging the top three windows is a softer version of the multiple-instance rule: it still looks for evidence concentrated somewhere in the document, but it wants that evidence in more than one place, which guards against the single false window that Section 4.2.1 named as max-pooling’s weakness. The mean over all windows fails at 0.5 (recall 0.090) for an obvious reason. An average over about 35 windows, one of them positive, cannot reach 0.5, so its best threshold is just 0.10.

Once each rule gets its own threshold, max-pooling is the best of the six. At 0.90 it reaches 0.7529, ahead of 0.7253 for the top-3 mean. Its scores just do not suit a line

drawn at 0.5, which is what Section 5.3.2 finds from the other direction. Much of the transformer’s gap is about where the line is drawn, and less about how window scores are combined. The last column of Table 5.16 shows the same inside the AND-ensemble: with max-pooling the ensemble scores 0.7789, and every other rule does worse.

No rule reaches the baseline. The best value in the whole table (max-pooling at a threshold chosen on the test set, already a generous upper limit) is 0.7529, against 0.7747 for the baseline, and every other rule stays behind at every threshold we tried. With its 256-token input, the window-based transformer loses whatever we change: its training data (Section 5.4.3), its seed (Section 5.4.2), its way of combining windows or its threshold. The list missed the explanation that mattered. Every one of these runs read only about half of each window, and Section 5.3.3 shows that fixing this, with thresholds tuned on validation, closes the gap.

This also settles a question raised in Section 5.4.3, where the way precision and recall moved when the training data was doubled pointed at max-pooling as the main limit. At the 0.5 threshold the best other rule gains only 0.019 micro-F1, and even the best pair of rule and threshold found on the test set gains 0.061 and stays below the baseline. The way window scores are combined matters, but it is a secondary factor.

This test has limits. It changes only the rule applied to the window scores. It does not change the window size (2,000 characters, step 1,500), which would need new windows and retraining, and it does not try a learned rule such as attention pooling over the windows. Both remain untested. Attention pooling is the obvious next step, since it could learn to weigh windows by how relevant they are.

5.3.2 The Threshold and What the Confidence Score Means

In the first setup, every presence decision came from comparing a score with 0.5. Section 4.4.1 notes that nobody chose this threshold; it is the default for a two-class model. Clausify no longer uses it: it compares each score with a threshold chosen for that clause type on the validation contracts (Section 5.3.3). Figure 5.4 tests whether 0.5 is a good place to draw the line and whether the score compared with it means anything.

0.5 is a poor operating point for the transformer

Trying other thresholds after the fact, the transformer reaches a micro-F1 of 0.753 at a threshold of 0.9, against 0.692 at the default. The gain of 0.061 closes most of its gap with the TF-IDF baseline (0.775). Section 4.2.1 predicted the reason: the maximum over about 35 windows is pushed upwards by design, so the scores bunch up near 1 and a line at 0.5 lets far too much through. Precision is 0.558 at 0.5 and 0.726 at 0.9.

We are not switching to 0.9. It was found by looking at the test set, and using it would turn a held-out test into a tuned one, which Section 4.4.1 says this project avoided. What we can say is that part of the transformer’s gap comes from a

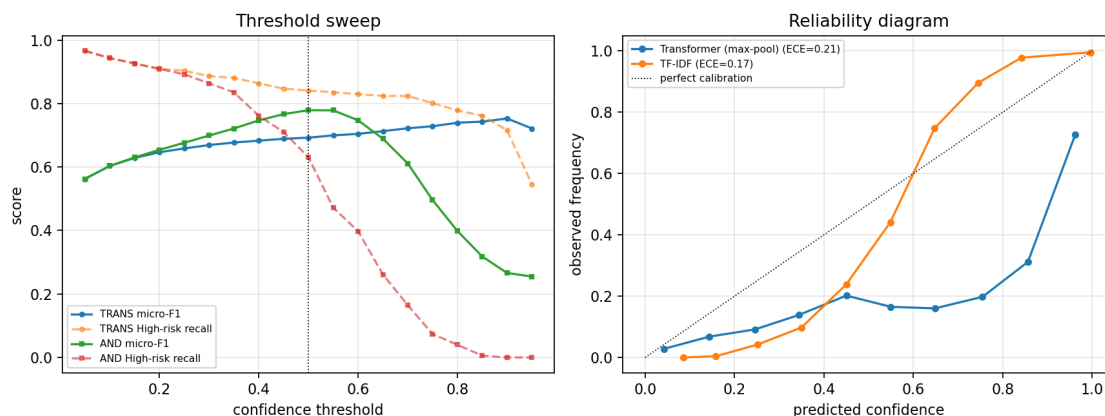


Figure 5.4: Threshold and calibration

threshold nobody chose, and part comes from how well it tells clauses apart. The comparison in Section 5.4.1 therefore sets a baseline whose scores already suit 0.5 against a transformer whose scores do not. A fair rematch chooses each model’s threshold on a validation split taken from the 408 training contracts. Section 5.3.3 runs it, and with the thresholds chosen there the transformer ties the baseline.

The AND-ensemble behaves the other way. Its best threshold is already 0.5 (0.779), and raising it makes things much worse. At 0.7 micro-F1 falls to 0.611 and High-risk recall to 0.165, because asking both models to pass a high bar is close to asking both to be very sure at once. The dashed curves in Figure 5.4 show the conflict. For the transformer, the threshold with the best micro-F1 (0.9) lowers High-risk recall from 0.841 to 0.716, so no single threshold is best for both the overall score and the clauses that matter most.

The raw score is not a probability

The first version of Clausify showed the raw presence score to the user as a confidence bar. A reliability diagram tests whether such a bar means what a user would think it means, and the raw score does not.

The transformer’s score has an Expected Calibration Error [33] of 0.205 and a Brier score of 0.195, and it is too confident across the whole range. Among predictions between 0.5 and 0.6 the clause is really there only 16.5% of the time. Even in the top group (scores above 0.9, which the app shows as an almost full bar) it is there 72.6% of the time, so a user who reads “96% confident” is looking at about a 73% chance. Max-pooling is again the cause. The maximum over dozens of windows is not a probability, and nothing in training asked it to be one.

The TF-IDF baseline is also off (ECE 0.172), mostly in the safer direction: in the middle range it is less confident than it should be. Its predictions between 0.7 and 0.8 are right 89.6% of the time, and those between 0.8 and 0.9 are right 97.8% of the time.

For that first version of the app, the score on every card was therefore not a probability of being right. The fix needs no retraining. Platt scaling or isotonic regression, fitted on a validation split, would turn the scores into real probabilities, and it is the

cheapest single improvement to the interface. The original runs had no validation split to fit it on (Section 4.4.1).

The re-run of Section 5.3.3 sets one aside, so we fitted the map there. On the 81 validation contracts we fitted isotonic regression, a monotone map from the re-run transformer’s highest window score to the share of times the clause is really present, and applied it unchanged to the test contracts. The raw score has an Expected Calibration Error of 0.206 on the test set, almost the same as the first setup’s model. After the map it is 0.016, and the Brier score falls from 0.195 to 0.091. For comparison, Platt scaling, which fits a single logistic curve, reaches an Expected Calibration Error of 0.084 and a Brier score of 0.103, so isotonic regression is better on both measures. The map keeps the order of the scores, so it changes no detection. Clausify now shows this calibrated chance on every card, limited to the range 1% to 99%, in place of the raw score. On the test set, contracts whose raw score was at least 0.99 contained the clause 99.0% of the time, and those between 0.90 and 0.95 only 32.3% of the time.

5.3.3 Reading the Whole Window, and Tuning Both Models the Same Way

The presence transformer reads each (question, window) pair up to a limit of 256 tokens. The CUAD question alone takes a median of 53 tokens, so only about 200 tokens are left for the 2,000-character window. Measured with the trained tokenizer, the model reads a median of 49.3% of each window (about 986 characters; 41% to 57% for the middle 80% of windows) and never sees the rest. Windows start every 1,500 characters, so about a third of every contract is never read by any window. Training was hurt as well. A window is labelled positive when it contains a marked clause anywhere in its 2,000 characters, and in 23.2% of positive training windows (4,875 of 21,028) the clause starts after the point where the model stops reading. Those examples taught the model to say “present” about text that did not contain the clause.

None of the checks earlier in this chapter could find this, because each one changed something else while keeping the 256-token cut. The comparison with the baseline had a second weakness, which Section 5.3.2 already pointed out: the transformer was held to a 0.5 threshold that suits its scores badly, and no setting of either model was chosen on data other than the test set.

What we changed

We fixed both problems and ran the presence comparison again. The input limit is now 512 tokens, which covers the question and the whole window (100% of the window is read, and only 0.1% of positive training windows still have the clause past the cut). We also set aside a validation split: 81 of the 408 training contracts, chosen at random with seed 42, are held out, and the models are trained on the other 327. The 102 test contracts are the same as before. Everything else about the transformer is unchanged (2,000-character windows, 24,000 training windows, DistilBERT, learning rate 2×10^{-5} , seed 42), except that it trained for three epochs

and kept the epoch with the lowest validation loss (the second; the losses after the three epochs were 0.252, 0.245 and 0.259). Training took 191 minutes, and scoring every window of the validation and test contracts another 198 minutes.

Both models were then tuned on the validation contracts in the same way, with every rule fixed before any test result was seen:

- TF-IDF baseline: refitted on the 327 contracts, with the regularization strength C chosen from $\{0.1, 0.3, 1, 3, 10\}$.
- Transformer: the way window scores are combined, chosen from the maximum and the mean of the top 2, 3 or 5 windows.
- Thresholds: one threshold per model chosen on validation, then a separate threshold for each category with at least eight validation positives. Rarer categories keep the shared threshold so that they are not fitted to noise.
- Ensemble: the rule chosen from AND, OR, the average of the two scores, and a mixed rule (OR for High-risk categories, AND for the rest).

Each choice was made twice, once to maximize micro-F1, as in every earlier result, and once to maximize micro-F2. F2 treats recall as twice as important as precision, so it counts a missed clause as worse than a false alarm, which Section 3.4 argues is right for this tool. The test set was scored once for each finished setup. Table 5.17 gives the results, with 95% intervals from the cluster bootstrap over test contracts.

Table 5.17: Presence models after the re-run

Setup	Model	micro-F1 [95% CI]	P	R	F2	High-risk found (missed)
First setup: 256 tokens, 408 contracts, threshold 0.5	TF-IDF baseline	0.775	0.742	0.810	0.796	64.8% (62)
	Transformer	0.692	0.558	0.913	0.810	84.1% (28)
	AND-ensemble	0.779	0.774	0.784	0.782	63.1% (65)
Whole window, no tuning: 327 contracts, threshold 0.5	TF-IDF baseline	0.773 [0.755, 0.790]	0.743	0.805	0.792	65.3% (61)
	Transformer	0.717 [0.699, 0.734]	0.575	0.953	0.842	88.6% (20)
Tuned on validation for micro-F1	TF-IDF baseline	0.785 [0.768, 0.801]	0.750	0.824	0.808	67.6% (57)
	Transformer	0.797 [0.781, 0.812]	0.740	0.864	0.836	67.6% (57)
	Ensemble (AND)	0.809 [0.793, 0.825]	0.864	0.761	0.780	54.0% (81)
Tuned on validation for micro-F2 (recall first)	TF-IDF baseline	0.743 [0.727, 0.759]	0.631	0.904	0.832	79.5% (36)
	Transformer	0.757 [0.740, 0.773]	0.638	0.930	0.852	90.3% (17)
	Ensemble (AVG)	0.783 [0.768, 0.798]	0.681	0.922	0.861	86.9% (23)

Note. All rows are scored on the same 102 test contracts (4,182 decisions, 176 High-risk clauses).

The first three rows are the original models of Table 5.1. Rule and threshold choices, made on the 81 validation contracts: TF-IDF $C = 10$ for both goals; maximum pooling for both goals; shared transformer threshold 0.96 (F1) and 0.88 (F2); ensemble rule AND (F1) and the average of the two scores (F2).

What the re-run shows

Tuning both models the same way, with the transformer reading the whole window, changes the answer to RQ1. Tuned for micro-F1, the transformer scores 0.797 against 0.785 for the tuned baseline. Its lead of +0.012 has an interval of $[-0.005, +0.030]$ and appears in 91.2% of resamples, so on micro-F1 the two models

are tied. The transformer is clearly better on the categories with enough data to judge: over the 34 categories with at least ten test positives, its macro-F1 is 0.711 against 0.673. Tuned for micro-F2, it beats the tuned baseline by +0.020 (interval [+0.004, +0.036]).

The re-run changed three things at once: the input length, the training contracts (327 instead of 408, with the epoch chosen on validation) and the tuning. To separate them we ran the whole protocol a second time with the input length left at 256 tokens, so that the length is the only difference (Table 5.18). Without tuning, the 256-token model scores 0.693, the same as the original 0.692, so the change of training contracts and epoch has no measurable effect. Every other row shows the length alone helping, on the same test contracts and with the same tuning. The paired bootstrap gives +0.024 micro-F1 without tuning (interval [+0.014, +0.035]), +0.036 when both are tuned for micro-F1 ([+0.021, +0.051]), and +0.028 micro-F2 when both are tuned for recall ([+0.015, +0.041]), where the share of High-risk clauses found rises by 8.0 points ([+3.4, +12.9]). Tuning alone is not enough. At 256 tokens the tuned transformer still loses to the tuned baseline, by 0.024 (interval [−0.041, −0.007]). The earlier gap came from both flaws, and closing it needed both fixes: the window had to be read in full, and the threshold had to suit the scores.

Table 5.18: Effect of reading the whole window

Tuning	Model	256 tokens (half the window)		512 tokens (whole window)	
		micro-F1 / F2	High-risk found	micro-F1 / F2	High-risk found
None (threshold 0.5)	Transformer	0.693 / 0.815	85.8% (25)	0.717 / 0.842	88.6% (20)
For micro-F1	Transformer	0.761 / 0.800	64.8% (62)	0.797 / 0.836	67.6% (57)
	Ensemble	0.810 / 0.829	67.6% (57)	0.809 / 0.780	54.0% (81)
For micro-F2	Transformer	0.716 / 0.824	82.4% (31)	0.757 / 0.852	90.3% (17)
	Ensemble	0.777 / 0.826	72.7% (48)	0.783 / 0.861	86.9% (23)

Note. Both columns use the same 327 training contracts, the same 81 validation contracts, the same tuning rules and the same 102 test contracts. Rules chosen on validation for the ensembles: averaging (256, F1), AND (512, F1), AND (256, F2), averaging (512, F2). The 256-token transformer tuned for micro-F2 combines windows with the mean of the top two; all other transformer rows use the maximum.

The ensemble claim that Section 5.4.1 withdrew now holds, and the control shows that it comes from the tuning. Tuned for micro-F1, the ensemble scores 0.809 with the 512-token transformer (the validation split chose the AND rule) and 0.810 with the 256-token one (it chose averaging). With 512 tokens it leads the tuned baseline by +0.024, with an interval of [+0.011, +0.037] that excludes zero, and the lead is 3.8 times the baseline’s split-to-split standard deviation (Section 5.4.4). The first setup’s ensemble, with both thresholds fixed at 0.5, could not be told apart from the baseline. With thresholds chosen on validation it can. We have not measured the ensemble’s own variation across splits or seeds after the re-run.

The conflict between micro-F1 and the purpose of the tool from Section 5.2.2 appears again, and this time it can be resolved without looking at the test set. The setup with the best micro-F1 finds only 54.0% of High-risk clauses. The recall-first setups, chosen on validation before the test was scored, find far more. The ensemble that averages the two scores finds 86.9% (missing 23 of 176) at a micro-F1 of 0.783, so it beats the original headline setup on both measures (0.779 and 63.1%). The

transformer alone finds 90.3% (missing 17), the most of any setup, at a micro-F1 of 0.757. Both recall-first setups beat the tuned baseline on micro-F2, the measure they were tuned for (the ensemble by +0.029, interval [+0.018, +0.040]). Reading the whole window matters most for these recall-first setups: with 256 tokens, neither recall-first setup beats the tuned baseline, and they find only 82.4% and 72.7% of High-risk clauses.

What the re-run does not change

The TF-IDF baseline is still strong. Tuned, it reaches 0.785, and on micro-F1 the transformer’s lead over it is not proven. The transformer’s raw scores are still badly calibrated: the shared threshold chosen on validation is 0.96 for micro-F1 and 0.88 for micro-F2, so its scores still bunch near 1. A calibration map fitted on the validation contracts fixes this for the app (Section 5.3.2). The whole-system test, the risk-weighted breakdown, the error examples and the rare categories were first run on the original models. Sections 5.2.1 to 5.2.4 now report the re-run setups next to them. The seed, training-budget and split checks of Section 5.4.1 onwards have not been repeated for the re-run model.

Which setup the app uses

Clausify’s default is the recall-first transformer alone, with the per-category thresholds chosen on validation. It finds the most High-risk clauses on the test set, and it does not depend on the TF-IDF model, which does not transfer well to the short contracts a user is likely to paste. The TF-IDF model learnt its word statistics from long SEC filings. The 7,033-character demonstration contract of Section 4.4.9 has section headings that name four High-risk clause types (*Exclusivity*, *IP Ownership Assignment*, *Non-Compete* and *Liquidated Damages*). The TF-IDF model scores its intellectual-property assignment clause at 0.08, so the averaging ensemble misses it and reports three of the four. The transformer alone reports all four. It also reports *Uncapped Liability* for the liability clause, which in fact caps liability, and it misses the revenue-sharing and warranty-period clauses (Sections 8 and 10 of that contract). The first setup’s model made three High-risk false alarms on the same contract: the same *Uncapped Liability*, plus *Minimum Commitment* and *Irrevocable or Perpetual License*, which the contract does not contain either. The averaging ensemble and the AND rule remain in the app as settings, described with their test-set results.

5.4 Statistical Analysis

Our main statistical tool is a paired bootstrap over contracts [30]. We resample the 102 test contracts with replacement 10,000 times and score every model on the same resample, so the 41 linked decisions from each contract stay together. We then measure three sources of variation the bootstrap cannot see: the amount of training data, the random seed and the train/test split.

5.4.1 How Much of That Ranking Is Real?

Table 5.1 is a ranking, and readers tend to treat the order of a ranking as a result. A lead of 0.779 over 0.775 on 102 test contracts is about four decisions in a thousand, too small to take at face value, so we measured it.

A bootstrap confidence interval is the natural tool, but what we resample matters. The 4,182 test decisions are not independent. Each contract contributes 41 of them, and an unusual contract (very long, or oddly written) moves all 41 together. Re-sampling single (contract, category) decisions would ignore this and give intervals that are far too narrow. We therefore resample whole contracts with replacement (a cluster bootstrap): we draw 102 contracts from the 102 we have, recompute micro-F1 for both models, and repeat 10,000 times. Both models are scored on the same resample each time, so the difference is paired, and its interval is much narrower than the two separate intervals would suggest.

Table 5.19: Cluster bootstrap results

Comparison	Difference in micro-F1	95% CI	$P(> 0)$
AND-ensemble vs. TF-IDF baseline	+0.0042	$[-0.0037, +0.0115]$	0.855
TF-IDF baseline vs. transformer	+0.0824	$[+0.0647, +0.1013]$	1.000
AND-ensemble vs. transformer	+0.0866	$[+0.0705, +0.1035]$	1.000

Note. 10,000 resamples. $P(> 0)$ is the share of resamples in which the first model wins.

The three comparisons come out differently.

The ensemble’s win over the baseline is not proven. The interval is $[-0.0037, +0.0115]$, which includes zero, and the ensemble is ahead in 85.5% of resamples, more often than not but well short of the 97.5% a clear win would need. On their own the two models score 0.779 (95% CI $[0.762, 0.795]$) and 0.775 (95% CI $[0.758, 0.791]$), and the two intervals almost completely overlap. On this test set we cannot tell the AND-ensemble and the TF-IDF baseline apart, so the top two rows of Table 5.1 should be read as a tie.

The transformer’s loss is real for this setup. The baseline beats the window-based transformer by 0.0824, with an interval of $[+0.0647, +0.1013]$ that is nowhere near zero, and it wins in all 10,000 resamples. The two claims about the first setup therefore stand on different ground: the baseline’s win over the transformer passes the test easily, and the AND-ensemble’s lead over the baseline does not. Section 5.3.3 reverses both after the re-run. The transformer ties the tuned baseline, and the tuned ensemble leads it with an interval that excludes zero.

The case for the ensemble never rested on 0.004 of micro-F1, so it survives this. The AND rule earns its place by the kind of mistakes it makes: it turns the transformer’s many false alarms into balanced precision and recall (Table 5.3). Sections 5.4.2 and 5.2.4 look at what this costs.

5.4.2 How Much Changes If We Only Change the Random Seed?

Section 5.4.1 measured how much the score depends on which contracts ended up in the test set. A second kind of uncertainty is invisible to it. Every result so far comes from one training run per model, and training is partly random: the starting weights of the classifier, the order of the data and the sample of negative windows all depend on the seed. If changing only the seed moves the score as much as the effects we discuss, a ranking built on single runs is reporting noise.

This is easy to check. We trained the presence transformer three times with 24,000 training examples, changing only the seed (42, 43 and 44), and tested each run on the same 102 contracts. Seed 42 is the October 2026 run used in the rest of this chapter; seeds 43 and 44 were trained in August 2026 with the same settings. Table 5.20 shows the spread.

Table 5.20: Seed variance

Model	Seed 42	Seed 43	Seed 44	Mean	SD	Range
Transformer (max-pool)	0.6924	0.6928	0.7082	0.6978	0.0090	0.0158
AND ensemble	0.7789	0.7774	0.7803	0.7789	0.0015	0.0029
TF-IDF baseline	0.7747 (deterministic)			0.7747	–	–

Note. All three runs use 24,000 training examples and differ only in seed. Seed 42 is the October 2026 run; seeds 43 and 44 are August 2026 runs. The TF-IDF baseline has no randomness, so it appears once.

Seed noise does not explain the ensemble’s lead, but other noise can. The AND-ensemble leads the baseline by +0.0042. Across seeds the ensemble’s standard deviation is only 0.0015 and its range is 0.0029, so the lead is larger than the seed-to-seed spread, and the ensemble beats the baseline with every seed (by up to +0.0056 with seed 44). This is a change from the August models, whose lead (+0.0014) was smaller than the seed noise. Seed noise, however, is the smallest uncertainty we measured. The bootstrap interval still includes zero (Section 5.4.1), and a different train/test split moves the baseline by 0.0063 (Section 5.4.4), 1.5 times the lead, so we still read the top two rows as a tie.

Seed noise does not explain the transformer’s loss in the first setup. The baseline beats the transformer by 0.0824, which is 9.2 times the transformer’s seed standard deviation of 0.0090. Even the luckiest seed, at 0.7082, still loses to the baseline by 0.0665. Section 5.4.3 shows that more than doubling the training data does not close the gap either. The explanation was in the setup: Section 5.3.3 finds that the model read only about half of each window.

The noisier model is the transformer alone, which varies about six times as much across seeds (SD = 0.0090) as the ensemble that contains it (0.0015). Requiring agreement with a fixed word model calms the transformer’s run-to-run changes along with its false alarms. A false detection has to get past a second opinion that never changes, so most of the seed-driven changes in what the transformer reports are filtered out. This steadiness is a real property of the AND rule, although Sec-

tion 5.2.4 shows it costs recall on rare categories, and it is a better argument for the ensemble than its rank.

This check covers only the presence transformer. The span extractor and the summarizer were each trained twice with the same seed (August and October 2026) and gave the same scores to within 0.001 (token-F1 0.763 and 0.764; ROUGE-L 0.775 both times). That limits run-to-run noise on this hardware but says nothing about seed variance, which is still unmeasured for those two models. We list it in Section 6.3 instead of assuming it is small.

5.4.3 Was the Transformer Simply Undertrained?

One objection to Section 5.4.1 cannot be answered by the bootstrap, and it concerns the project’s main claim. The TF-IDF baseline is trained on all 408 training contracts. The window-based transformer used a balanced sample of 24,000 of the 53,662 available training windows, to keep training possible on a laptop (Section 4.4.3). When the transformer loses by about 0.08 micro-F1, the comparison alone cannot say whether max-pooling over windows is weaker for this task or whether the transformer simply never saw 55% of its own training data. The first would be a finding about the method and the second a side effect of our computing budget, and they lead to very different conclusions.

To settle this, in August 2026 we retrained the presence transformer on all 53,662 training windows and kept everything else the same: windows, negative sampling, settings, seed and test. The run took 140.7 minutes on the same laptop, against about 50 minutes for the smaller version. Table 5.21 shows the result. Both runs in the table are from August 2026, so they are compared with each other and not with the October numbers.

Table 5.21: Training-budget control

Model	Training windows	micro-F1	Precision	Recall
Transformer (max-pool)	24,000 (44.7%)	0.6943	0.5620	0.9080
Transformer (max-pool)	53,662 (100%)	0.6939	0.5546	0.9266
TF-IDF baseline	all 408 contracts	0.7747	0.7424	0.8101
<i>Paired differences (95% CI over 10,000 contract resamples)</i>				
Full-data transformer – subsampled transformer		−0.0004	[−0.0098, +0.0095]	
TF-IDF baseline – full-data transformer		+0.0809	[+0.0649, +0.0980]	

Note. August 2026 runs, both tested in the same way on the 102 test contracts; differences use the paired bootstrap of Section 5.4.1. The October 2026 re-run of the 24,000-window model scores 0.6924.

Too little training data does not explain the gap. Using 124% more training data changed micro-F1 by −0.0004, with an interval of [−0.0098, +0.0095] that sits across zero, so the two budgets cannot be told apart. The gap to the baseline also stayed put: +0.0809 with all the data against +0.0805 with the sample (August runs), with an interval nowhere near zero and the baseline winning in all 10,000 resamples.

How the extra data changed things is still informative. Recall rose from 0.9080 to 0.9266 and precision fell from 0.5620 to 0.5546, so more training data made the model report clauses more readily without making it more accurate. Section 4.2.1 predicted this weakness of max-pooling. With about 35 windows per contract, the decision for the whole document is a maximum over many chances to be wrong, and a better-trained window scorer that is still imperfect produces more false maxima. Section 5.3.1 tests this directly and finds that the way window scores are combined is a secondary factor, not the main limit: at the 0.5 threshold the best of five other rules gains only 0.019 micro-F1, and no rule at any threshold reaches the baseline.

We also saw a smaller effect that leaves our conclusion unchanged. The AND-ensemble built on the full-data transformer scores 0.7820, and its lead over the baseline is +0.0073 with a 95% interval of $[-0.0000, +0.0140]$. It wins in 97.4% of resamples, but the interval still touches zero, so more training data moves the ensemble’s lead from a clear tie to a borderline one. We do not claim it, because an interval whose lower end sits on zero does not pass the bar we set in Section 5.4.1, and we will not lower the bar because the number moved our way. The ensemble’s balance between precision and recall stays the same with more data.

The first-setup results in this chapter (the confusion matrix, the per-category results and the first-setup rows of Section 5.2.1 onwards) use the October 2026 model trained on 24,000 examples. Clausify uses the re-run model of Section 5.3.3. The full-data run is a control experiment that tests one possible explanation, and we report it that way instead of swapping in a slightly better number afterwards. The two cannot be told apart, so nothing later would change if we had.

5.4.4 A Third Source of Variation: The Split Itself

Sections 5.4.3 and 5.4.2 tested two things that could move our numbers, the amount of training data and the random seed, and neither changed the conclusions. A third source may be the largest of all: the split. Every result in this project comes from one contract-level 80/20 split made with seed 42. With only 510 contracts, a different split puts different rare categories in the test set and could move the scores by itself. Seed variance does not capture this, because changing the training seed keeps the test set the same.

Retraining the transformer on twenty splits is too expensive, but the TF-IDF baseline retrains in under a minute. Every comparison in this project is made against it, so its spread across splits shows how far the comparison itself could move. We made 20 separate contract-level 80/20 splits and retrained the baseline on each.

Table 5.22: Split variance

	Mean	SD	Min	Max	Range
micro-F1 over 20 splits	0.7829	0.0063	0.7702	0.7918	0.0216
<i>The split used throughout this project (seed 42): 0.7747</i>					

The split is the largest of the three uncertainties we measured. Its standard deviation is 0.0063, against 0.0015 for the ensemble’s seed variance and a bootstrap half-width

of about 0.008 for the comparison. That is 1.5 times the ensemble’s +0.0042 lead, so a different split could easily remove the lead, which gives one more reason to treat it as unproven.

Our split is not a lucky one. At 0.7747 the baseline sits below the 20-split mean of 0.7829, in the lower part of the range, so if anything this project reports the baseline on a slightly unfavourable split. A split that happened to favour the baseline would have been a real worry, since it would make the transformer’s loss look worse than it is.

In the first setup, the baseline’s lead over the transformer (0.0824, about thirteen split-level standard deviations) is far outside this variation, so no realistic new split would reverse it. The re-run in Section 5.3.3 is a different matter. There the tuned ensemble leads the tuned baseline by 0.024, about 3.8 split-level standard deviations, and the transformer’s lead of 0.012 is within two.

We can say nothing about how stable single categories are across splits, and that is where the split would matter most. A rare category with two test examples in one split may have eight in another, and its F1 means little in either case. Repeated splits or cross-validation for the whole system remain future work, and Section 6.4 lists this as a limitation.

5.5 Comparisons and Relationships

Table 5.23 collects the test results of all three models. The separate test notebook produced every number of the first setup, running on the saved models trained on the 80/20 split, and `retrain/v2_fullwindow/tune_and_test.py` produced the re-run numbers.

Table 5.23: Final test results

Stage	Model	Metric	Score
2A Presence, re-run	Ensemble, tuned for F1 (AND)	micro-F1	0.809
2A Presence, re-run	Ensemble, tuned for F1 (AND)	Accuracy / Specificity	88.43% / 94.28%
2A Presence, re-run	Ensemble, tuned for F1 (AND)	Precision / Recall	86.36% / 76.11%
2A Presence, re-run	Transformer, recall-first (app default)	micro-F1 / High-risk found	0.757 / 90.3%
2A Presence, app	The same, reading what the app reads	micro-F1 / High-risk found	0.765 / 88.6%
2A Presence, re-run	Ensemble, recall-first (AVG)	micro-F1 / High-risk found	0.783 / 86.9%
2A Presence, first setup	AND-ensemble (TF-IDF + DistilBERT)	micro-F1 / Accuracy	0.779 / 85.65%
2A Presence, first setup	AND-ensemble	Precision / Recall	77.38% / 78.41%
2B Span	DistilBERT-QA, first model	token-F1 (centred window with the answer)	0.764
2B Span	DistilBERT-QA, first model	token-F1 (window chosen by 2A, app setting)	0.350
2B Span	DistilBERT-QA, retrained (app)	token-F1 (window chosen by 2A, app setting)	0.673
2B Span	DistilBERT-QA, first model	Exact Match (centred window)	35.5%
2B Span	DistilBERT-QA, retrained (app)	token-F1 (plain window with the clause)	0.779
1 Summarizer	FLAN-T5-small	ROUGE-L (41 templates)	0.775
1 Summarizer	FLAN-T5-small	ROUGE-L (clause-specific)	0.116
Full pipeline	all three steps, first span model	real clauses that get through	22.1%
Full pipeline	all three steps (app setting)	real clauses that get through	59.6%

Table 2.1 in Chapter 2 compares this project with earlier work. The original CUAD study reports span extraction with DeBERTa-xlarge, scored over whole documents with a precision-at-recall measure. Our models are about ten times smaller, and our span scores are measured inside windows that contain the answer, so the numbers cannot be compared directly and we do not present them that way. The scope

does differ. Earlier work stops at finding clauses, while this project adds presence decisions, a risk level for each category and plain-English output, and tests the whole system from start to finish.

Among the presence models, the TF-IDF baseline and the window-based transformer make different kinds of mistakes. The transformer finds more clauses but raises many false alarms (recall 0.913, precision 0.558), while the baseline is more careful (recall 0.810, precision 0.742). This is why the AND-ensemble, which needs both, matches the baseline with balanced precision and recall. After the re-run the transformer reads whole windows and both models are tuned, and the same pairing beats the baseline outright (Section 5.3.3). Across the steps, mistakes in the window chosen by the presence step carry over into span extraction, which is why the whole-system rate is far below the product of the separate scores.

5.6 Discussions

The results of this chapter fall into four groups.

The main presence result changed during the project, and the change is itself a finding.

1. In the first setup, a word-based baseline that reads the whole document beat the window-based DistilBERT by +0.082 micro-F1, and the gap survived every check we ran: all 53,662 training windows instead of 24,000 (Section 5.4.3), three seeds (Section 5.4.2), six ways of combining window scores (Section 5.3.1) and a range of thresholds (Section 5.3.2).
2. Those checks all shared one flaw: the transformer read only about half of each window, and 23.2% of its positive training examples were labelled for text it never saw. With the whole window read and both models tuned on the same validation split, the transformer ties the tuned baseline on micro-F1 (0.797 against 0.785, interval $[-0.005, +0.030]$) and beats it on macro-F1 over the common categories (0.711 against 0.673) and on the recall-first score (Section 5.3.3).
3. The tuned ensemble beats the tuned baseline, 0.809 against 0.785, with an interval of $[+0.011, +0.037]$. Tuned for recall instead, the ensemble finds 86.9% of High-risk clauses at a micro-F1 of 0.783, and the transformer alone finds 90.3%, against 63.1% for the first setup’s headline ensemble.

Two further findings held throughout.

1. For finding the clause text, a bag-of-words model is no help because it cannot point to positions in the text, so the transformer is needed here. The first span model did well only on windows centred on the answer (token-F1 0.764, 95% CI $[0.743, 0.782]$) and reached 0.366 on plain windows. Trained on the chunks it really reads, the same small model reaches 0.779 on plain windows (Section 5.2.1).

2. For summaries, a set of 41 template sentences turns the task into classification. The fine-tuned model copies a template word for word for 100% of test clauses, and against clause-specific references it scores no better than the template alone (Section 5.1.3). A high automatic score should be checked by reading the outputs.

For the first setup we withdrew one claim. The AND-ensemble’s first place in Table 5.1 is not proven: its lead over the baseline is +0.0042, the bootstrap over contracts puts it inside $[-0.0037, +0.0115]$ (Section 5.4.1), and a different split moves the baseline by 0.0063 (Section 5.4.4). After the re-run the claim holds, with a lead of +0.024 whose interval excludes zero.

Two costs only showed up once we looked for them, and neither is flattering.

- In the first setup, the AND rule does not help everywhere. On the ten rarest categories it gains nothing in F1 over the transformer alone (0.364 against 0.368): it buys 34.6 points of precision with 30.0 points of recall and cuts the clauses found from 39 of 70 to 18 of 70 (Section 5.2.4). All of its overall benefit comes from the common categories, and one of the categories it scores zero on is High-risk.
- The separate scores do not multiply. Only 22.0% of real clauses got through the whole system with the first setup and 22.1% with the app’s setting, against about 65% and 77% predicted by multiplying the separate scores (Section 5.2.1). There were two causes: the presence model’s highest-scoring window was the wrong window 30.9% of the time, and the span model was trained on centred windows it never sees in real use. Reading the whole window cut wrong-window choices to 17.4%, and retraining the span model on the chunks it really reads raised the app’s figure to 59.6% (Section 5.2.1). What is left is split about evenly between the two causes.

The numbers in this chapter that we first trusted least (the ensemble’s rank, the summarizer’s ROUGE, the span model’s token-F1) each turned out to measure something narrower than it seemed. Each time, we found out by measuring the thing the score stood for: we resampled the contracts, changed the seed, changed the reference sentences, or ran the steps in a chain. None of these checks needed a bigger model or more data.

5.6.1 Usability

We did not run a usability study. The app’s main claim, that grouping clauses by risk helps a non-lawyer act on a contract, is therefore untested, and Section 6.3 lists it under construct validity.

5.6.2 Honest Limitations of the Demo

By default the app finds clauses with the re-run transformer alone, using the recall-first thresholds chosen on validation (Section 5.3.3). The two re-run ensembles are

offered as settings. The default finds the most High-risk clauses on the test set (90.3% on whole contracts, 88.6% as the app reads long contracts; see below) at the cost of the lowest micro-F1 of the three (0.757), because a missed High-risk clause is the mistake this tool exists to prevent (Section 3.4). The ensembles also depend on the TF-IDF model, which scores short contracts too low (Section 5.3.3).

The model’s raw score is not a probability. Section 5.3.2 shows that the original transformer’s score has an Expected Calibration Error of 0.205, and the re-run model’s thresholds (0.88 shared, chosen on validation) show that its scores still bunch near 1. The app therefore shows a calibrated chance instead: the isotonic map of Section 5.3.2, fitted on the validation contracts, turns each score into the share of validation contracts with that score that really contained the clause, and on the test set it reaches an Expected Calibration Error of 0.016. It was measured on CUAD contracts, so on contracts unlike CUAD’s it is only a guide, and the app says so.

The chance and the decision did not always agree. The recall-first thresholds are set for each clause type, but the calibration map is shared by all types, so a type with a low threshold can be reported at a chance well under 50%. On the 102 test contracts, scored whole, the default setting shows 1,963 cards. Of these, 728 (37.1%) give a chance below 50% and 204 (10.4%) below 20%; a *Non-Compete* card, for example, could appear as a found High-risk clause with a chance of about 8%. The low-chance cards still matter: 29.8% of the 728 are real clauses, and they include 46 of the 159 High-risk clauses the default setting finds, so dropping them would lower its High-risk recall from 90.3% to 64.2%. The app therefore keeps them but labels them *Possible* — *check* and lists them after the other cards of their risk level under their own heading. The risk summary at the top counts them, and the PDF and CSV reports mark them as well. Of the 1,235 cards at 50% or more, 83.9% are real clauses.

Long contracts are not read in full, and this changes the app’s own figures. To keep the app fast, every clause type is checked in the first 30 windows only (about 45,500 characters), and 38.4% of CUAD contracts (196 of 510) are longer than that. Every other number in this chapter scores whole contracts, so none of them shows the cost. We measured it by replaying the saved window scores of the recall-first transformer with the app’s limit (Table 5.24). Reading only the first 30 windows, the default setting finds 77.3% of High-risk clauses (40 of 176 missed) instead of 90.3%, and only 68.8% on the 38 test contracts that are longer than the limit.

Table 5.24: What the app reads in long contracts

Windows read	High-risk found (missed)	micro-F1	micro-F2	High-risk found, 38 long contracts
Every window (all other numbers in this chapter)	90.3% (17)	0.757	0.852	93.5%
First 30 windows (the app before)	77.3% (40)	0.738	0.801	68.8%
First 30, plus 5 per clause type picked by TF-IDF (the app now)	88.6% (20)	0.765	0.852	90.3%

Note. Recall-first transformer with its validation-chosen thresholds, on the 102 test contracts (176 High-risk clauses). The first two rows replay the saved window scores; the last row is measured through the app’s own presence code, and the replay gives the same figures.

Because the saved scores cover every window, other reading rules can be tested without running the model again. The rule the app now uses lets the re-run TF-IDF model, applied to each later window on its own, pick the 5 windows most likely

to contain each clause type, and the presence model reads those windows as well. The number 5 was chosen on the 81 validation contracts, as the smallest number of extra windows whose micro-F2 comes within 0.005 of reading every window (we tried 0 to 20 extra windows, and a ranking by TF-IDF similarity to the CUAD question instead). On the test contracts, measured through the app’s own code, the default setting then finds 88.6% of High-risk clauses (20 missed) and 90.3% on the long contracts, at a micro-F1 of 0.765 and the same micro-F2 as reading every window (0.852). The cost is at most 5 more windows for each clause type, about 15% more time on a long contract (Table 4.6). The app still never reads most of a very long contract: for the longest test contract (289,615 characters, 193 windows) each clause type is checked in at most 35 windows, and the notice above every analysis says so. The quoted text is also less reliable than the label. With the first span model, the extracted text overlapped the real clause in only about a fifth of cases overall (Section 5.2.1). The app now uses the retrained span model and quotes the paragraph around its answer, and on the test contracts 71.7% of real clauses get a card whose quote contains at least half of the clause (Section 5.2.1). The other 28.3% are clauses whose category was missed, whose window was chosen wrongly, or whose quoted paragraph is the wrong one, so for a user the risk badge and the category name remain more reliable than the quoted text under them, and the app says this too. A card means “this contract seems to contain a clause of this type; look here” and should not be read as “this is the clause”. The risk levels are also set per category. The risk table says a *Cap on Liability* clause needs attention, but it does not check whether a particular cap is generous or harsh. The notice above every analysis states all of this. The demonstration does show that the models work on contracts they have never seen, and that the output can be presented in a form a non-lawyer can act on.

Chapter 6

Conclusion

6.1 Summary of Findings

We set out to turn CUAD’s clause-finding benchmark into something a person signing a contract could use: a system that finds the clauses, says how risky they are, and explains them in plain English. On a strict contract-level 80/20 split, the final presence ensemble reaches a micro-F1 of 0.809 and 88.43% accuracy for finding which clauses are present, and a recall-first setting finds 90.3% of High-risk clauses. The retrained span model extracts the clause text with a token-F1 of 0.779 on any window that contains it, and 59.6% of real clauses get through the whole system. The system runs live in the *Clausify* app.

Answering our five research questions (Section 1.3.1) gave two results that changed during the project and one negative one:

- The window-based transformer first appeared to lose to the simple word model (RQ1), by 0.082 micro-F1, and no change of training data, seed, pooling rule or threshold closed the gap. All of those runs shared one flaw: the model read only about half of each window, and 23.2% of its positive training examples were labelled for text it never saw. Reading the whole window and tuning both models on the same validation split, the transformer ties the tuned baseline (0.797 against 0.785), the ensemble beats it (0.809, interval of the lead [+0.011, +0.037]), and recall-first settings find 86.9% to 90.3% of High-risk clauses, against 63.1% for the first setup’s headline ensemble (RQ1 and RQ5).
- The summarizer does not summarize (RQ3). It outputs a category template word for word for every input, and a fixed lookup table matches its score against clause-specific references. We describe it everywhere as a proof-of-concept “find the category, show its template” component.
- The system first lost most of what its parts found (RQ4). Only 22.1% of real clauses got through all three steps with the app’s setting (22.0% with the first setup), against about 77% predicted from the separate scores. Both causes were in our setup. Reading the whole window cut wrong-window choices from 30.9% to 17.4%, and retraining the span model on the chunks it really reads, instead of windows centred on the answer, raised extraction in the right

window from 28.0% to 76.4%. Now 59.6% of real clauses get through, and what is left is split about evenly between the two causes. Neither fix needed a larger model.

RQ5 gave the result we least expected, and we think it is the most useful one for anyone building a system like this. Our evaluation score and our stated goal did not match. In the first setup the AND-ensemble wins on micro-F1 but misses 36.9% of High-risk clauses, while the OR rule, which comes last on micro-F1, misses only 14.2%. After the re-run the same conflict appears (the setup with the best micro-F1 finds only 54.0% of High-risk clauses), and this time we resolved it properly, by fixing a recall-first goal in advance and choosing the setup on a validation split. Half of the clauses behind the overall score are Low-risk ones such as dates, party names and governing law, so a score like micro-F1 should not choose a system built to warn a signer.

Five lessons seem likely to apply beyond this project.

First, a strong simple baseline deserves to be taken seriously, and so does the setup of the model it is compared with. We tested one explanation after another for the transformer’s loss and missed the one that mattered, because every run kept the same 256-token cut. A comparison is fair only when both sides read the same evidence and get the same chance to be tuned.

Second, a lead should be compared with its own noise before it is reported. We measured three sources: the bootstrap over contracts (half-width about 0.008), the model seed (SD 0.0015) and the train/test split (SD 0.0063). Two of the three are larger than the +0.0042 lead we first used to rank a table. The re-run ensemble’s lead of +0.024 clears the bootstrap and is 3.8 times the split SD.

Third, an automatic score measures its reference sentences as much as the task. ROUGE-L 0.775 became 0.116 when only the reference sentences changed.

Fourth, test the whole system as well as its parts, and test it against the harm it is meant to prevent. An average that treats a governing-law clause and an unlimited indemnity as equally important hides that harm.

Fifth, measure a limit before relying on it. A one-minute retrieval check (64%) ruled out a design before it cost any training time. The same habit, applied to sparse attention, corrected us: a Longformer training step does fit in this laptop’s memory at batch size 1, which is the opposite of what we had first claimed.

This project delivers a working prototype, a presence ensemble that beats a tuned baseline, and a detailed account of where the system fails. One gap keeps it a prototype: no user has ever been watched using the app, so its main claim, that a report sorted by risk helps a non-lawyer, is untested. Closing that gap needs a study with real users (Section 6.5).

6.2 Contributions to the Field

The main contributions of this work are:

1. A first risk table for the CUAD categories. Each of the 41 categories gets a High/Medium/Low level and a written reason, judged from the weaker party’s side (8 High, 22 Medium, 11 Low), with clear rules for how the levels were given (Section 4.2.4). It gives risk to a category and not to the wording of one clause, which is why we call it category-level risk sorting throughout.
2. A controlled comparison for finding clauses in long documents. A TF-IDF baseline that reads the whole document (micro-F1 0.775) first looked unbeatable: the window-based DistilBERT lost by 0.082, and more data, other seeds, other pooling rules and other thresholds did not close the gap. The cause was in our setup, since the transformer read only about half of each window. With the whole window read and both models tuned on the same validation split, they tie (0.797 against 0.785) and their ensemble wins (0.809, interval of the lead $[+0.011, +0.037]$). A control run that changes only the input length shows that the whole window is worth +0.036 micro-F1 to the tuned transformer, and that without it tuning alone leaves the transformer behind. Recall-first settings chosen the same way find up to 90.3% of High-risk clauses.
3. A measurement of how errors add up. Testing the three steps in a chain showed that only about 22% of real clauses got through the whole system, against 65–77% predicted from the separate scores. The test also found the two causes, a wrong choice of window and a mismatch between training and real use, and fixing both (reading the whole window, and retraining the span model on the chunks it really reads) raises the share to 59.6%. Neither fix needed a larger model.
4. A demonstration that a small set of template sentences turns summarizing into classification. Testing the same 150 clauses against our 41 category templates and against clause-specific references drops ROUGE-L from 0.775 to 0.116, and the fine-tuned model outputs a template word for word for 100% of inputs.
5. A simple safeguard: measure the limit before training. Question-guided retrieval of windows caps the possible recall at 64%, and that one measurement was enough to reject the retrieval approach before any training time was spent on it.
6. A system that can be rebuilt on a laptop. All three models train on a single Apple-Silicon laptop. The split, the trained weights and a separate test notebook are saved, so every number in this project can be produced again from disk.
7. Clausify, a prototype web app that runs the trained models live and shows the results grouped by risk. It demonstrates the system but has not been tested with users, and Section 5.6.2 records what it does not do well.

6.3 Threats to Validity

The results in Chapter 5 depend on choices that could change the conclusions if they were wrong. We list them under the four usual headings and say in each case whether we measured the threat or only note it.

6.3.1 Internal Validity: Could Our Own Method Have Produced These Numbers?

The first setup tuned no settings (Section 4.4.1). That removed the usual threat, because no setting was chosen on the test set, but it created the opposite one: the transformer competed with default settings, and with half of each window cut off, against a baseline that has almost no settings to get wrong. This turned out to be the most serious threat in the project, and the re-run in Section 5.3.3 answers it by tuning both models in the same way on a validation split.

The re-run trained on 327 contracts instead of 408 and chose the transformer’s epoch on validation. A control run with the original 256-token input under the same protocol shows that these changes have no measurable effect (0.693 against the original 0.692 without tuning), so the gain we report comes from reading the whole window (Section 5.3.3). One threat remains. The re-run made many choices on the same 81 validation contracts, including up to 41 thresholds per model. With so few contracts some per-category thresholds will fit noise. That makes the validation scores optimistic (0.857 for the tuned ensemble, against 0.809 on the test set) but leaves the test scores fair, because the test set played no part in any choice.

An early test run of the summarizer tracked its loss on test-split clauses (Section 4.4.1). No model was chosen from that run, and the final model was trained again without any evaluation set, so no reported score depends on it. It was still the wrong practice, and the “validation” loss it printed should be ignored. Figure 4.11 now shows the October 2026 re-training, which used no evaluation set.

Some design choices were made after looking at the training data. This is allowed, but the line needs care. The window size and step follow from the median clause length measured on training contracts. The 64% retrieval limit, however, was measured on the whole dataset, test contracts included. That measurement ruled out a design and did not choose a setting, and it was made before any model existed, so we judge the risk of leakage to be negligible. It is not zero, and if we did the work again we would make the split first.

The span model and the summarizer have no seed variation. Only the presence model was re-run with new seeds (Section 5.4.2). The first span model and the summarizer were each trained again with the same seed and gave the same scores to within 0.001, and the retrained span model was trained once, so their seed variance is still unmeasured.

6.3.2 Construct Validity: Do Our Scores Measure What We Claim?

ROUGE-L does not measure summary quality here, and we can show it. Against our 41 templates it gives 0.775. Against clause-specific references it gives 0.116, and a fixed lookup table scores almost the same, 0.115 (Section 5.1.3). We report both numbers because the first one alone would give a false picture. We still have no measure of faithfulness or readability (Section 2.2.6), so even 0.116 does not show that the output keeps the legal meaning.

Micro-F1 does not measure harm to the user, and Section 5.2.2 shows the two can point in opposite directions. The setup that wins on micro-F1 misses 36.9% of High-risk clauses, while a setup that comes last misses 14.2%. Half of the clauses behind the overall score are Low-risk ones such as dates, party names and governing law. Any claim like “this system is 77.9% good” has this problem.

Category-level risk is not clause-level risk (Section 4.2.4). Our levels say that a category usually deserves attention. They say nothing about whether a particular cap is generous or harsh, so calling the output a risk assessment would claim too much, and we call it sorting by risk.

The risk levels are our own. The risk-weighted results use our choice of which categories are High, Medium and Low (Section 4.2.4), and a different choice would change those numbers.

Token-F1 against one marked span assumes that the clause has clear edges. Section 5.1.2 notes that the edges of legal clauses are often debatable, and Section 5.2.3 shows at least one “false negative” that looks like a disagreement with the labels.

6.3.3 External Validity: Where Do These Results Apply?

Every number comes from CUAD, which holds 510 US business contracts taken from SEC filings, in English. We have not tested a single contract from outside that dataset. We therefore have no evidence about other legal traditions or countries, including Bangladeshi contracts, which are the setting that motivated this work. The same goes for consumer and employment contracts, which are built differently and have a different weaker party, for contracts not in English, for contracts much shorter or longer than CUAD’s, and for recent machine-written contracts. The risk table has one more threat of this kind. Whether a non-compete can be enforced depends on the country or state, so a level that is right in one place may be wrong in another. Because CUAD comes from SEC filings, it also leans towards large-company contracts where both sides have lawyers, which is close to the opposite of the small businesses and individuals the system is meant for.

6.3.4 Statistical Conclusion Validity: Can the Intervals Be Trusted?

We measured three sources of variation, and they differ in size. The bootstrap over contracts gives an interval half-width of about 0.008 (Section 5.4.1), the ensemble’s model-seed SD is 0.0015 over three runs (Section 5.4.2), and the split SD is 0.0063 over twenty splits (Section 5.4.4). The bootstrap and the split variation are both larger than the ensemble’s +0.0042 lead, which is why we call it a tie. The split variation is the largest, and we could measure it only for the baseline.

Three seeds is a small sample for estimating a standard deviation, so the uncertainty around 0.0015 is itself large.

Scores for single rare categories are not meaningful. Five categories have at most seven test examples. We report their F1 values for completeness, and no claim in this project depends on any one of them.

We used one split and one dataset. The bootstrap resamples contracts within our test set. It cannot show how results would change on another dataset, and no interval in this project covers that.

6.4 Limitations

Section 6.3 covered the threats to each kind of validity. This section lists the limits of the system itself, meaning what it does not do. Each limit is described where it comes up in the text, so these are pointers.

There has been no usability study (Section 5.6.1), and nobody outside the three of us has used Clausify. The project argues that a report sorted by risk is more useful to a non-lawyer than a plain list of clauses, without evidence: there is no measure of task completion, understanding, trust or over-reliance. Section 5.3.2 shows that the confidence bar is too confident, and Section 5.1.3 shows that the plain-English sentence is the same for every clause of a category, so over-reliance is what a usability study would need to look for.

We used small models (DistilBERT, FLAN-T5-small) to fit a laptop’s computing budget, so our scores are lower than the same method would reach with larger models on CUDA hardware.

Training used one seed and one data budget, which we checked only in part. Section 5.4.3 reports what happened when the presence transformer was trained on all 53,662 window examples instead of a sample of 24,000, and Section 5.4.2 reports the spread across three training seeds. The first span model and the summarizer were each trained twice with the same seed (matching to within 0.001), never with a new seed, and on a sample of the data (8,000 of 11,156 and 4,000 of 11,156); the retrained span model was trained once, on 6,000 clauses. Their seed variance is unmeasured, and their scores should be read with that in mind.

The rarest categories are close to impossible to learn with this much data, and Section 5.2.4 shows that the AND-ensemble makes this worse. On the ten rarest categories the AND rule gains no F1 over the transformer alone, buying precision only with recall, and it scores zero on three categories that the transformer alone was partly finding. One of the rare categories is High-risk in our table, so this limits the real system as well as the ranking.

The span test covers less than the CUAD paper’s test. Section 5.1.2 measures extraction inside windows that contain the answer, and we did not run the heavier full-document precision-at-recall test. Section 5.2.1 closes part of that gap by measuring what gets through the real system, but it uses a different method, and we do not present the two as comparable.

The summarizer does not summarize. Section 5.1.3 shows this with numbers: the fine-tuned model outputs one of 41 pre-written templates word for word for every input, and against clause-specific references it scores no better than the template

alone with no model at all. It is a “find the category, show its template” component and should be described as one.

The risk table is set per category: it flags how risky a category usually is, and it does not judge how harsh the exact wording in front of the user is.

Everything rests on one dataset from one country. Every number in this project comes from CUAD’s 510 US business contracts taken from SEC filings. We have not tested the system on a single contract from outside that dataset, so we have no evidence about how it handles other ways of writing contracts, and none at all about the Bangladeshi setting that motivated the work.

6.5 Recommendations for Future Work

- Watch people use Clausify. Five to ten people would each review a contract with and without the app, and we would measure the time taken, the clauses correctly found and, most of all, whether users trust the output too much. The project claims that a report sorted by risk helps a non-lawyer, and nothing in it tests that claim yet.
- Compare with a large language model. Run a prompted LLM on the same 102 test contracts with the same scores, either an open legal model run locally (such as SaulLM-7B [16]) or a hosted one, and count how often it reports clauses that are not there (Section 2.2.3). It is the comparison a reader is most likely to ask for, and we did not run it.
- Train a model to choose the window. With the retrained span model, the window the presence model chooses is one of the two remaining losses: it misses the clause for 17.4% of detected categories, 206 clauses against the 244 the span model misses in the right window (Section 5.2.1). A model trained to locate the clause, instead of reusing the presence score, would target it, and the span model, which can now say “not here”, could score several candidate windows.
- Train the span model further. The retrained model used 6,000 clauses, one seed and two epochs, and still misses 23.6% of clauses in the right window. Training on all the clauses, and measuring the spread across seeds, are the obvious next steps.
- Test summaries for faithfulness. Word overlap is not enough; BERTScore, a check that the summary follows from the clause, a readability score and a small human rating of whether the legal meaning is kept would all help (Section 2.2.6).
- Learn how to pool windows. The test in Section 5.3.1 tried only fixed rules for combining window scores. A learned rule that weighs windows by relevance (attention pooling) is the untested option most likely to help.
- Use cross-validation. The split is the largest uncertainty we measured (Section 5.4.4), and we could only measure it for the baseline.

- Build a summarizer that really summarizes. Grow the 41 templates into a set of targets that differ from clause to clause, and fine-tune again. The 150-clause trial set in Section 5.1.3 is a working first version of both the labelling method and the test that would show success, and scaling it to a few thousand clauses is the way from picking templates to simplifying clauses.
- Tune for risk directly. The re-run chose its recall-first settings with micro-F2, which treats every category alike. A goal that weights High-risk categories more, chosen on validation in the same way, would aim the thresholds at the clauses that matter most. The mixed rule suggested in Section 5.2.4 was among the candidates in the re-run, and the validation split did not choose it.
- Try meaning-based retrieval. Our 64% retrieval limit (Section 4.2.1) was measured only with TF-IDF matching. A sentence-encoder retriever matches on meaning and might raise that limit enough to make a “retrieve, then read” design competitive. The measurement is cheap, and we should have run it.
- Train a sparse-attention baseline. Section 2.1.2 argues against Longformer partly from arithmetic and partly from cost, and Section 4.4.7 measures the cost, but nobody has trained one on this task. On CUDA hardware a Longformer baseline would be the fair comparison that our argument currently replaces with reasoning.
- Use larger models. Re-running the same system with DeBERTa-v3-base or large and FLAN-T5-base on CUDA hardware would close the gap to the published CUAD results.
- Run the full-document span test. Building the ranked whole-document test would let us report precision at 80% recall and compare directly with the CUAD paper.
- Find conflicts between clauses. The rule engine designed in an earlier phase of the project (for example, *Exclusivity* against *Most Favored Nation*) could be brought into the app.
- Give risk to each clause. Moving from risk per category to risk per clause means reading the extracted text itself, such as the amount of a cap or the length of a non-compete.
- Test contracts from outside CUAD. Even three local English-language business contracts, run through the system and checked by hand, would say more about how it generalizes than any further tuning on CUAD, and they are the first step towards the Bangladeshi use this work was aimed at.

Bibliography

- [1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An expert-annotated NLP dataset for legal contract review,” in *Proc. 35th Conf. Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [3] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” in *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (NeurIPS)*, 2019.
- [6] C. Raffel, N. Shazeer, A. Roberts, *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [7] H. W. Chung *et al.*, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
- [8] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out (ACL Workshop)*, 2004, pp. 74–81.
- [9] The Atticus Project, *Contract understanding atticus dataset (CUAD) v1*, 2021. [Online]. Available: <https://www.atticusprojectai.org/cuad>.
- [10] Y. Koreeda and C. D. Manning, “ContractNLI: A dataset for document-level natural language inference for contracts,” in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 1907–1919.
- [11] S. H. Wang, A. Scardigli, L. Tang, *et al.*, “MAUD: An expert-annotated legal NLP dataset for merger agreement understanding,” in *Proc. EMNLP*, 2023, pp. 16 369–16 382.
- [12] N. Guha, J. Nyarko, D. E. Ho, C. Ré, *et al.*, “LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models,” in *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

- [13] N. Pipitone and G. Houir Alami, “LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain,” *arXiv preprint arXiv:2408.10343*, 2024.
- [14] D. M. Katz, M. J. Bommarito, S. Gao, and P. Arredondo, “GPT-4 passes the bar exam,” *Philosophical Transactions of the Royal Society A*, vol. 382, no. 2270, p. 20 230 254, 2024.
- [15] L. Martin, N. Whitehouse, S. Yiu, L. Catterson, and R. Perera, “Better call GPT, comparing large language models against lawyers,” *arXiv preprint arXiv:2401.16212*, 2024.
- [16] P. Colombo *et al.*, “SaulLM-7B: A pioneering large language model for law,” *arXiv preprint arXiv:2403.03883*, 2024.
- [17] I. Chalkidis, “ChatGPT may pass the bar exam soon, but has a long way to go for the LexGLUE benchmark,” *arXiv preprint arXiv:2304.12202*, 2023.
- [18] I. Chalkidis, A. Jana, D. Hartung, *et al.*, “LexGLUE: A benchmark dataset for legal language understanding in English,” in *Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [19] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, “Large legal fictions: Profiling legal hallucinations in large language models,” *Journal of Legal Analysis*, vol. 16, no. 1, pp. 64–93, 2024.
- [20] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, “Hallucination-free? assessing the reliability of leading AI legal research tools,” *Journal of Empirical Legal Studies*, vol. 22, no. 2, pp. 216–242, 2025.
- [21] N. F. Liu, K. Lin, J. Hewitt, *et al.*, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [22] M. Lippi, P. Pałka, G. Contissa, *et al.*, “CLAUDETTE: An automated detector of potentially unfair clauses in online terms of service,” *Artificial Intelligence and Law*, vol. 27, no. 2, pp. 117–139, 2019.
- [23] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The muppets straight out of law school,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 2898–2904.
- [24] L. Manor and J. J. Li, “Plain English summarization of contracts,” in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2019, pp. 1–11.
- [25] R. Flesch, “A new readability yardstick,” *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.
- [26] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On faithfulness and factuality in abstractive summarization,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1906–1919.
- [27] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9332–9346.

- [28] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Int. Conf. Learning Representations (ICLR)*, 2020.
- [29] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [30] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [31] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proc. ACL*, 2018, pp. 784–789.
- [32] O. Maron and T. Lozano-Pérez, “A framework for multiple-instance learning,” in *Advances in Neural Information Processing Systems 10 (NIPS)*, 1998, pp. 570–576.
- [33] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

Appendix A

Full Risk Taxonomy

The full risk table for all 41 categories (8 High, 22 Medium, 11 Low), judged from the weaker party's side, with the one-line reason that the Clausify app shows to users.

Table A.1: Full risk taxonomy

Category	Risk	Reason
Non-Compete	High	Limits your future income and freedom to work.
Uncapped Liability	High	Unlimited damages: no ceiling on what you can owe.
IP Ownership	High	You lose ownership of a core asset for good.
Assignment	High	Locks you to a single source or buyer.
Exclusivity	High	You must always give the other side your best terms.
Most Favored Nation	High	Fixed penalties trigger automatically on breach.
Liquidated Damages	High	A grant you can never take back.
Irrevocable or Perpetual License	High	You are forced to buy or deliver a minimum amount.
Minimum Commitment	High	
Renewal Term	Medium	Auto-renewal can extend the contract unexpectedly.
Notice Period to Terminate	Medium	Miss the window and you are locked in for another term.
Renewal Exception	Medium	Carve-outs to competitive limits: read carefully.
Competitive Restriction	Medium	Restricts approaching the other side's customers.
No-Solicit of Customers	Medium	Restricts hiring the other side's staff.
No-Solicit of Employees	Medium	Limits what you can publicly say.
Non-Disparagement	Medium	The other side can walk away with notice.
Termination for Convenience	Medium	

Category	Risk	Reason
ROFR / ROFO / ROFN	Medium	First-refusal rights can constrain your options.
Change of Control	Medium	A sale or merger may trigger termination.
Anti-Assignment	Medium	You cannot transfer the contract without consent.
Revenue/Profit Sharing	Medium	You must share a slice of revenue or profit.
Price Restrictions	Medium	Limits what prices you may set or change.
Volume Restriction	Medium	Caps how much you can sell or distribute.
Joint IP Ownership	Medium	IP is co-owned: control is shared, not sole.
Non-Transferable License	Medium	The licence cannot be passed on.
Affiliate License (Licensor)	Medium	Licensor's affiliates are pulled into the grant.
Unlimited / All-You-Can-Eat License	Medium	Broad, unlimited licence scope: check who benefits.
Source Code Escrow	Medium	Source is held by a third party under conditions.
Post-Termination Services	Medium	Obligations continue after the contract ends.
Audit Rights	Medium	The other side can inspect your records.
Cap on Liability	Medium	Caps recovery: may under-compensate real losses.
Covenant Not to Sue	Medium	You waive the right to sue over listed matters.
Document Name	Low	Identifies the agreement: informational.
Parties	Low	Names who is signing: informational.
Agreement Date	Low	The signing date: informational.
Effective Date	Low	When terms start: informational.
Expiration Date	Low	When the contract ends: informational.
Governing Law	Low	Which jurisdiction's law applies: neutral.
License Grant	Low	You receive usage rights: generally favorable.
Warranty Duration	Low	How long the warranty protects you.
Insurance	Low	Required coverage: a protection.
Affiliate License (Licensee)	Low	Extends rights to your affiliates: favorable.
Third Party Beneficiary	Low	Names an outside beneficiary: informational.

Appendix B

Reproducibility Checklist

This appendix describes the setup that produced every number in this report, so that a reader with the repository and this appendix can produce any table again without asking us.

Environment

Python	3.13.5
Operating system	macOS 26.5.2 (August 2026 runs) and 26.6.2 (October 2026 runs), arm64
Hardware	Apple M4, 10 cores (4 performance / 6 efficiency), 16 GB unified memory
Accelerator	Metal Performance Shaders (MPS); no CUDA at any point
<code>torch</code>	2.8.0
<code>transformers</code>	5.12.1
<code>datasets</code>	5.0.0
<code>scikit-learn</code>	1.6.1
<code>numpy</code>	2.1.3
<code>pandas</code>	2.2.3
<code>rouge_score</code>	installed (ROUGE-L is also implemented inline; see below)
<code>matplotlib</code>	figures only

Data

Dataset	CUAD v1, Hugging Face <code>theatticusproject/cuad</code>
Files used	<code>CUAD_v1/CUAD_v1.json</code> (contexts, character offsets); <code>CUAD_v1/master_clauses.csv</code> (exploratory analysis only)
Contracts	510; 41 categories; 20,910 (contract, category) rows
Note	The default <code>load_dataset()</code> option gives only the original PDFs, which cannot be used for training (Section 4.3).

Seeds and splits

- Seed 42 for the split, all three trainings and every bootstrap.
- The split is *made once* with `numpy.random.default_rng(42).permutation`, saved to `artifacts/split.json`, and read from disk by every later script, so no script makes its own split.
- Extra presence-model seeds 43 and 44 (Section 5.4.2); split seeds 0–19 for Section 5.4.4.
- Bootstrap: 10,000 resamples, `default_rng(42)`, resampling whole contracts.
- Re-run validation split (Section 5.3.3): 81 of the 408 training contracts, drawn with `default_rng(42).permutation` over the training titles and saved as `retrain/v2_fullwindow/runs/<tag>/val_split.json`.

Artifact fingerprints

The first 16 characters of the SHA-256 hash of each file (October 2026 models, and the re-run presence model, TF-IDF model and retrained span model the app uses), so a reader can check that they have the same files:

<code>CUAD_v1.json</code>	<code>ed0b77d85bdf4014</code>
<code>artifacts/split.json</code>	<code>177e2ef48dfab9cd</code>
<code>artifacts/baseline.pkl</code>	<code>903e527500409af2</code>
<code>outputs/presence_mil/final/model.safetensors</code>	<code>a5482a0302ae2c7c</code>
<code>outputs/span/final/model.safetensors</code>	<code>380a233a3c03e052</code>
<code>outputs/summarizer/final/model.safetensors</code>	<code>9fcb6bdf777d033c</code>
<code>outputs/presence_v2/final/model.safetensors</code>	<code>e5f84bd42eb8bf27</code>
<code>artifacts/baseline_v2.pkl</code>	<code>3fc0d273065d0a11</code>
<code>outputs/span_v2/final/model.safetensors</code>	<code>20ac289e746e84fb</code>
<code>outputs/clause_v2/tfidf.pkl</code>	<code>9d288ab0c8231f95</code>

Commands, and which table each regenerates

The four stage scripts are run from the `notebooks/` folder, exactly as listed in `retrain/README.md`; every other command works from the project folder. Rows marked “August” are the August 2026 control runs.

Command	Regenerates
<code>python3 ../retrain/smoke_test.py</code>	Checks that the four saved models load and run
<code>python3 ../retrain/stage2a_presence_train.py</code>	Split, TF-IDF baseline and presence transformer (≈ 50 min)
<code>python3 ../retrain/stage2a_presence_test.py</code>	Tables 5.1–5.4, Figures 5.1–5.3 (≈ 50 min)
<code>python3 ../retrain/stage2b_span_train.py</code> , then <code>_test.py</code>	Span model and its scores in Table 5.5 ($\approx 25 + 2$ min)
<code>python3 ../retrain/stage1_summarizer_train.py</code> , then <code>_test.py</code>	Summarizer and its template score in Table 5.6 ($\approx 13 + 3$ min)
<code>python3 retrain/analysis_oct2026/make_probs.py</code>	Saves the presence scores in the form the analysis scripts below read (run once after the presence test)
<code>python3 retrain/analysis_oct2026/span_ci.py</code>	Intervals in Table 5.5
<code>python3 retrain/analysis_oct2026/summ_pilot.py</code>	Tables 5.6 and 5.7
<code>python3 retrain/analysis_oct2026/e2e.py</code>	Tables 5.8 and 5.9
<code>python3 retrain/analysis_oct2026/risk_threshold_calib.py</code>	Table 5.11, Figure 5.4
<code>python3 retrain/analysis_oct2026/error_examples.py</code>	Table 5.13
<code>python3 retrain/analysis_oct2026/rare_tail.py</code>	Table 5.14
<code>python3 retrain/analysis_oct2026/pooling_ablation.py</code>	Table 5.16 (≈ 55 min)
<code>python3 retrain/analysis_oct2026/bootstrap_ci.py</code>	Table 5.19
<code>python3 analysis/presence_run.py --tag s43 --seed 43</code>	Seeds 43 and 44 in Table 5.20 (August; repeat for 44)
<code>python3 analysis/presence_run.py --tag full53k --max-train 0</code>	Table 5.21 (August; ≈ 140 min)
<code>python3 analysis/compare_runs.py</code>	Differences in Table 5.21 (August)
<code>python3 analysis/split_variance.py</code>	Table 5.22
<code>python3 analysis/lf_one.py longformer 4096 1 train</code>	Table 4.5 (one row per run)
<code>caffeinate -i python3 -u ../retrain/v2_fullwindow/presence_v2.py --max-len 512 --tag L512</code>	Re-run presence model and its window scores for Section 5.3.3 (≈ 6.5 h)
<code>caffeinate -i python3 -u ../retrain/v2_fullwindow/presence_v2.py --max-len 256 --tag L256</code>	256-token control run (≈ 3.2 h)
<code>python3 ../retrain/v2_fullwindow/tune_and_test.py --tag L512 (then L256)</code>	Tables 5.17 and 5.18; the app’s thresholds (<code>decision.json</code>) and TF-IDF model
<code>python3 retrain/v2_fullwindow/compare_window.py</code>	Paired 512 vs. 256 differences in Section 5.3.3
<code>python3 retrain/v2_fullwindow/measure_truncation.py</code>	How much of each window the 256-token model reads (Section 5.3.3)

Command	Regenerates
<code>caffeinate -i python3 -u retrain/span_v2/span_chunks.py</code> <code>python3 retrain/span_v2/evaluate_span.py</code>	Retrained span model of Section 5.2.1 (≈ 46 min) Validation choices and Table 5.10, including the app’s quote; the settings in the app’s <code>span_v2.json</code> (≈ 20 min)
<code>python3 retrain/v2_fullwindow/card_chance.py</code>	Cards under 50% and 20% chance (Section 5.6.2)
<code>python3 retrain/v2_fullwindow/window_cap.py</code>	Reading rules for long contracts, number of extra windows chosen on validation (Table 5.24)
<code>python3 retrain/v2_fullwindow/app_presence_test.py</code> <code>python3 retrain/clause_v2/</code> <code>clause_classifier.py tfidf (then bert, then test)</code>	The app’s own detection on the test contracts (≈ 85 min on MPS) Clause-mode classifier, chosen on validation (Section 4.4.9)
<code>python3 retrain/plain_v2/plain_english.py</code> (then <code>plain_fewshot.py</code>) <code>python3 retrain/analysis_oct2026/app_bench.py</code> <code>python3 retrain/verify_report.py</code>	Larger FLAN-T5 models for the plain-English line (Section 5.1.3) Table 4.6 Checks the reported numbers in this report against the result files; prints 0 <code>mismatched</code> when they agree
<code>tectonic project_report/main.tex</code>	This report

Known limits on exact repeatability

- Training on MPS does not give exactly the same weights every time, even on the same machine with the same library versions. Re-training every model in October 2026 with the same code and seed gave the August scores to within about 0.003 (transformer micro-F1 0.6943 $\rightarrow$ 0.6924; span token-F1 0.763 $\rightarrow$ 0.764), but not exactly.
- ROUGE-L in `summ_pilot.py` is computed by our own longest-common-subsequence code (the same as in `test.ipynb`), not by the `rouge_score` package, so scores from the two should not be mixed.
- The retrieval limit in Section 4.2.1 was measured over all 510 contracts, test contracts included (Section 6.3).

Appendix C

Repository and Artifact Layout

```
clausify-p3/ (github.com/afnanmz168/clausify-p3) -- research deliverable
|- project_report/ -- this report (LaTeX, CSE400 template)
| |- main.tex / main.pdf
| |- chapters/ core/ appendix/ bibliography/ images/
|- paper/ -- IEEE-format paper (LaTeX + figures)
|- IEEE_Paper_Clausify.pdf
|- data/
| |- train_80.csv -- 80% split (16,728 rows, 408 contracts)
| |- test_20.csv -- 20% split (4,182 rows, 102 contracts)
|- notebooks/
| |- train.ipynb -- training notebook (EDA -> 3 models)
| |- test.ipynb -- STANDALONE evaluation notebook
| |- build_artifacts.py -- retrains & saves all artifacts
| |- artifacts/ -- split.json, baseline.pkl, baseline_v2.pkl
| |- outputs/ -- presence (and presence_v2), span (and span_v2), summarizer
|- retrain/ -- October 2026 re-training & re-testing
| |- README.md -- exact terminal commands
| |- stage2a_* stage2b_* stage1_* -- train/test scripts, logs, results
| |- smoke_test.py, make_figures.py
| |- analysis_oct2026/ -- analyses re-run on the October models
| |- v2_fullwindow/ -- 512-token re-run and 256-token control
| |- span_v2/ -- span model retrained on the chunks it reads
| |- clause_v2/ -- clause-mode classifier (41 types + none)
| |- plain_v2/ -- larger FLAN-T5 models for the plain-English line
| |- verify_report.py -- checks report numbers against results
|- analysis/
| |- bootstrap_ci.py -- cluster bootstrap, presence models
| |- span_ci.py -- cluster bootstrap, span extraction
| |- rare_tail.py -- rare-category AND-rule breakdown
| |- e2e.py -- end-to-end pipeline evaluation
| |- summ_pilot.py -- two-reference-set summarizer pilot
| |- presence_run.py -- presence training at any data budget
| |- longformer_bench.py -- sparse-attention cost benchmark
| |- diverse_refs.json -- 150 clause-specific pilot references
|- demo_clause.txt -- 18-clause demonstration contract

clausify-p3-app/ (github.com/afnanmz168/clausify-p3-app)
|- app.py -- Streamlit UI
|- model_utils.py -- inference: all four models, clause mode
|- features.py -- file upload, highlighting, reports
|- calibrate_clause_mode.py -- clause-mode calibration (+ .json)
|- cat_questions.json -- 41 category->question strings
|- sample_contract.txt / test_contract.txt
|- decision_v2.json -- tested rules and per-category thresholds
|- tests/test_app.py -- 43 end-to-end checks
```